

Arm[®] CoreSight[™] Architecture Specification

Document number	IHI0029
Document quality	EAC
Document version	G
Document confidentiality	Non-Confidential
Date of issue	30 March 2026



Release information

The following releases of this document have been made.

Date	Version	Changes
30/Mar/2026	G	• Non-Confidential. Third release for v3.0.
25/Feb/2022	F	• Non-Confidential. Second release for v3.0.
27/Feb/2017	E	• Non-Confidential. First release for v3.0.
26/Sep/2013	D	• Non-Confidential. First release for v2.0.
27/Mar/2012	C	• Confidential. Limited beta release for v2.0.
24/Mar/2005	B	• Non-Confidential. Second release for v1.0. Editorial changes and clarifications.
29/Sep/2004	A	• Non-Confidential. First release for v1.0.

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited (“Arm”). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm’s view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED “AS IS”. ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party’s products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2004, 2005, 2012, 2013, 2017, 2022, 2026 Arm Limited or its affiliates. All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-20349

8 March 2024

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

The information in this manual is at EAC quality, which means that all features of the specification are described in the manual.

Web Address

<https://www.arm.com>

Contents

Arm CoreSight Architecture Specification

Preface

About this document	9
Using this document.....	10
Conventions	12
Additional reading.....	14
Feedback	15

Part A CoreSight Architecture

Chapter A1 About the CoreSight Architecture

A1.1	Purpose of the CoreSight architecture.....	18
A1.2	Structure of the CoreSight architecture.....	19
A1.3	CoreSight component types	21
A1.4	CoreSight topology detection.....	23
A1.5	Forms of the designer identification codes.....	25

Part B CoreSight Visible Component Architecture

Chapter B1 About the Visible Component Architecture

Chapter B2 CoreSight Programmers' Model

B2.1	About the programmers' model	30
B2.2	Component and Peripheral Identification Registers	36
B2.3	Component-specific registers for Class 0x9 CoreSight components.....	46
B2.4	Component-specific registers for Class 0xF CoreLink, PrimeCell, and system components	70

Chapter B3 Topology Detection

B3.1	About topology detection	72
B3.2	Requirements for topology detection signals	73
B3.3	Combination with integration registers.....	74
B3.4	Interfaces that are not connected or implemented	75
B3.5	Variant interfaces	76
B3.6	Documentation requirements for topology detection registers	77

Part C CoreSight Reusable Component Architecture

Chapter C1	About the Reusable Component Architecture	
Chapter C2	AMBA APB and ATB Interfaces	
C2.1	AMBA APB interface	81
C2.2	AMBA ATB interface	83
Chapter C3	Event Interface	
Chapter C4	Channel interface	
C4.1	About the channel interface	86
C4.2	Channels	88
C4.3	Channel interface signals	89
C4.4	Channel connections	90
C4.5	Synchronous and asynchronous conversions	91
Chapter C5	Authentication Interface	
C5.1	About the authentication interface	93
C5.2	Definitions of different types of debug	94
C5.3	Authentication interface signals	95
C5.4	Authentication rules	96
C5.5	User mode debugging	102
C5.6	Control of the authentication interface	103
C5.7	Exemptions from implementing the authentication interface	104
Chapter C6	Timestamp Interface	
Chapter C7	Topology Detection at the Component Level	
C7.1	About topology detection at the component level	107
C7.2	Interface types for topology detection	108
C7.3	Interface requirements for topology detection	110
C7.4	Signals for topology detection	111

Part D CoreSight System Architecture

Chapter D1	About the System Architecture	
Chapter D2	System Considerations	
D2.1	Clock and power domains	115
D2.2	Control of authentication interfaces	116
D2.3	Memory system design	117

Chapter D3	Physical Interface	
D3.1	Arm JTAG 20	121
D3.2	CoreSight 10 and CoreSight 20 connectors	123
D3.3	Arm MICTOR	127
D3.4	Target Connector Signal details	133
Chapter D4	Trace Formatter	
D4.1	About trace formatters	137
D4.2	Frame descriptions	138
D4.3	Modes of operation	143
D4.4	Flush of trace data at the end of operation	144
Chapter D5	About ROM Tables	
D5.1	ROM Tables Overview	146
D5.2	ROM Table Types	147
D5.3	Component and Peripheral ID Registers for ROM Tables	148
Chapter D6	Topology Detection at the System Level	
D6.1	About topology detection at the system level	150
D6.2	Detection	151
D6.3	Components that are not recognized	152
D6.4	Detection algorithm	153
Chapter D7	Compliance Requirements	
D7.1	About compliance classes	155
D7.2	CoreSight debug	156
D7.3	CoreSight trace	159
D7.4	Multiple DPs	163
Part E	Appendixes	
Chapter E1	Power Requester	
E1.1	About the power requester	167
E1.2	Register descriptions	168
E1.3	Powering non-visible components	186
Chapter E2	Revisions	

Preface

This preface introduces the *Arm[®] CoreSight[™] Architecture Specification*. It contains the following sections:

- [About this document.](#)
- [Using this document.](#)
- [Conventions.](#)
- [Additional reading.](#)
- [Feedback.](#)

About this document

This document describes the CoreSight architecture that all versions of the CoreSight compliant cores, components, platforms, and systems use.

Intended audience

This specification targets the following audiences:

- Hardware engineers integrating CoreSight components into a CoreSight system.
- Hardware engineers designing CoreSight components.
- Software engineers writing development tools that support CoreSight system functionality.
- Designers of debug hardware that is used to connect to a CoreSight system, for example JTAG emulators, SWD emulators, and Trace Port Analyzers.
- Advanced designers of development tools that support CoreSight functionality.

This specification does not document the behavior of individual components unless they form a fundamental part of the architecture.

Arm recommends that engineers who use this document have experience of the Arm architecture.

Using this document

This document is organized into the following parts:

Part A, CoreSight Architecture

Part A contains an introduction to the CoreSight architecture. It contains the following chapter:

About the CoreSight Architecture

Read this chapter for an outline description of the components, memory maps, clock and reset requirements, system integration, and the test interface.

Part B, CoreSight Visible Component Architecture

Part B describes the CoreSight visible component architecture, which must be implemented by all CoreSight components that are visible to a debugger. It contains the following chapters:

About the Visible Component Architecture

Read this chapter for a description of the visible component architecture.

CoreSight programmers' model

Read this chapter for a description of the CoreSight technology programmers' model.

Topology Detection

Read this chapter for a description of the topology detection registers in CoreSight systems.

Part C, CoreSight Reusable Component Architecture

Part C describes the CoreSight reusable component architecture, which must be implemented by CoreSight components so that they can be used with other CoreSight components. It contains the following chapters:

About the Reusable Component Architecture

Read this chapter for a description of the reusable component architecture.

AMBA APB and ATB Interfaces

Read this chapter for a description of the AMBA[®] APB interface and the AMBA ATB interface.

Event Interface

Read this chapter for a description of the event interface.

Channel interface

Read this chapter for a description of the channel interface.

Authentication Interface

Read this chapter for a description of the authentication interface.

Timestamp Interface

Read this chapter for a description of the timestamp interface.

Topology Detection at the Component Level

Read this chapter for a description of topology detection at the component level.

Part D, CoreSight System Architecture

Part D describes the CoreSight system architecture, which must be implemented by all CoreSight systems and provides information that is required by debuggers to enable them to use CoreSight systems. It contains the following chapters:

About the System Architecture

Read this chapter for a description of the CoreSight system architecture.

System Considerations

Read this chapter for a description of system design with the CoreSight system architecture.

Physical Interface

Read this chapter for a description of the physical interface for CoreSight connection to a debugger.

Trace Formatter

Read this chapter for a description of the CoreSight trace formatter.

About ROM Tables

Read this chapter for a general description of CoreSight ROM Tables.

Topology Detection at the System Level

Read this chapter for a description of topology detection at the system level.

Compliance Requirements

Read this chapter for a description of the criteria that systems must comply with to satisfy CoreSight requirements.

Part E, Appendixes

This specification contains the following appendixes:

Power Requester

Read this chapter for a description of the power requester.

Revisions

Read this chapter for a description of the technical changes between released versions of this specification.

Glossary

Read this chapter for definitions of some terms that are used in this specification. The glossary does not contain terms that are industry standard unless the Arm meaning differs from the accepted meaning.

Conventions

The following sections describe conventions that this document can use:

- [Typographic conventions.](#)
- [Signals.](#)
- [Timing diagrams.](#)
- [Numbers.](#)

Typographic conventions

The typographical conventions are:

<i>italic</i>	Introduces special terminology, and denotes internal cross-references and citations, or highlights an important note.
bold	Denotes signal names, and is used for terms in descriptive lists, where appropriate.
<code>monospace</code>	Used for assembler syntax descriptions, pseudocode, and source code examples. Also used in the main text for instruction mnemonics and for references to other items appearing in assembler syntax descriptions, pseudocode, and source code examples.
small capitals	 SMALL CAPITALS are used for a few terms that have specific technical meanings, and are included in the Glossary.
Colored text	• Indicates a link, for example: https://developer.arm.com. • A link, to a chapter or appendix, or to a glossary entry, or to the section of the document that defines the colored term, for example Embedded Trace Buffer (ETB) .

Signals

In general, this document does not define processor signals, but it does include some signal examples and recommendations. The signal conventions are:

Signal level	The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means: • HIGH for active-HIGH signals. • LOW for active-LOW signals.
Lowercase n	At the start or end of a signal name denotes an active-LOW signal.

Timing diagrams

The figure, [Figure 0-1](#), explains the components that are used in timing diagrams. Variations, when they occur, have clear labels. Do not assume any timing information that is not explicit in the diagrams.

Shaded bus and signal areas are undefined so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

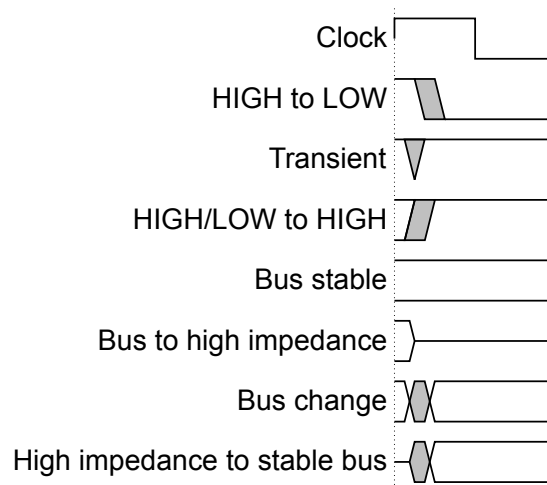


Figure 0-1 Key to timing diagram conventions

Timing diagrams sometimes show single-bit signals as HIGH and LOW at the same time and they look similar to the bus change shown in [Figure 0-1](#). If a timing diagram shows a single-bit signal in this way, its value does not affect the accompanying description.

Numbers

Numbers are normally written in decimal notation. Binary numbers are preceded by `0b`, and hexadecimal numbers by `0x`. In both cases, the prefix and the associated value are written in a `monospace` font, for example `0xFFFF0000`.

Pseudocode descriptions

This document uses a form of pseudocode to provide precise descriptions of the specified functionality.

Additional reading

This section lists relevant publications from Arm and third parties.

See <https://developer.arm.com>, for access to Arm documentation.

Arm publications

This specification contains information that is specific to CoreSight. See the following documents for other relevant information:

- *Arm® CoreSight™ SoC-400 Technical Reference Manual* (ARM 100536).
- *Arm® CoreSight™ Technology System Design Guide* (ARM DGI 0012).
- *CoreSight™ Program Flow Trace Architecture Specification, PFTv1.0 and PFTv1.1* (ARM IHI 0035).
- *Arm® Embedded Trace Macrocell Architecture Specification, ETM v1.0 to ETMv3.5* (ARM IHI 0014).
- *Arm® Embedded Trace Macrocell Architecture Specification ETMv4.0 to ETM4.6* (ARM IHI 0064).
- *Arm® Debug Interface Architecture Specification v6* (ARM IHI 0074).
- *Arm® Debug Interface Architecture Specification v5* (ARM IHI 0031).
- *Arm® Architecture Reference Manual ARMv7-A and ARMv7-R edition* (ARM DDI 0406).
- *Arm® Architecture Reference Manual, for A-profile architecture* (ARM DDI 0487).
- *Arm® Realm Management Extension (RME) System Architecture* (ARM DEN 0129).
- *Arm® AMBA® AXI and ACE Protocol Specification* (ARM IHI 0022).
- *Arm® AMBA® APB Protocol Specification* (ARM IHI 0024).
- *Arm® AMBA® ATB Protocol Specification* (ARM IHI 0032).

Other publications

This section lists relevant documents that are published by third parties:

- IEEE, *Standard Test Access Port and Boundary Scan Architecture*, IEEE Std 1149.1-2013
- JEDEC, *Standard Manufacturer's Identification Code*, JEP106

Feedback

Arm welcomes feedback on its documentation.

Feedback on this book

If you have any comments or queries about this document, create a ticket at <https://support.developer.arm.com>. Give:

- The title, Arm CoreSight Architecture Specification.
- The number, IHI0029 G.
- The page numbers to which your comments apply.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.

Note

Arm tests PDFs only in Adobe Acrobat and Acrobat Reader, and cannot guarantee the appearance or behavior of any document when viewed with any other PDF reader.

Progressive terminology commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive.

Arm strives to lead the industry and create change.

Previous issues of this document included terms that can be offensive. We have replaced these terms. If you find offensive terms in this document, please contact terms@arm.com.

Part A **CoreSight Architecture**

Chapter A1

About the CoreSight Architecture

This chapter introduces the CoreSight architecture. It contains the following sections:

- [Purpose of the CoreSight architecture.](#)
- [Structure of the CoreSight architecture.](#)
- [CoreSight component types.](#)
- [CoreSight topology detection.](#)
- [Forms of the designer identification codes.](#)

A1.1 Purpose of the CoreSight architecture

The CoreSight architecture provides a system-wide solution for real-time debug and collecting trace information. It addresses the following:

- The requirement for multi-core debug and trace.
- The requirement to debug and trace system components beyond the core, for example buses.
- The requirement for sharing resources, such as pins and trace storage, between debug and trace components, to reduce silicon costs.
- The requirement for debug and trace components from multiple vendors to be able to work together.
- The requirement for minimizing pin count.
- The requirement for supporting increasing trace bandwidth from many sources.
- The requirement to accommodate the existing trace solutions, rather than expecting them to be rewritten to support a new trace architecture.
- The requirement for development tools to automatically identify and configure themselves for different systems.
- The requirement for controlling access to debug and trace functionality in the field.
- The fact that the clock and power to parts of the system can be varied or disabled independently while debugging the rest of the system.
- The fact that the time available to design debug and trace functionality is often limited and the number of options must be minimized where possible.
- The requirement for debug monitors and other on-chip debug software to have access to the same debug and trace functionality as an external debugger.
- The fact that systems are often built from a hierarchy of reusable platforms, where each level must hide its internal complexities, which prevents designers from changing the platform when using it in another system.

The CoreSight architecture satisfies the following requirements for debug and trace:

- To access debug functionality without software interaction.
- To connect to a running system without performing a reset.
- To perform certain operations, such as real-time tracing, non-invasively, with no effect on the behavior of the system.
- To access noninvasive functionality non-invasively.
- To minimize power consumption of debug logic when it is not in use.
- To capture trace over a large period.

The CoreSight architecture can be used to design CoreSight components, that can be combined with other CoreSight components and processors that comply with the CoreSight architecture to make up a CoreSight system.

A1.2 Structure of the CoreSight architecture

The CoreSight architecture comprises:

- [Visible component architecture](#).
- [Reusable component architecture](#).
- [System architecture](#).

A1.2.1 Visible component architecture

The CoreSight visible component architecture specifies what a component must implement to comply with the CoreSight architecture.

The visible component architecture is visible to the programming interface and to tools that access the device. All CoreSight components must comply with the visible component architecture. The visible component architecture specifies:

- The requirements that all CoreSight components must conform to the programmers' model.
- The requirements that enable discovery of the component layout for topology detection.

For details of the visible component architecture, see [CoreSight Visible Component Architecture](#).

A1.2.2 Reusable component architecture

The CoreSight reusable component architecture specifies the rules that the implementation of the physical interface of a CoreSight component must follow to work correctly with other reusable CoreSight components.

The reusable component architecture specifies:

- The AMBA APB interface for access to the registers in CoreSight components.
- The AMBA ATB interface for trace data transfer between CoreSight components.
- The channel interface for the communication of trigger events between CoreSight components.
- The authentication interface for control of access for debug.
- Topology detection infrastructure that specifies the signals that must be controlled at each interface.

It is possible to create a homogeneous component that performs various functions internally as separate components implementing the visible component architecture, provided one set of reusable component interfaces is available to enable integration into a larger CoreSight system.

Note

Self-contained systems that implement only the visible, and not the reusable component architecture, are compatible with development tools, but have the following limitations:

- Integrating them with other CoreSight components might be impossible.
 - Detecting them during topology detection might fail.
-

For details about the reusable component architecture, see [CoreSight Reusable Component Architecture](#).

A1.2.3 System architecture

The following CoreSight architectural requirements ensure seamless integration of elements that comply with the CoreSight architecture:

- System-level requirements for:
 - Clock and power domains visible to debuggers.
 - Control of the authentication interface.

- Distinction between external and internal accesses through the AMBA APB interface memory map.
- The requirements for the physical interface to the debugger.
- The format that is used by the trace formatter. See [CoreSight component types](#) and [Trace Formatter](#).
- The design of the ROM Table. See [CoreSight component types](#) and [About ROM Tables](#).
- How to enable CoreSight topology detection.
- Compliance requirements for CoreSight systems.

For details about the system architecture, see [CoreSight System Architecture](#).

A1.3 CoreSight component types

The CoreSight architecture specifies a set of components for implementing specific SoC subsystems that support collection of debug and trace information. This section shows some example implementations of CoreSight components that are based on the CoreSight architecture.

Note

A CoreSight component is a component that implements the CoreSight visible component architecture.

The main elements are:

Control components

CoreSight systems can include *Embedded Cross Trigger* (ECT) control components. The ECT includes:

- *Cross Trigger Interface* (CTI).
- *Cross Trigger Matrix* (CTM).

Trace sources

CoreSight systems can include the following trace sources:

- *Embedded Trace Macrocells* (ETMs).
- *AMBA Trace Macrocells*.
- *Program Flow Trace Macrocells* (PTMs).
- *System Trace Macrocells* (STMs).

Trace links

CoreSight systems can include the following trace links:

- Trace funnels.
- Replicators.
- ATB bridges.

Trace sinks

CoreSight systems can include the following trace sinks:

- *Trace Port Interface Units* (TPIUs).
- *Embedded Trace Buffers* (ETBs).
- *Trace Memory Controllers* (TMCs).

Each trace sink can include a Trace Formatter.

Debug Ports (DPs) and Access Ports (APs)

DPs and APs provide access to CoreSight components and other system features. DPs and APs are described by the Arm Debug Interface (ADI) Architecture Specification, see *Arm® Debug Interface Architecture Specification ADIv5.0-5.2* and *Arm® Debug Interface Architecture Specification ADIv6.0*. A DP provides a mechanism that is specific to a wire protocol, and enables access to various components, including APs. Some examples of common DPs are:

- A *Serial Wire Debug Port* (SW-DP).
- A *JTAG Debug Port* (JTAG-DP).
- A *Serial Wire JTAG Debug Port* (SWJ-DP).

An AP provides a mechanism to access buses or other protocols, in particular to access CoreSight components. Some examples of common APs are:

- An *APB Access Port* (APB-AP).
- An *AHB Access Port* (AHB-AP).
- An *AXI Access Port* (AXI-AP).

- A *JTAG Access Port* (JTAG-AP).

For more information on specific components, see the appropriate Technical Reference Manual.

A1.4 CoreSight topology detection

Depending on the requirements of the system, CoreSight components can be connected together in many different ways. Debuggers use a process that is called topology detection to detect the component connections. The infrastructure for topology detection is reflected at each of the following levels:

- A visible component architecture.
- A reusable component architecture.
- The system design.

CoreSight systems can have several interface types, as Transmitters or Receivers. Each CoreSight component specifies which interfaces are present. The debugger probes each interface to determine which other components are connected to it.

Each interface type defines which signals must be controllable by the Transmitter and Receiver interfaces, and how the debugger can determine the connectivity using these signals. These signals are referred to as topology detection signals. For the specification of the requirements for standard interfaces that are used by Arm CoreSight components, see [Topology Detection at the Component Level](#). Interface vendors must define the requirements for other interfaces, following the rules in [Topology Detection at the System Level](#).

A1.4.1 Basic topology detection infrastructure

This section describes the topology detection infrastructure in a bottom-up fashion, from the visible component level to the system level.

At the visible component architecture level, a CoreSight system provides topology detection registers. These registers are accessible through the programmers' model and contain information about the components in the system and permit a debugger to identify the components. See [Topology Detection](#).

At the reusable component architecture level, the system defines interfaces that enable communication between the various components and enable debuggers access to the system. See [Topology Detection at the Component Level](#).

At the system level there is:

- A hierarchy of one or more ROM Tables that describe the address map for the CoreSight system. See [About ROM Tables](#).
- A description of physical connections for the debugger hardware. See [Physical Interface](#).

There are registers to control the wires where buses exist, and enough of the system must be controllable to establish the existence of the link. For example, for ATB interface signals, only **ATVALID** and **ATREADY** must be controlled.

A1.4.2 Mechanism for topology detection

Topology detection is only required when the debugger does not already have information about the system being debugged.

Before it performs topology detection, the debugger uses the following procedure to determine which components are present in the system:

- It connects to the physical interface. See [Physical Interface](#).
- It establishes communication with the system, for example through an interface that complies with the ADI architecture.
- It uses the ROM Table to determine which components are present.

The debugger continues with the following steps:

- For each component, it uses the component type to determine which interfaces are present and how to access signals on these interfaces.
- For each interface, it uses the interface type to determine which signals to access. [Topology Detection at the Component Level](#) describes how to perform topology detection for each of the CoreSight interfaces.

- It uses the algorithm that is described in [Topology Detection at the System Level](#) to perform topology detection. Topology detection asserts and deasserts signals on each Transmitter interface in turn to check each Receiver interface and determine where interfaces are connected together.
- It resets the system and saves the description.

A1.5 Forms of the designer identification codes

The following formats are defined for identifying designers:

- An 11-bit compressed form of JEP106.
- A 16-bit extended form of JEP106.

These codes indicate the designer of the component and not the implementer, except where the designer and implementer are the same. To obtain a number, or to see the assignment of these codes, visit <http://www.jedec.org>.

The 11-bit compressed format has the following form:

- A 4-bit continuation code, indicating the bank of the designer. This is the count of $0 \times 7F$ continuation codes in the designer's JEP106 code. This format is limited to any designer using a code in the first 16 banks of JEP106 codes.
- A 7-bit identity code, indicating the entry in the bank that identifies the designer. This is bits [6:0] of the final number in the designer's JEP106 code. The values 0×00 and $0 \times 7F$ are never allocated to companies by JEDEC.

The 16-bit extended format has the following form:

- A 9-bit continuation code, indicating the bank of the designer. This is the count of $0 \times 7F$ continuation codes in the designer's JEP106 code.
- A 7-bit identity code, indicating the entry in the bank that identifies the designer. This is bits [6:0] of the final number in the designer's JEP106 code. The values 0×00 and $0 \times 7F$ are never allocated to companies by JEDEC.

The contents of the 11-bit compressed format indicate which format is used to identify the designer as follows:

- When the 7-bit identity code in the 11-bit compressed form is not $0 \times 7F$, then the 11-bit compressed form is used to identify the designer.
- When the 7-bit identity code in the 11-bit compressed form is $0 \times 7F$:
 - If the 4-bit continuation code is 0×0 , the 16-bit extended form is present and used to identify the designer.
 - If the 4-bit continuation code is $0 \times F$, the designer is not specified.
 - The values 0×1 through $0 \times E$ of the 4-bit continuation code are reserved.

Example A1-1

For situations where Arm is the designer, the following possible scenarios exist:

- The 11-bit compressed form is used:
 - The 4-bit continuation code is 0×4 , indicating bank 5.
 - The 7-bit identity code is $0 \times 3B$.
- No designer is specified (this is not recommended):
 - The 4-bit continuation code is $0 \times F$.
 - The 7-bit identity code is $0 \times 7F$.
- The 16-bit extended form is used:
 - The 11-bit compressed form indicates the 16-bit extended form is present, as follows:
 - The 4-bit continuation code is 0×0 .
 - The 7-bit identity code is $0 \times 7F$.
 - The 16-bit extended form is as follows:
 - The 9-bit continuation code is 0×004 , indicating bank 5.
 - The 7-bit identity code is $0 \times 3B$.

Example A1-2

For situations where the designer is allocated a code from bank 17, the following possible scenarios exist:

- No designer is specified:
 - The 4-bit continuation code is 0xF.
 - The 7-bit identity code is 0x7F.
- The 16-bit extended form is used:
 - The 11-bit compressed form indicates the 16-bit extended form is present, as follows:
 - The 4-bit continuation code is 0x0.
 - The 7-bit identity code is 0x7F.
 - The 16-bit extended form is as follows:
 - The 9-bit continuation code is 0x010, indicating bank 17.
 - The 7-bit identity code is the index in bank 17 for the designer.

Note

In this example, using the 11-bit compressed form to identify the designer is not possible.

Part B **CoreSight Visible Component Architecture**

Chapter B1

About the Visible Component Architecture

The visible component architecture specifies aspects of components that are visible to the programming interface and to tools that access the device.

The visible component architecture is described in the following chapters:

- [CoreSight programmers' model](#). The programmers' model specifies various registers for the identification and control of the component.
- [Topology Detection](#). The topology detection registers provide the means for the process of topology detection in the CoreSight system.

Chapter B2

CoreSight Programmers' Model

This chapter describes the CoreSight programmers' model. It contains the following sections:

- [About the programmers' model.](#)
- [Component and Peripheral Identification Registers.](#)
- [Component-specific registers for Class 0x9 CoreSight components.](#)
- [Component-specific registers for Class 0xF CoreLink, PrimeCell, and system components.](#)

B2.1 About the programmers' model

This section has the following goals:

- Define the standard set of registers that every CoreSight component must implement in addition to the control registers specific to that component.
- Explain how software can use integration registers for determining the topology of a CoreSight system.

It contains the following subsections:

- [Basic structure of the programmers' model.](#)
- [The Unique Component Identifier.](#)
- [Conventions for registers with less than 32 valid bits.](#)
- [Components that occupy more than 4KB of address space.](#)
- [Programmers' Model Quick Reference.](#)

B2.1.1 Basic structure of the programmers' model

The basic register structure is outlined in [Table B2-3](#). The structure is based on the Peripheral ID Register structure for Arm CoreLink components, and comprises a set of word-aligned 32-bit registers that can be divided into the following categories:

- Component and Peripheral Identification Registers:
 - A Component Identification Register, which extends the original CoreLink specification with a component class that indicates whether extra registers are present. For a description of the requirements for the Component Identification Register, see [Component and Peripheral Identification Registers](#).
 - A Peripheral Identification Register that uniquely identifies the component. For a description of the requirements for the Peripheral Identification Register, see [PIDR0-PIDR7, Peripheral Identification Registers](#).
- Component-specific control registers. The set of required control registers depends on the component class. For a list of valid components classes, see the description of the [CIDR1.CLASS](#) field in [CIDR0-CIDR3, Component Identification Registers](#).

This register structure must be supported by every component that implements a CoreSight compliant programmers' model.

B2.1.2 The Unique Component Identifier

To ensure that a debugger can identify a component, unique components must have a Unique Component Identifier. A Unique Component Identifier is a unique combination of values that are assigned to fields from the Component Identification Registers, the Peripheral Identification Registers, and, if implemented, several component-specific registers.

The fields that can contribute to the Unique Component Identifier are listed in [Table B2-1](#). For details about the individual fields, see the field descriptions in the relevant register descriptions.

Table B2-1 Register fields that contribute to the Unique Component Identifier

Register Type	Register	Fields	Size (bits)
Component Identification Register	CIDR1	CLASS	4
Peripheral Identification Registers	PIDR0	PART_0	8
	PIDR1	DES_0	4
		PART_1	4

Register Type	Register	Fields	Size (bits)
Component-specific registers	PIDR2	REVISION	4
		DES_1	3
	PIDR3	REVAND	4
	PIDR4	DES_2	4
	PIDR4	DES_3	16
	DEVARCH ^a	ARCHITECT	11
		REVISION	4
		ARCHID	16
	DEVARCH1 ^a	ARCHITECT1	16
	DEVTYPE ^a	SUB	4
		MAJOR	4

a. These registers only contribute to the Unique Component Identifier of components with a [CIDR1.CLASS](#) value of 0x9.

The following rules apply to the Unique Component Identifier:

- A common function is defined as a cluster of homogeneous processor components. An example of a common function is a set of debug control registers, a *Performance Monitor Unit* (PMU), a CTI, an ETM, and a ROM Table describing the layout of these components.
The following rules apply to components that are part of a common function:
 - Arm recommends that the values of [PIDR0.PART_0](#) and [PIDR1.PART_1](#) are different for components that are not part of the same common function.
 - Although components that are part of the same common function can share the value of [PIDR0.PART_0](#) and [PIDR1.PART_1](#), each component must have its own Unique Component Identifier.
 - Because [CIDR1.CLASS](#) is part of the Unique Component Identifier, CoreSight version 3.0 permits ROM Tables that are part of a common function to share the part number as the other components of different classes that are part of the same the common function, and to use [CIDR1.CLASS](#) to distinguish between them.
- Multiple instances of the same component are not considered to be unique and usually have the same Unique Component Identifier.
- Where a component has multiple possible configurations and each configuration is a subset of one single configuration, it is not necessary for each configuration to have a separate Unique Component Identifier. For example, a trace macrocell that has a configurable number of comparators does not need a separate Unique Component Identifier for each configuration with a different number of comparators.
- If a component in a subsystem that is described by a ROM Table is changed, the revision number in its Unique Component Identifier must be changed, even if the revision number that is part of the Unique Component Identifier of the ROM Table is not changed.

Note

- If multiple components share the value for the part number in [PIDR0.PART_0](#) and [PIDR1.PART_1](#), they are differentiated by the different values of the other fields in the Unique Component Identifier.
- Component designers who require more than 12 bits for the part number, for example when using a 16-bit part numbering scheme, can use the [PIDR2.REVISION](#) in addition to the part number fields to indicate the part number. In this case, [PIDR3.REVAND](#) indicates the revision of the part. Effectively, the [PIDR0.PART_0](#), [PIDR1.PART_1](#), [PIDR2.REVISION](#), and [PIDR3.REVAND](#) fields provide a total of 20 bits that can be used to create part numbers and revisions of the component.
- CoreSight versions before version 3.0 permitted using the [PIDR3.CMOD](#) field to distinguish between different components. This permission is removed in CoreSight version 3.0.

B2.1.3 Conventions for registers with less than 32 valid bits

The CoreSight programmers' model presents registers as a set of word-aligned registers, meaning every register occupies exactly one word (32-bits), regardless of its information content. The following convention is used in cases where a register has less than 32 bits of valid information:

- Valid information is stored in the least significant bits of the register.
- The most significant bits of the register are reserved, with access permissions that depend on the register.
- When accessed as a 32-bit register, all registers are accessed in little-endian format.

Note

Although a component can be designed to implement only the valid bits of a CoreSight register, software can always access the register as a 32-bit register.

B2.1.4 Components that occupy more than 4KB of address space

The memory layout that is shown in [Table B2-3](#) covers all components that are contained in a single 4KB block of address space. If the registers that are defined for a component do not fit within 4KB, including the 256 bytes reserved for CoreSight management registers, it is necessary to allocate extra address space. The following rules apply:

- Extra address space must be allocated in 4KB blocks, and the total number of blocks must be a power-of-2. The maximum number of blocks is 16,384 for an address space of 64MBytes.
- All 4KB blocks comprising a component must be allocated as a contiguous segment, without gaps.
- The CoreSight programmers' model must be implemented in one of the 4KB blocks making up the component. It is not necessary to implement the programmers' model in the other 4KB blocks that are part of the same component.

Note

From CoreSight version 3.0 onwards, the CoreSight programmers' model does not need to be implemented in the last 4KB block of each individual component.

- Arm recommends that debug tools determine the size of the component from the part number in the Peripheral ID registers and other IMPLEMENTATION DEFINED registers in the component.
From v3.0 onwards, using the [PIDR4.SIZE](#) field to indicate the size of the component is deprecated:
 - Arm recommends that [PIDR4.SIZE](#) is always 0x0. The [PIDR4.SIZE](#) field might not correctly indicate the size of the component.
 - Arm recommends that debug tools ignore the value of [PIDR4.SIZE](#).
 For more information about [PIDR4](#), see [PIDR0-PIDR7, Peripheral Identification Registers](#).
- The bus that is used to access the component must have enough address lines to span the entire allocated address space. See [Table B2-2](#).

The following alternative methods for extending the address space available to a component are possible, but not recommended:

Implementing a second CoreSight component

The address space can be extended by implementing an additional CoreSight component. However, it is not recommended as it requires a method for linking this component back to the original component through topology detection.

Implementing an extra linked address space

The address space can be extended with the address space of a component with an area that is not used by any CoreSight components that were designed according to the programmers' model. However, it is not recommended as it requires a method for determining the address of the extended address space in the programmers' model of the component. This limits the system design options.

Table B2-2 summarizes the relationship between the address space size, the number of available registers, and the number of required address lines.

Table B2-2 Spanning multiple 4KB windows

Number of 4KB blocks	Total memory window used	Component-specific registers available (words)	Expected PADDRDBG input ^a
1	4KB, 1K words	960	PADDRDBG[11:2]
2	8KB	1984	PADDRDBG[12:2]
4	16KB, 4K words	4032	PADDRDBG[13:2]
8	32KB, 8K words	8128	PADDRDBG[14:2]
16	64KB	16320	PADDRDBG[15:2]
32	128KB	32704	PADDRDBG[16:2]
64	256KB, 64K words	65472, 63.94K	PADDRDBG[17:2]
128	512KB	127.94K	PADDRDBG[18:2]
256	1MB, 256K words	255.9K	PADDRDBG[19:2]
512	2MB	~512K	PADDRDBG[20:2]
1024	4MB	~1M	PADDRDBG[21:2]
2048	8MB	~2M	PADDRDBG[22:2]
4096	16MB	~4M	PADDRDBG[23:2]
8192	32MB	~8M	PADDRDBG[24:2]
16384	64MB, 16M words	~16M	PADDRDBG[25:2]
Reserved	-	-	-

a. This table uses the AMBA APB protocol as an example protocol used to access a component, where the PADDRDBG bus is the address bus that is used to select the component registers. PADDRDBG[1:0] are not required on components because all transfers are 32-bit word-aligned.

B2.1.5 Programmers' Model Quick Reference

Table B2-3 shows the address offsets for the CoreSight component registers, in order of their offset in the 4KB block.

Table B2-3 CoreSight component register address offsets

Offset	Type	Name	Description	
0xF00	RW	ITCTRL	Integration Mode Control Register	
0xF04-0xF9C	RES0	-	Reserved	
0xFA0	RW	CLAIMSET	Claim Tag Set Register	Claim Tag Registers
0xFA4	RW	CLAIMCLR	Claim Tag Clear Register	
0xFA8	RO	DEVAFF0	Device Affinity Registers	
0xFAC	RO	DEVAFF1		
0xFB0	WO	LAR	Software Lock Access Register	Software Lock Registers
0xFB4	RO	LSR	Software Lock Status Register	

Offset	Type	Name	Description	
0xFB8	RO	AUTHSTATUS	Authentication Status Register	
0xFBC	RO	DEVARCH	Device Architecture Register	
0xFC0	RO	DEVID2	Device Configuration Register 2	
0xFC4	RO	DEVID1	Device Configuration Register 1	
0xFC8	RO	DEVID	Device Configuration Register	
0xFCC	RO	DEVTYPE	Device Type Identifier Register	
0xFD0	RO	PIDR4	Component size (deprecated) and JEP106 identification	Peripheral Identification Registers
0xFD4	RO	PIDR5		
0xFD8	RO	PIDR6		
0xFDC	RO	PIDR7		
0xFE0	RO	PIDR0	Part number	
0xFE4	RO	PIDR1	JEP106 identification and Part number	Component Identification Registers
0xFE8	RO	PIDR2	Revision and JEP106 identification	
0xFEC	RO	PIDR3	RevAnd and Customer modified	
0xFF0	RO	CIDR0	Preamble	
0xFF4	RO	CIDR1	Component class and Preamble	
0xFF8	RO	CIDR2	Preamble	
0xFFC	RO	CIDR3	Preamble	

The words at offsets 0xF00-0xFFC are the CoreSight management registers. These registers are common to all CoreSight components. This area is reserved for CoreSight registers, and device-specific control registers must not use it. The CoreSight management registers include:

- The Peripheral ID Registers, at offsets 0xFD0-0xFEC.
- The Component ID Registers, at offsets 0xFF0-0xFFC.

The remaining words can be used for component-specific registers. Arm recommends using the following conventions:

- Control registers start at address 0x000 and continue upwards.
- Any registers that are used purely for integration purposes start at address 0xEFC and continue downwards.

The register type of device-specific registers is IMPLEMENTATION DEFINED and can be RW, RO, or WO.

[Table B2-4](#) defines the behavior on accesses to reserved registers and fields.

Table B2-4 Behavior on accesses to reserved registers and fields

Access to	Behavior
Reserved registers	RES0
Unimplemented registers	RAZ/WI
Reserved fields in registers	RES0 or RES1
Unimplemented fields in registers	RES0 or RES1

Access to	Behavior
Unimplemented bits in implemented fields	RAZ/WI

Locations that are marked as RES0 are reserved. Reads of write-only registers are considered accesses to Reserved registers. Writes to read-only registers are considered accesses to Reserved registers. The following tables show the specific meaning of each of the behaviors.

Table B2-5 shows the required behavior of a CoreSight component for registers that are defined as RW, RO, or WO.

Table B2-5 CoreSight component behavior

Behavior	Component behavior on reads			Component behavior on writes		
	RW	RO	WO	RW	RO	WO
RES0	RAZ	RAZ	RAZ	WI	WI	WI
RES1	RAO	RAO	RAO	WI	WI	WI
RAZ/WI	RAZ	RAZ	RAZ	WI	WI	WI

Table B2-6 shows the required behavior of software when accessing a CoreSight component for registers that are defined as RW, RO, or WO.

Table B2-6 Software behavior

Behavior	Software behavior on reads			Software behavior on writes		
	RW	RO	WO	RW	RO	WO
RES0	Treat as UNKNOWN	Treat as UNKNOWN	Do not read	Preserve	Do not write	Preserve
RES1	Treat as UNKNOWN	Treat as UNKNOWN	Do not read	Preserve	Do not write	Preserve
RAZ/WI	Expect zero	Expect zero	Do not read	Are ignored	Do not write	Are ignored

Programming a reserved value into a register, or a field within a register, might result in CONSTRAINED UNPREDICTABLE behavior of the component. Usually, this involves mapping the behavior to one or more permitted behaviors.

B2.2 Component and Peripheral Identification Registers

This section describes the following registers:

- CIDR0-CIDR3, Component Identification Registers.
- PIDR0-PIDR7, Peripheral Identification Registers.

B2.2.1 CIDR0-CIDR3, Component Identification Registers

The CIDR0-CIDR3 characteristics are:

Purpose Provide information that can be used to identify a CoreSight component.

Usage constraints

CIDR0-CIDR3 are accessible as follows:

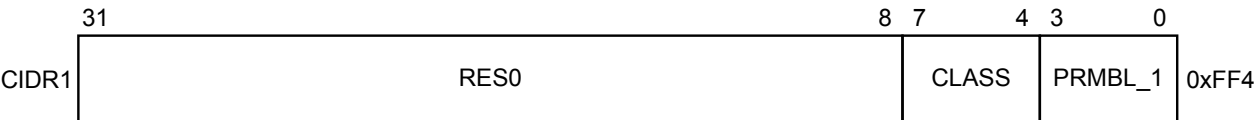
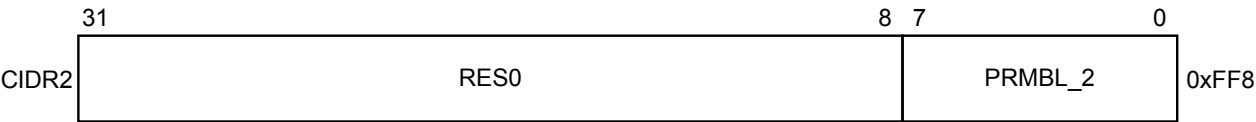
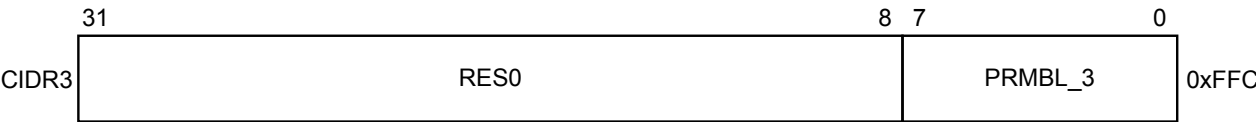
Default
RO

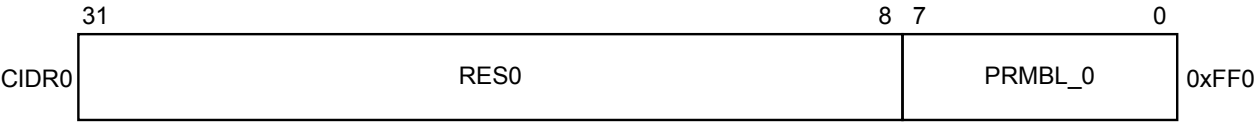
Configurations Included in all implementations.

Attributes A set of four 32-bit management registers.

B2.2.1.1 Field Descriptions

The CIDR0-CIDR3 bit assignments are:





Bits[31:8] of CIDR3

RES0.

PRMBL_3, CIDR3 bits[7:0]

Preamble, segment 3. Must be 0xB1.

Bits[31:8] of CIDR2

RES0.

PRMBL_2, CIDR2 bits[7:0]

Preamble, segment 2. Must be 0x05.

Bits[31:8] of CIDR1

RES0.

CLASS, CIDR1 bits[7:4]

The component class, which can be one of the values that are listed in [Table B2-8](#).

Table B2-8 CLASS field encodings

Value	Description
0x0	Generic verification component.
0x1	ROM Table. See ROM Table Types .
0x2-0x8	Reserved.
0x9	CoreSight component. See Component-specific registers for Class 0x9 CoreSight components .
0xA	Reserved.
0xB	Peripheral Test Block.
0xC-0xD	Reserved.
0xE	Generic IP component.
0xF	CoreLink, PrimeCell, or system component with no standardized register layout, for backwards compatibility. See Component-specific registers for Class 0xF CoreLink, PrimeCell, and system components .

PRMBL_1, CIDR1 bits[3:0]

Preamble, segment 1. Must be 0x0.

Bits[31:8] of CIDR0

RES0.

PRMBL_0, CIDR0 bits[7:0]

Preamble, segment 0. Must be 0x0D.

B2.2.1.2 Accessing CIDR

CIDR0-CIDR3 can be accessed at the following addresses:

Offset			
CIDR0	CIDR1	CIDR2	CIDR3
0xFF0	0xFF4	0xFF8	0xFFC

B2.2.2 PIDR0-PIDR7, Peripheral Identification Registers

The [PIDR0-PIDR7](#) characteristics are:

Purpose Provide information that can be used to identify a CoreSight component. Most of the fields making up [PIDR0-PIDR7](#) are included in the Unique Component Identifier. The Unique Component Identifier can also include fields from the [CIDR1](#), [DEVARCH](#), and [DEVTYPE](#) registers. For details, see [The Unique Component Identifier](#).

Usage constraints

[PIDR0-PIDR7](#) are accessible as follows:

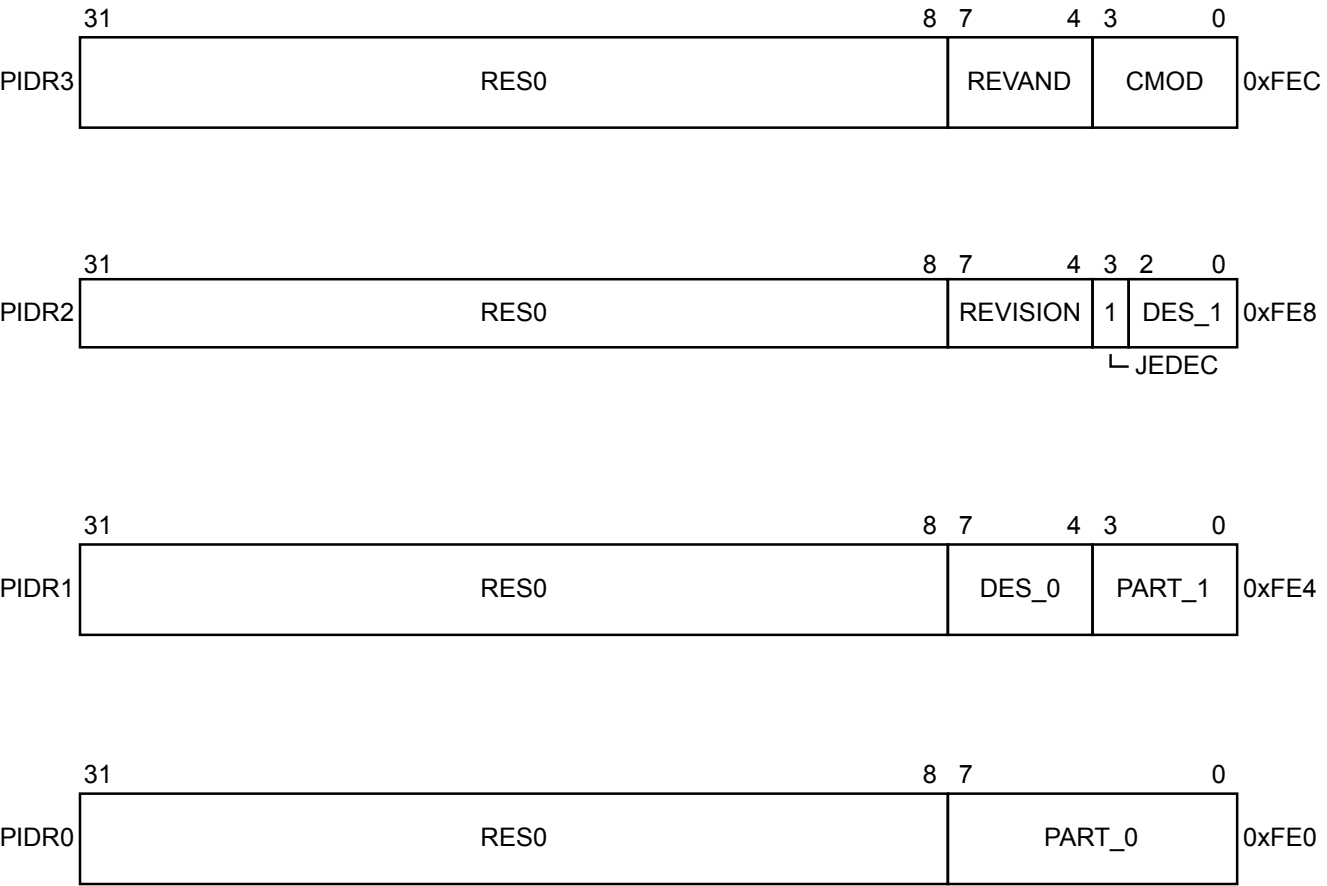
Default
RO

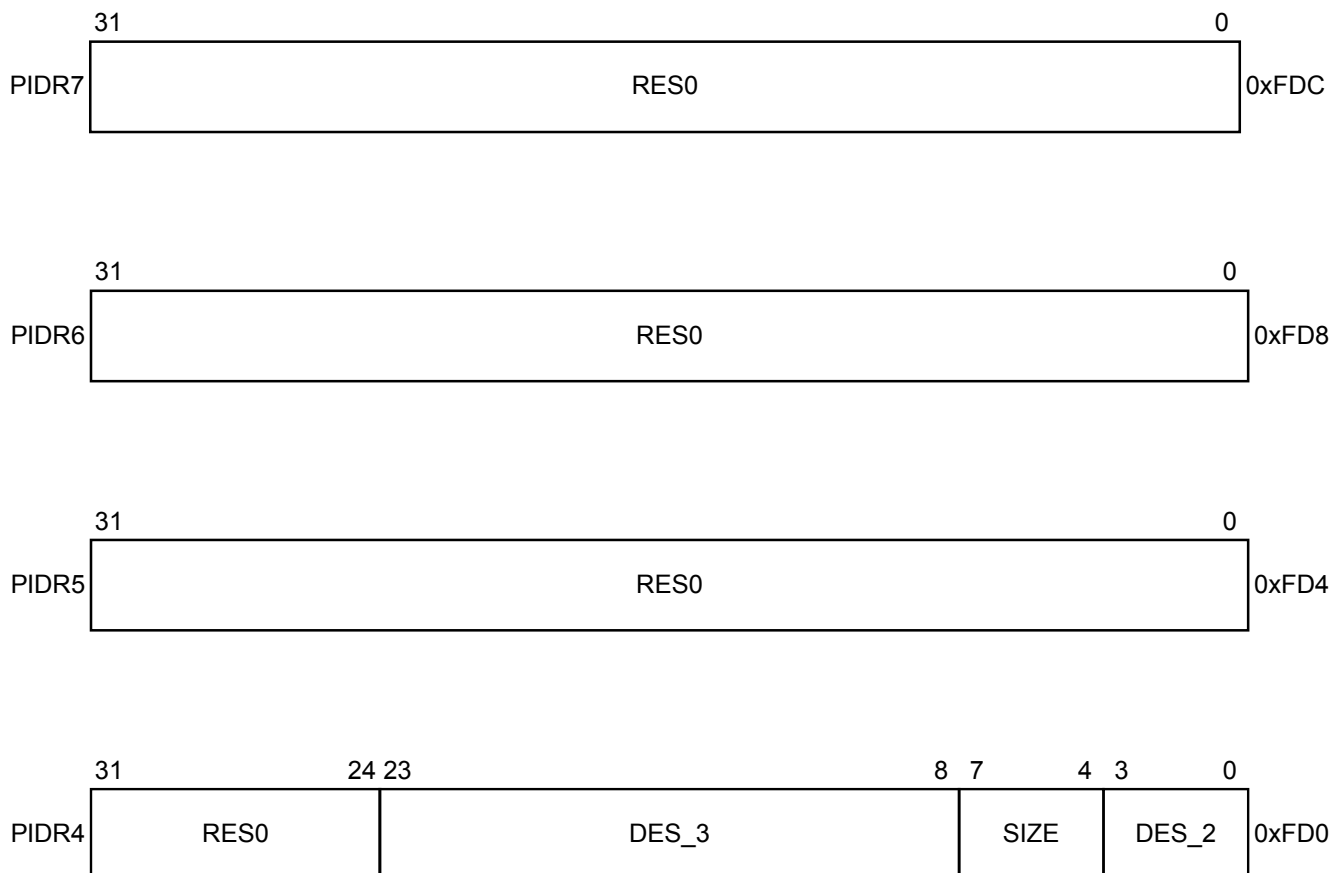
Configurations Included in all implementations.

Attributes A set of eight 32-bit management registers.

B2.2.2.1 Field Descriptions

The [PIDR0-PIDR7](#) bit assignments are:





PIDR3 bits[31:8]

RES0.

REVAND, PIDR3 bits[7:4]

Indicates either the revision of the component, or whether the component has been modified.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- [PIDR2.REVISION](#) indicates the 4-bit revision number.
- [PIDR3.REVAND](#) indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- [PIDR2.REVISION](#) indicates the 4-bit major revision number.
- [PIDR3.REVAND](#) indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- [PIDR2.REVISION](#) contains part number bits [3:0].
- [PIDR3.REVAND](#) indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

Where REVAND indicates component modifications, this indicates modifications such as errata fixes or metal fixes after implementation. Usually this value would be zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

[PIDR3](#).CMOD might also indicate component modifications.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

CMOD, PIDR3 bits[3:0]

Indicates whether the component has been modified from its original behavior. Examples of modifications include errata fixes or metal fixes after implementation. Usually this value would be zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

0x0 The component is not modified from the original design.

Any other value The component has been modified in an IMPLEMENTATION DEFINED way.

For any two components with the same Unique Component Identifier:

- If the value of the CMOD fields of both components equals zero, the components are identical.
- If the CMOD fields of both components have the same nonzero value, it does not necessarily mean that they have the same modifications.
- If the value of the CMOD field of either of the two components is nonzero, they might not be identical, even though they have the same Unique Component Identifier.

See also [The Unique Component Identifier](#).

[PIDR3](#).REVAND might also indicate component modifications.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

PIDR2 bits[31:8]

RES0.

REVISION, PIDR2 bits[7:4]

Indicates either the revision of the component, or a portion of the part number of the component.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- [PIDR2](#).REVISION indicates the 4-bit revision number.
- [PIDR3](#).REVAND indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- [PIDR2](#).REVISION indicates the 4-bit major revision number.
- [PIDR3](#).REVAND indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- [PIDR2](#).REVISION contains part number bits [3:0].
- [PIDR3](#).REVAND indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

JEDEC, PIDR2 bits[3]

Must be 0b1 to indicate that a JEDEC-assigned value is used.

DES_1, PIDR2 bits[2:0]

Indicates the designer of the component using the 11-bit compressed form of a JEP106 code.

When using the 11-bit compressed form of the JEP106 code, all of the following apply:

- [PIDR1](#).DES_0 indicates bits [3:0] of the JEP106 identification code.
- [PIDR2](#).DES_1 indicates bits [6:4] of the JEP106 identification code.
- [PIDR4](#).DES_2 indicates bits [3:0] of the JEP106 continuation code.
- [PIDR4](#).DES_3 is RES0.

When using the 16-bit extended form of the JEP106 code, all of the following apply:

- [PIDR1](#).DES_0 is 0xF.
- [PIDR2](#).DES_1 is 0x7.
- [PIDR4](#).DES_2 is 0x0.
- [PIDR4](#).DES_3 indicates the 16-bit extended form of the JEP106 code.

Otherwise, the designer is not identified and all of the following apply:

- [PIDR1](#).DES_0 is 0xF.
- [PIDR2](#).DES_1 is 0x7.
- [PIDR4](#).DES_2 is 0xF.
- [PIDR4](#).DES_3 is RES0.

For details about the JEP106 codes, see [Forms of the designer identification codes](#).

PIDR1 bits[31:8]

RES0.

DES_0, PIDR1 bits[7:4]

Indicates the designer of the component using the 11-bit compressed form of a JEP106 code.

When using the 11-bit compressed form of the JEP106 code, all of the following apply:

- [PIDR1](#).DES_0 indicates bits [3:0] of the JEP106 identification code.
- [PIDR2](#).DES_1 indicates bits [6:4] of the JEP106 identification code.
- [PIDR4](#).DES_2 indicates bits [3:0] of the JEP106 continuation code.
- [PIDR4](#).DES_3 is RES0.

When using the 16-bit extended form of the JEP106 code, all of the following apply:

- [PIDR1](#).DES_0 is 0xF.
- [PIDR2](#).DES_1 is 0x7.
- [PIDR4](#).DES_2 is 0x0.
- [PIDR4](#).DES_3 indicates the 16-bit extended form of the JEP106 code.

Otherwise, the designer is not identified and all of the following apply:

- [PIDR1.DES_0](#) is 0xF.
- [PIDR2.DES_1](#) is 0x7.
- [PIDR4.DES_2](#) is 0xF.
- [PIDR4.DES_3](#) is RES0.

For details about the JEP106 codes, see [Forms of the designer identification codes](#).

PART_1, PIDR1 bits[3:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - [PIDR1.PART_1](#) contains part number bits [11:8].
 - [PIDR0.PART_0](#) contains part number bits [7:0].
- For a component with a 16-bit part number:
 - [PIDR1.PART_1](#) contains part number bits [15:12].
 - [PIDR0.PART_0](#) contains part number bits [11:4].
 - [PIDR2.REVISION](#) contains part number bits [3:0].

When a 12-bit part number is used, [PIDR2.REVISION](#) indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

PIDR0 bits[31:8]

RES0.

PART_0, PIDR0 bits[7:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - [PIDR1.PART_1](#) contains part number bits [11:8].
 - [PIDR0.PART_0](#) contains part number bits [7:0].
- For a component with a 16-bit part number:
 - [PIDR1.PART_1](#) contains part number bits [15:12].
 - [PIDR0.PART_0](#) contains part number bits [11:4].
 - [PIDR2.REVISION](#) contains part number bits [3:0].

When a 12-bit part number is used, [PIDR2.REVISION](#) indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

PIDR7 bits[31:0]

RES0.

PIDR6 bits[31:0]

RES0.

PIDR5 bits[31:0]

RES0.

PIDR4 bits[31:24]

RES0.

DES_3, PIDR4 bits[23:8]

Indicates the designer of the component using the 16-bit extended form of a JEP106 code.

When using the 16-bit extended form of the JEP106 code:

- [PIDR1.DES_0](#) is 0xF.
- [PIDR2.DES_1](#) is 0x7.
- [PIDR4.DES_2](#) is 0x0.
- [PIDR4.DES_3](#) indicates the 16-bit extended form of the JEP106 code as follows:
 - [DES_3](#) bits [6:0] indicate the JEP106 identification code.
 - [DES_3](#) bits [15:7] indicate the JEP106 continuation code.

Otherwise, this field is RES0.

For details about the JEP106 codes, see [Forms of the designer identification codes](#).

SIZE, PIDR4 bits[7:4]

The SIZE field indicates the memory size that is used by this component. It is expressed as the logarithm to the base 2 of the number of 4KB blocks the component occupies. The value 0x0 indicates that either:

- The component uses a single 4KB block.
- The component uses an UNKNOWN number of 4KB blocks.

Using the SIZE field to indicate the size of the component is deprecated. The SIZE field might not correctly indicate the size of the component. Arm recommends that debug tools determine the size of the component from the [Unique Component Identifier](#) fields, and other IMPLEMENTATION DEFINED registers in the component.

DES_2, PIDR4 bits[3:0]

Indicates the designer of the component using the 11-bit compressed form of a JEP106 code.

When using the 11-bit compressed form of the JEP106 code, all of the following apply:

- [PIDR1.DES_0](#) indicates bits [3:0] of the JEP106 identification code.
- [PIDR2.DES_1](#) indicates bits [6:4] of the JEP106 identification code.
- [PIDR4.DES_2](#) indicates bits [3:0] of the JEP106 continuation code.
- [PIDR4.DES_3](#) is RES0.

When using the 16-bit extended form of the JEP106 code, all of the following apply:

- [PIDR1.DES_0](#) is 0xF.
- [PIDR2.DES_1](#) is 0x7.
- [PIDR4.DES_2](#) is 0x0.
- [PIDR4.DES_3](#) indicates the 16-bit extended form of the JEP106 code.

Otherwise, the designer is not identified and all of the following apply:

- [PIDR1.DES_0](#) is 0xF.
- [PIDR2.DES_1](#) is 0x7.
- [PIDR4.DES_2](#) is 0xF.
- [PIDR4.DES_3](#) is RES0.

For details about the JEP106 codes, see [Forms of the designer identification codes](#).

B2.2.2.2 Accessing the PIDR

[PIDR0-PIDR7](#) can be accessed at the following addresses:

Offset							
PIDR0	PIDR1	PIDR2	PIDR3	PIDR4	PIDR5	PIDR6	PIDR7
0xFE0	0xFE4	0xFE8	0xFEC	0xFD0	0xFD4	0xFD8	0xFDC

B2.3 Component-specific registers for Class 0x9 CoreSight components

Components that have the value 0x9 assigned to the [CIDR1.CLASS](#) field in the Component Identification Register are CoreSight components. For details, see [CIDR0-CIDR3, Component Identification Registers](#).

CoreSight components must implement an extra set of registers, referred to as the CoreSight management registers, which are described in this section. Addresses 0xF00 to 0xFCC are reserved for use by CoreSight management registers.

When implementing a CoreSight component, ensure that the following requirements are met:

- Any reads from unimplemented or reserved registers in 0xF00 to 0xFFF must return zero, and writes must be ignored. For details about the required behavior of reserved locations, see [Programmers' Model Quick Reference](#).
- Two or more functionally different CoreSight components are permitted to share a part number, as long as they each have a different Unique Component Identifier. See [The Unique Component Identifier](#).

B2.3.1 AUTHSTATUS, Authentication Status Register

The AUTHSTATUS characteristics are:

Purpose Reports the required security level and status of the authentication interface. Where functionality changes on a given security level, the change in status must be reported in this register. For details about the authentication interface, see Authentication Interface.

Usage constraints

Some components might not distinguish between different Security states but be used for debug of different Security states.

For example, a trace component for a simple bus might be connected to a bus for only Secure transactions or might be connected to a bus for only Non-secure transactions, and the component itself is unaware of which type of bus to which it is connected. The Authentication interface signals connected to the component must be driven according to the type of debug permitted and the type of bus it is connected to. This lack of disambiguation by the component might result in:

- A component that indicates only Non-secure debug capabilities while performing only Secure debug functions.
- A component that indicates only Secure debug capabilities while performing only Non-secure debug functions.

Debuggers must be able to accommodate this possibility.

AUTHSTATUS is accessible as follows:

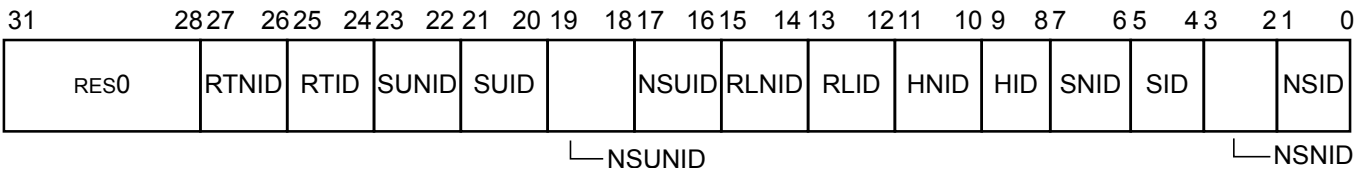
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register.

B2.3.1.1 Field Descriptions

The AUTHSTATUS bit assignments are:



Bits[31:28]
RES0.

RTNID, bits [27:26]
Root Non-invasive Debug. Indicates whether a separate enable control for Root state non-invasive debug features is implemented and enabled. The defined values of this field are:

0b00	Separate Root non-invasive debug enable not implemented or Root state non-invasive debug features not implemented.
0b10	Implemented and disabled.

0b11 Implemented and enabled.

All other values are reserved.

RTID, bits [25:24]

Root Invasive Debug. Indicates whether a separate enable control for Root state invasive debug is implemented and enabled. The defined values of this field are:

0b00 Separate Root invasive debug enable not implemented or Root state invasive debug features not implemented.

0b10 Implemented and disabled.

0b11 Implemented and enabled.

All other values are reserved.

SUNID, bits [23:22]

Secure Unprivileged non-invasive debug. Indicates whether a separate enable control for Secure state non-invasive debug features is implemented and enabled. The defined values of this field are:

0b00 Debug level is not supported.

0b01 Reserved.

0b10 Supported and disabled.

0b11 Supported and enabled.

All other values are reserved.

SUID, bits [21:20]

Secure Unprivileged invasive debug. Indicates whether a separate enable control for Secure state invasive debug features is implemented and enabled. The defined values of this field are:

0b00 Debug level is not supported.

0b01 Reserved.

0b10 Supported and disabled.

0b11 Supported and enabled.

All other values are reserved.

NSUNID, bits [19:18]

Non-secure Unprivileged non-invasive debug. Indicates whether a separate enable control for Non-secure state non-invasive debug features is implemented and enabled. The defined values of this field are:

0b00 Debug level is not supported.

0b01 Reserved.

0b10 Supported and disabled.

0b11 Supported and enabled.

All other values are reserved.

NSUID, bits [17:16]

Non-secure Unprivileged invasive debug. Indicates whether a separate enable control for Non-secure state invasive debug features is implemented and enabled. The defined values of this field are:

0b00 Debug level is not supported.

0b01	Reserved.
0b10	Supported and disabled.
0b11	Supported and enabled.

All other values are reserved.

RLNID, bits[15:14]

Realm Non-invasive debug. Indicates whether a separate enable control for Realm state non-invasive debug features is implemented and enabled. The defined values of this field are:

0b00	Separate Realm non-invasive debug enable not implemented or Realm state non-invasive debug features not implemented.
0b10	Implemented and disabled.
0b11	Implemented and enabled.

All other values are reserved.

RLID, bits[13:12]

Realm Invasive Debug. Indicates whether a separate enable control for Realm state invasive debug is implemented and enabled. The defined values of this field are:

0b00	Separate Realm invasive debug enable not implemented or Realm state invasive debug features not implemented.
0b10	Implemented and disabled.
0b11	Implemented and enabled.

All other values are reserved.

HNID, bits[11:10]

Hypervisor non-invasive debug.

This field can have one of the following values:

0b00	Debug level not supported.
0b10	Supported and disabled.
0b11	Supported and enabled.

All other values are reserved.

HID, bits[9:8]

Hypervisor invasive debug.

This field can have one of the following values:

0b00	Debug level is not supported.
0b10	Supported and disabled.
0b11	Supported and enabled.

All other values are reserved.

SNID, bits[7:6]

Secure noninvasive debug.

This field can have one of the following values:

0b00	Debug level is not supported.
------	-------------------------------

0b10 Supported and disabled.
0b11 Supported and enabled.
All other values are reserved.

SID, bits[5:4]

Secure invasive debug.
This field can have one of the following values:
0b00 Debug level is not supported.
0b10 Supported and disabled.
0b11 Supported and enabled.
All other values are reserved.

NSNID, bits[3:2]

Non-secure noninvasive debug.
This field can have one of the following values:
0b00 Debug level is not supported.
0b10 Supported and disabled.
0b11 Supported and enabled.
All other values are reserved.

NSID, bits[1:0]

Non-secure invasive debug.
This field can have one of the following values:
0b00 Debug level is not supported.
0b10 Supported and disabled.
0b11 Supported and enabled.
All other values are reserved.

B2.3.1.2 Accessing AUTHSTATUS

AUTHSTATUS can be accessed at the following address:

Offset
0xFB8

B2.3.2 CLAIMSET and CLAIMCLR, Claim Tag Set Register and Claim Tag Clear Register

The characteristics of CLAIMSET and CLAIMCLR are:

Purpose	Often there are several debug agents that must cooperate to control the resources that the CoreSight components make available. For example, an external debugger and a debug monitor running on the target might both require control of the breakpoint resources of a processor. It is important that a debug agent does not reprogram debug resources that another debug agent is using.
----------------	---

The Claim tag registers provide various bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent. All debug agents must implement a common protocol to use these bits.

This specification does not define the claim tag protocol, but consider the following examples that illustrate how these bits can be used:

Protocol 1: Set common bit to claim

In this scenario, debug functionality is only claimed on a few rare, well-defined points, for example when the target is powered up or when a debugger is connected.

Each bit in the claim tag corresponds to an area of debug functionality, which is shared between all debug agents. For example, 4 bits can control four areas of functionality. The following shows a pseudocode implementation of this protocol:

```
read claim tag bit
if (bit is set)
    functionality is not available
else
    set bit
    use functionality
```

Protocol 2: Set private bit to claim

In this scenario, debug functionality is also only claimed on a few rare, well-defined points, but it is necessary to be able to determine which other agent has claimed functionality.

Each bit in the claim tag corresponds to an area of debug functionality for a debug agent. For example, 4 bits can control two areas of functionality each for two debug agents. The following shows a pseudocode implementation of this protocol:

```
read all claim tag bits for this functionality
if (any bits are set)
    functionality is not available
else
    set bit for this agent
    use functionality
```

Protocol 3: Set private bit and check for race

In this scenario, debug functionality is claimed regularly and it is possible for two debug agents to attempt to claim it at the same time. Each bit in the claim tag corresponds to an area of debug functionality for a debug agent, as in protocol 2. The following shows a pseudocode implementation of this protocol:

```
read all claim tag bits for this functionality
if (any bits are set)
    functionality is not available
else
    set bit for this agent
    read all claim tag bits for this functionality
    if (any bits are set by other agents)
        clear bit for this agent
        wait a random amount of time
        go back to start
    else
        use functionality
```

Usage constraints

The value of [CLAIMCLR](#) must be zero at reset.

[CLAIMSET](#) and [CLAIMCLR](#) are accessible as follows:

Default

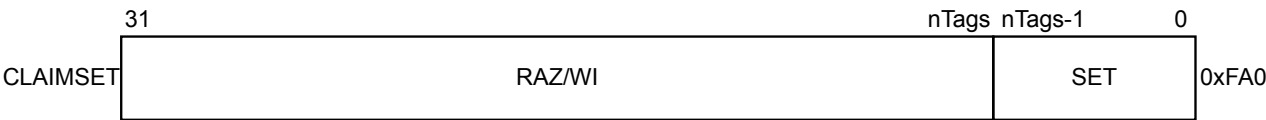
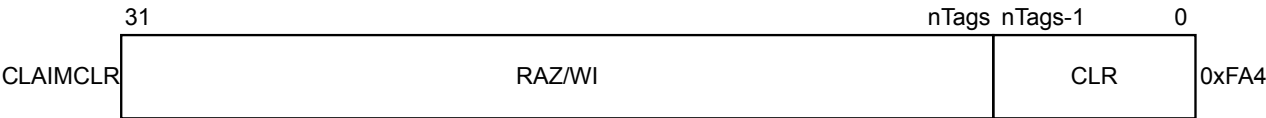
RW

Configurations Included in all implementations.

Attributes 32-bit registers.

B2.3.2.1 Field Descriptions

The [CLAIMSET](#) and [CLAIMCLR](#) bit assignments are:



CLAIMCLR bits[31:nTags]

RAZ/WI

CLR, CLAIMCLR bits[nTags-1:0]

The size of this field, nTags, is IMPLEMENTATION DEFINED, and equals the number of bits set in [CLAIMSET](#).

Allowed values of CLR[n] are:

- | | |
|----------------|---------------------------------|
| Write 0 | No effect. |
| Write 1 | Clear the claim tag for bit[n]. |
| Read 0 | The claim tag bit is not set. |
| Read 1 | The claim tag bit is set. |

CLAIMSET bits[31:nTags]

RAZ/WI

SET, CLAIMSET bits[nTags-1:0]

The size of this field, nTags, is IMPLEMENTATION DEFINED, and equals the number of claim bits that are implemented.

Permitted values of SET[n] are:

- | | |
|----------------|---|
| Write 0 | No effect. |
| Write 1 | Set the claim tag for bit[n]. |
| Read 0 | The claim tag that is represented by bit[n] is not implemented. |

Read 1 The claim tag that is represented by bit[*n*] is implemented.

B2.3.2.2 Accessing CLAIMCLR and CLAIMSET

[CLAIMCLR](#) and [CLAIMSET](#) can be accessed at the following address:

Offset	
CLAIMCLR	CLAIMSET
0xFA4	0xFA0

B2.3.3 DEVAFF0-DEVAFF1, Device Affinity Registers

The DEVAFF0-DEVAFF1 characteristics are:

Purpose Enables a debugger to determine whether two components have an affinity with each other.

For example, when a trace macrocell connects to a processor, the DEVAFF0 and DEVAFF1 in both components must contain identical values that are unique. Doing so enables the debugger to identify how the components relate to each other, without performing topology detection.

Usage constraints

DEVAFF0-DEVAFF1 are accessible as follows:

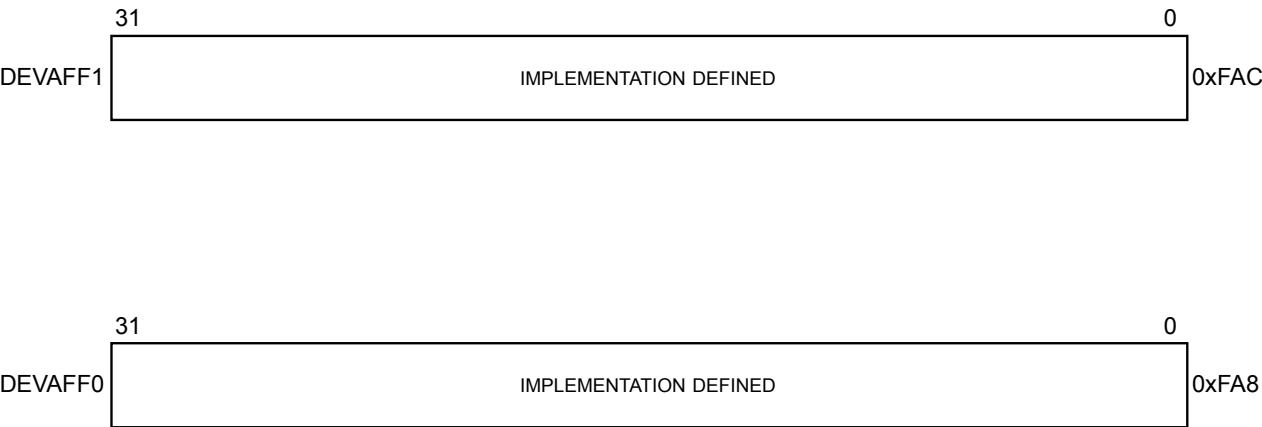
Default
RO

Configurations Included in all implementations.

Attributes A set of 32-bit registers that return an IMPLEMENTATION DEFINED value. A component might have affinity with a group of components, for example where a single component is shared between multiple PEs. The DEVAFF registers can be used to indicate the affinity with a group of components. See the *Arm® Architecture Reference Manual, for A-profile architecture* for examples of how affinity with a group of PEs is indicated.

B2.3.3.1 Field Descriptions

The DEVAFF0-DEVAFF1 bit assignments are:



DEVAFF1, bits[31:0] and DEVAFF0, bits[31:0]

IMPLEMENTATION DEFINED. If a component has no unique association with another component, these fields are RAZ.

Examples of the content that DEVAFF returns are the MPIDRs of Arm architecture processors and CTIs that connect to them:

- DEVAFF0 returns MPIDR, bits[31:0].
- DEVAFF1 returns MPIDR, bits[63:32].

B2.3.3.2 Accessing DEVAFF0-DEVAFF1

DEVAFF0-DEVAFF1 can be accessed at the following address:

Offset	
DEVAFF0	DEVAFF1
0xFA8	0xFAC

B2.3.4 DEVARCH, Device Architecture Register

The [DEVARCH](#) characteristics are:

Purpose Identifies the architect and architecture of a CoreSight component. The architect might differ from the designer of a component, for example when Arm defines the architecture but another company designs and implements the component.

Usage constraints

[DEVARCH](#) is accessible as follows:

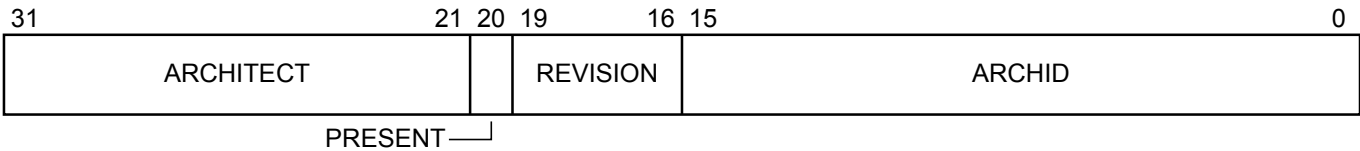
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register that returns an IMPLEMENTATION DEFINED value.

B2.3.4.1 Field Descriptions

The [DEVARCH](#) bit assignments are:



ARCHITECT, bits[31:21]

Indicates the architect of the component using the 11-bit compressed form of a JEP106 code.

When using the 11-bit compressed form of the JEP106 code, all of the following apply:

- [DEVARCH](#).ARCHITECT bits [6:0] indicate the JEP106 identification code.
- [DEVARCH](#).ARCHITECT bits [10:7] indicate the JEP106 continuation code.
- [DEVARCH1](#).ARCHITECT1 is RES0.

Note
For components where Arm is the architect, this field is 0x23B.

When using the 16-bit extended form of the JEP106 code, all of the following apply:

- [DEVARCH](#).ARCHITECT is 0x07F.
- [DEVARCH1](#).ARCHITECT1 indicates the 16-bit extended form of the JEP106 code.

Otherwise, the architect is not identified and all of the following apply:

- [DEVARCH](#).ARCHITECT is 0x7FF.
- [DEVARCH1](#).ARCHITECT1 is RES0.

For details about the JEP106 codes, see [Forms of the designer identification codes](#).

PRESENT, bit[20]

Indicates the presence of this register:

0 = **DEVARCH** is not present. Bits[31:0] must be RAZ.

1 = **DEVARCH** is present.

REVISION, bits[19:16]

Architecture revision. Returns the revision of the architecture that the ARCHID field specifies.

ARCHID, bits[15:0]

Architecture ID. Returns a value that identifies the architecture of the component.

[Table B2-19](#) lists the ARCHID values for some example components where Arm is the architect.

Table B2-19 Example ARCHID values

ARCHID	Description
0x0A00	RAS architecture
0x1A01	<i>Instrumentation Trace Macrocell</i> (ITM) architecture
0x1A02	DWT architecture
0x1A03	<i>Flash Patch and Breakpoint unit</i> (FPB) architecture
0x2A04	Processor debug architecture (Armv8-M)
0x6A05	Processor debug architecture (Armv8-R)
0x0A10	PC sample-based profiling
0x4A13	ETM architecture.
0x1A14	CTI architecture
0x6A15	Processor debug architecture (v8.0-A)
0x7A15	Processor debug architecture (v8.1-A)
0x8A15	Processor debug architecture (v8.2-A)
0x2A16	<i>Processor Performance Monitor</i> (PMU) architecture
0x0A17	Memory Access Port v2 architecture
0x0A27	JTAG Access Port v2 architecture
0x0A31	Basic trace router
0x0A34	Power requester
0x0A47	Unknown Access Port v2 architecture
0x0A50	HSSTP architecture
0x0A63	STM architecture
0x0A75	CoreSight ELA architecture
0x0AF7	CoreSight ROM architecture

B2.3.4.2 Accessing DEVARCH

DEVARCH can be accessed at the following address:

Offset
0xFBC

B2.3.5 DEVARCH1, Device Architecture Register 1

The DEVARCH1 characteristics are:

Purpose

Identifies the architect and architecture of a CoreSight component.

Usage constraints

DEVARCH1 is accessible as follows:

Default
RO

Configurations

Included in all implementations.

Attributes

A 32-bit register.

B2.3.5.1 Field Descriptions

The DEVARCH1 bit assignments are:



Bits[31:24]

RES0.

ARCHITECT1, bits[23:8]

Indicates the architect of the component using the 16-bit extended form of a JEP106 code.

When using the 16-bit extended form of the JEP106 code:

- DEVARCH.ARCHITECT is 0x07F.
- DEVARCH1.ARCHITECT1 indicates the 16-bit extended form of the JEP106 code as follows:
 - ARCHITECT1 bits [6:0] indicate the JEP106 identification code.
 - ARCHITECT1 bits [15:7] indicate the JEP106 continuation code.

Otherwise, this field is RES0.

For details about the JEP106 codes, see Forms of the designer identification codes.

Bits[7:0]

RES0.

B2.3.5.2 Accessing DEVARCH1

DEVARCH1 can be accessed at the following address:

Offset
0xF9C

B2.3.6 DEVID, Device Configuration Register

The DEVID characteristics are:

Purpose Indicates the capabilities of the component.

Usage constraints This register is IMPLEMENTATION DEFINED for each part number and designer.

The entire 32-bit field can be used because the data width is determined by the component itself.

Unused bits must be RES0.

If the component is configurable, Arm recommends that this register reflects any changes to a standard configuration.

DEVID is accessible as follows:

Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register that returns an IMPLEMENTATION DEFINED value.

B2.3.6.1 Field Descriptions

The DEVID bit assignments are:



Bits[31:0]

IMPLEMENTATION DEFINED.

B2.3.6.2 Accessing DEVID

DEVID can be accessed at the following address:

Offset
0xFC8

B2.3.7 DEVID1, Device Configuration Register 1

The DEVID1 characteristics are:

Purpose Indicates the capabilities of the component.

Usage constraints

This register is IMPLEMENTATION DEFINED for each part number and designer.

The entire 32-bit field can be used because the data width is determined by the component itself.

Unused bits must be RES0.

If the component is configurable, Arm recommends that this register reflects any changes to a standard configuration.

DEVID1 is accessible as follows:

Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register that returns an IMPLEMENTATION DEFINED value.

B2.3.7.1 Field Descriptions

The DEVID1 bit assignments are:



Bits[31:0]

IMPLEMENTATION DEFINED.

B2.3.7.2 Accessing DEVID1

DEVID1 can be accessed at the following address:

Offset
0xFC4

B2.3.8 DEVID2, Device Configuration Register 2

The DEVID2 characteristics are:

Purpose Indicates the capabilities of the component.

Usage constraints

This register is IMPLEMENTATION DEFINED for each part number and designer.

The entire 32-bit field can be used because the data width is determined by the component itself.

Unused bits must be RES0.

If the component is configurable, Arm recommends that this register reflects any changes to a standard configuration.

DEVID2 is accessible as follows:

Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register that returns an IMPLEMENTATION DEFINED value.

B2.3.8.1 Field Descriptions

The DEVID2 bit assignments are:



Bits[31:0]

IMPLEMENTATION DEFINED.

B2.3.8.2 Accessing DEVID2

DEVID2 can be accessed at the following address:

Offset
0xFC0

B2.3.9 DEVTYPE, Device Type Identifier Register

The DEVTYPE characteristics are:

Purpose If the part number field is not recognized, a debugger can report the information that is provided by DEVTYPE about the component instead.

Usage constraints

DEVTYPE is accessible as follows:

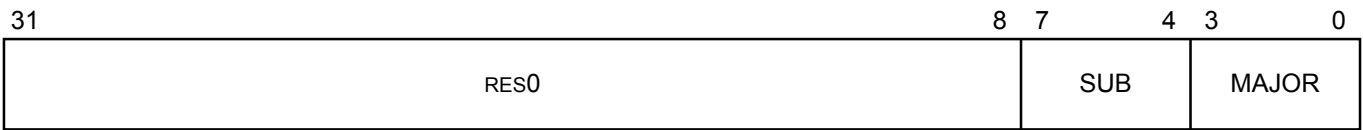
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register that returns an IMPLEMENTATION DEFINED value.

B2.3.9.1 Field Descriptions

The DEVTYPE bit assignments are:



Bits[31:8]

RES0.

SUB, bits[7:4]

Sub type for the component device type, as described in Table B2-30.

MAJOR, bits[3:0]

Major type for the component device type, as described in Table B2-30.

Table B2-30 Device type encoding

MAJOR type [3:0]		SUB type [7:4]	
Value	Description	Value	Description
0x0	Miscellaneous	0x0	Other, undefined.
		0x1-0x3	Reserved.
		0x4	Validation component.
		0x5-0xF	Reserved.
0x1	Trace Sink	0x0	Other.
		0x1	Trace port, for example TPIU.
		0x2	Buffer, for example ETB.
		0x3	Basic trace router.
		0x4-0xF	Reserved.
0x2	Trace Link	0x0	Other.
		0x1	Trace funnel, Router.

MAJOR type [3:0]		SUB type [7:4]	
Value	Description	Value	Description
0x3	Trace Source	0x2	Filter.
		0x3	FIFO, Large Buffer.
		0x4-0xF	Reserved.
		0x0	Other.
		0x1	Associated with a processor core.
		0x2	Associated with a DSP.
		0x3	Associated with a Data Engine or coprocessor.
		0x4	Associated with a Bus, stimulus-derived from bus activity.
		0x5	Reserved.
		0x6	Associated with software, stimulus-derived from software activity.
		0x7-0xF	Reserved.
0x4	Debug Control	0x0	Other.
		0x1	Trigger Matrix, for example ECT.
		0x2	Debug Authentication Module. See Control of authentication interfaces
		0x3	Power requester.
		0x4-0xF	Reserved.
0x5	Debug Logic	0x0	Other.
		0x1	Processor core.
		0x2	DSP.
		0x3	Data Engine or coprocessor.
		0x4	Bus, stimulus-derived from bus activity.
		0x5	Memory, tightly coupled device such as <i>Built In Self-Test</i> (BIST).
		0x6-0xF	Reserved.
0x6	Performance Monitor	0x0	Other.
		0x1	Associated with a processor.
		0x2	Associated with a DSP.
		0x3	Associated with a Data Engine or coprocessor.
		0x4	Associated with a bus, stimulus-derived from bus activity.
		0x5	Associated with a Memory Management Unit that conforms to the Arm System MMU Architecture.
		0x6-0xF	Reserved.
0x7-0xF	Reserved	-	-

B2.3.9.2 Accessing DEVTYPE

[DEVTYPE](#) can be accessed at the following address:

Offset
0xFCC

B2.3.10 ITCTRL, Integration Mode Control Register

The ITCTRL characteristics are:

Purpose A component can use this register to dynamically switch between functional mode and integration mode.

In integration mode, topology detection is enabled. For more information, see [Topology Detection](#).

Usage constraints

After switching to integration mode and performing integration tests or topology detection, reset the system to ensure correct behavior of CoreSight and other connected system components.

ITCTRL is accessible as follows:

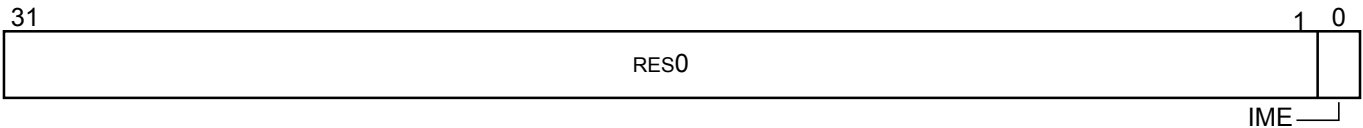
Default
RW

Configurations Included in all implementations.

Attributes A 32-bit register.

B2.3.10.1 Field Descriptions

The ITCTRL bit assignments are:



Bits[31:1]

RES0.

IME, bits[0]

Permitted values of IME are:

- | | |
|---|---|
| 0 | The component must enter functional mode. |
| 1 | The component must enter integration mode, and enable support for topology detection and integration testing. |

When no integration functionality is implemented, this field is RES0.

B2.3.10.2 Accessing ITCTRL

ITCTRL can be accessed at the following address:

Offset
0xF00

B2.3.11 LSR and LAR, Software Lock Status Register and Software Lock Access Register

The characteristics of the Software lock registers LSR and LAR are:

Purpose The Software lock mechanism prevents accidental access to the registers of CoreSight components. Software that is being debugged might accidentally write to memory used by CoreSight components. Accidental accesses might disable those components, making the software impossible to debug. The CoreSight programmers' model includes a Lock Status Register, [LSR](#), and a Lock Access Register, [LAR](#), to control software access to CoreSight components to ensure that the likelihood of accidental access to CoreSight components is small.

Note

From CoreSight version 3.0 onwards, implementation of the Software lock mechanism that is controlled by [LAR](#) and [LSR](#) is deprecated.

To ensure that the software being debugged can never access an unlocked CoreSight component, a software monitor that accesses debug registers must unlock the component before accessing any registers, and lock the component again before exiting the monitor.

Arm recommends that external accesses from a debugger are not subject to the Software lock, and therefore that external reads of the [LSR](#) return zero. For information on how CoreSight components can distinguish between external and internal accesses, see [Debug APB interface memory map](#).

A system can include several bus Requesters capable of accessing the same CoreSight component, for example in systems that include several processors. In this case, it is possible for software running on one processor, processor A, to accidentally access the component while it is being programmed by a debug monitor running on another processor, processor B. Because the component that is being accessed cannot distinguish between the two processors, processor A might disable the component and cause problems for processor B. The probability of this occurring is low, but must be considered if there are special circumstances that make this scenario more likely.

Note

The claim tag cannot be used to manage accesses to the Software lock registers, because access to the claim tag is subject to the Software lock mechanism.

Usage constraints

[LSR](#) and [LAR](#) are accessible as follows:

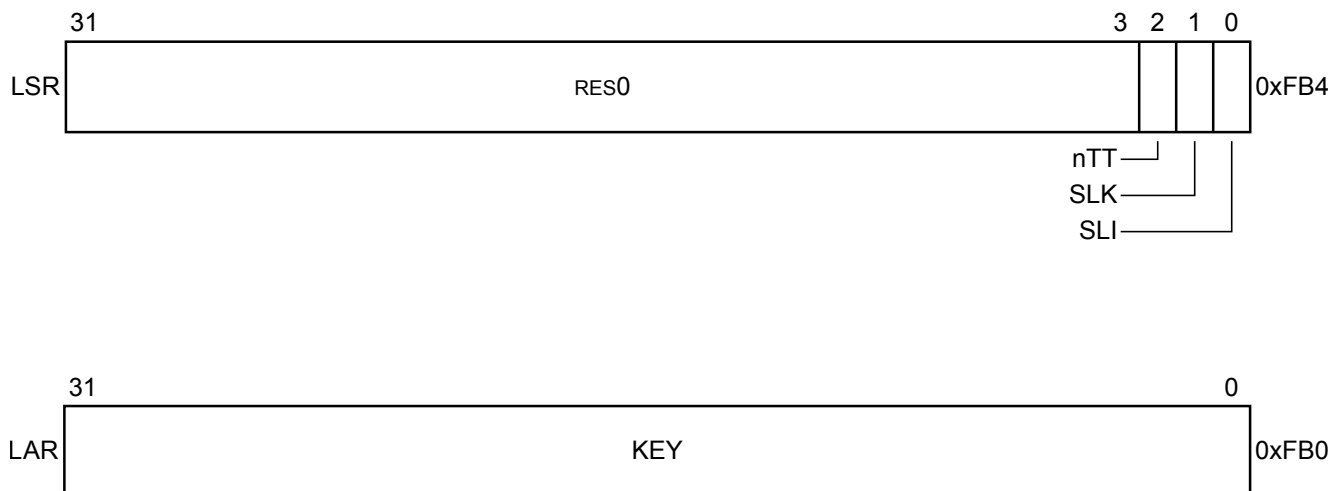
Default	
LSR	LAR
RO	WO

Configurations [LSR](#) is included in all implementations, and [LSR.SLI](#) indicates whether [LAR](#) is implemented.

Attributes 32-bit registers.

B2.3.11.1 Field Descriptions

The [LSR](#) and [LAR](#) bit assignments are:



LSR, bits[31:3]

RES0.

nTT, LSR bits[2]

This bit is always zero, which indicates that the component implements a 32-bit [LAR](#).

SLK, LSR bits[1]

This field is used to return the current software lock status.

Permitted values of SLK are:

- | | |
|---|---|
| 0 | Writing to the other registers in the component is permitted. |
| 1 | Writing to the other registers in the component is blocked. |

Note

When present, the reset value of this bit is 1.

SLI, LSR bits[0]

This field indicates whether a Software lock mechanism is implemented.

Permitted values of SLI are:

- | | |
|---|---|
| 0 | Software lock mechanism is not implemented. |
| 1 | Software lock mechanism is implemented. |

Note

Some components have two programmable views, one only visible from external tools and the other visible from software running on-chip. In this case:

- For accesses from external tools, the Software lock mechanism is not required and [LSR.SLI](#) and [LSR.SLK](#) both return a value of zero.
 - For accesses from software running on-chip the Software lock is optional, and, when implemented, [LSR.SLI](#) has the value 0b1 and [LSR.SLK](#) returns the status of the Software lock.
-

KEY, LAR bits[31:0]

Writing a value to this field controls write access to the other registers in the component.

Permitted values of KEY are:

Write 0xC5ACCE55

Signals that [LSR](#) must permit writing to the other registers in the component.

Write any other value

Signals that [LSR](#) must block writing to the other registers in the component.

B2.3.11.2 Accessing LSR and LAR

[LSR](#) and [LAR](#) can be accessed at the following address:

Offset	
LAR	LSR
0xFB0	0xFB4

B2.4 Component-specific registers for Class 0xF CoreLink, PrimeCell, and system components

Components that have the value 0xF assigned to the [CIDR1.CLASS](#) field in the Component Identification Register are CoreLink, PrimeCell, or system components. For details, see [CIDR0-CIDR3, Component Identification Registers](#).

CoreLink, PrimeCell, and system components are not related to the CoreSight system.

No component-specific registers are specified for this component class.

Chapter B3

Topology Detection

This chapter describes the CoreSight topology detection registers. It contains the following sections:

- [About topology detection.](#)
- [Requirements for topology detection signals.](#)
- [Combination with integration registers.](#)
- [Interfaces that are not connected or implemented.](#)
- [Variant interfaces.](#)
- [Documentation requirements for topology detection registers.](#)

B3.1 About topology detection

CoreSight system components can have various interface types. A component specifies which interfaces are present, and whether they act as Transmitter or Receiver. Each interface type defines a set of control signals that enable a debugger to determine which other components are connected to it. These signals are referred to as topology detection signals. During topology detection, a debugger probes each interface to determine which other components are connected to it.

For the specification of the requirements for the topology detection signals for standard interfaces that are used by Arm CoreSight components, see [Topology Detection at the Component Level](#). Interface vendors must define the requirements for other interfaces, following the rules in [Topology Detection at the System Level](#).

B3.2 Requirements for topology detection signals

Topology detection signals must observe the following requirements:

- For each topology detection input, it must be possible to read the state of that input.
- For each topology detection output, it must be possible to drive the state of that output without affecting other topology detection signals.

Note

It is not necessary to implement topology detection registers on the programming interface for the component, because this connectivity is described by the ROM Table.

Topology detection can be invasive. See [Topology Detection at the System Level](#).

B3.2.1 Recommended method

Arm recommends that topology detection registers are implemented as follows:

- Implement a topology detection mode that isolates the topology detection signals.
- For each topology detection output, provide a register that sets the value of that output in topology detection mode.
- For each topology detection input, provide a register that returns the value of that input in topology detection mode.

B3.3 Combination with integration registers

In addition to the registers that are required for topology detection, many components implement integration registers that provide the same control over most inputs and outputs. This technique enables rapid integration testing when validating a SoC built from these components, because a test bench can assess the connectivity between two components without knowledge of their underlying functionality.

For components that implement integration registers, Arm recommends reusing these registers for topology detection. Use the [ITCTRL](#) register to select both integration mode and topology detection mode. See also [ITCTRL](#), [Integration Mode Control Register](#).

B3.4 Interfaces that are not connected or implemented

Some components do not implement a fixed number of interfaces to allow for the possibility of interfaces not being connected. To the debugger, there is no difference between an interface that is not present and one that is not connected.

If the component requires that the interface is still usable when connected to a non-CoreSight component that is not capable of topology detection, the programmers' model must indicate whether the interface is connected or not.

If the component can only be connected to other CoreSight components, the tools can assume that the interface does not exist if they fail to find any connected interfaces during topology detection. In this case, the programmers' model does not need to indicate whether the interface is connected, but if it does, some time can be saved during topology detection.

B3.5 Variant interfaces

Usually, the connections between interfaces do not change after detection. However, sometimes it is necessary for some components to share a component. For example, a component tracing the operation of a processor might switch to tracing the operation of a different processor. It is important that the conditions under which a switch can occur are understood.

The connections between interfaces can only change if all the following conditions apply:

- An interface is defined as being variant between multiple connections.
- The programmers' model of the affected component controls the configuration by selecting between several alternative connections for that interface.
- The number of valid alternative connections that are indicated in the programmers' model, which is used to reduce the autodetection time, is less than 32, and remains constant during switching.

If these conditions are too stringent for your application, a separate CoreSight component that multiplexes the connections is required. Topology detection can then be performed between this new component and the components it is connected to.

When a switch has occurred, topology detection must be repeated to determine the new connections. Because topology detection can be invasive, Arm recommends performing topology detections for all configurations that are likely to occur in advance.

B3.5.1 External multiplexing

Figure B3-1 shows an example of how variable connections can be implemented using an external multiplexer. This example shows:

- A register that indicates that there are n inputs to select from. This register can be read by a debugger to determine which values of the selection register are valid. The register is tied to the value n outside the component.
- A selection register that selects the input to use.
- A variant connection receiving the selected input.

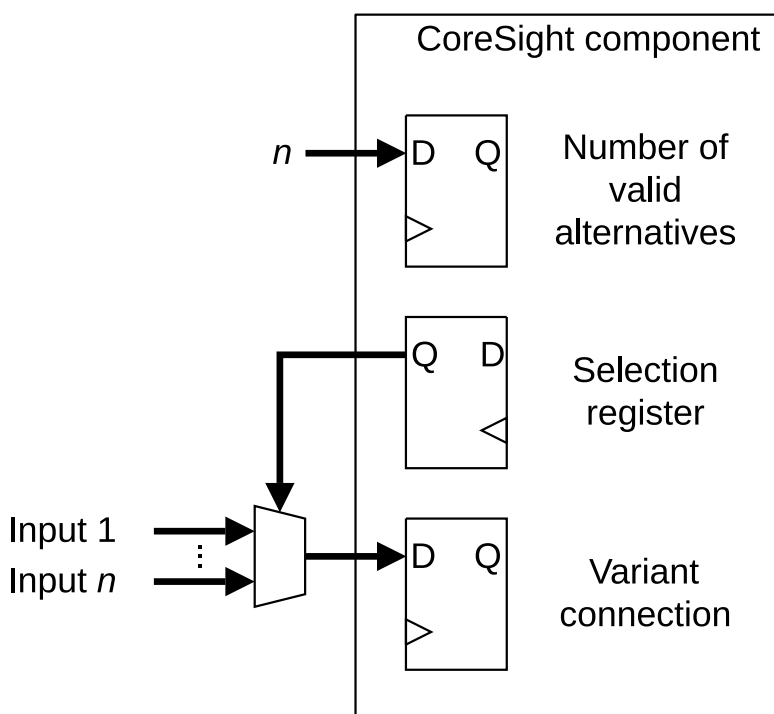


Figure B3-1 External multiplexing of connections

B3.6 Documentation requirements for topology detection registers

The component must have documentation that defines the interfaces present on that component. The definition of each interface must include:

- Its name, using the format that is listed in [Topology Detection at the System Level](#).
- If the interface supports variable connections:
 - How many connections are valid.
 - How to switch between connections.
- How to control the topology detection signals listed for that interface in [Topology Detection at the System Level](#).

B3.6.1 Interfaces where topology detection is not possible

If an interface can be connected to a non-CoreSight component, topology detection might not be possible. In this case, the documentation must define a method to determine from the programmers' model how such as component is connected.

Part C **CoreSight Reusable Component Architecture**

Chapter C1

About the Reusable Component Architecture

The CoreSight reusable component architecture specifies the rules that enable a component to be used with other components that use the CoreSight architecture, and defines the physical interfaces of the components so that they can be connected together easily.

Note

Unlike the visible component architecture, implementing the reusable component architecture is not mandatory. Omitting the reusable component architecture from a self-contained system does not compromise its compatibility with debuggers, but prevents it from being used with other CoreSight components.

If a component does not require the functionality that is provided by a particular interface, implementing the interface is optional. For example, a component with no programmable registers does not need to implement the AMBA APB interface.

It is possible to create a component that performs several functions internally, while presenting only one set of reusable component interfaces. Doing so allows implementing pre-built platforms with an integrated CoreSight infrastructure, enabling the platform to be integrated into a larger system as if it were a single CoreSight component.

The reusable component architecture is described in the following chapters:

- [AMBA APB and ATB Interfaces.](#)
- [Event Interface.](#)
- [Channel interface.](#)
- [Authentication Interface.](#)
- [Timestamp Interface.](#)
- [Topology Detection at the Component Level.](#)

Chapter C2

AMBA APB and ATB Interfaces

This chapter describes the following AMBA interfaces:

- The [AMBA APB interface](#), which is used to program CoreSight components.
- The [AMBA ATB interface](#), which transfers trace data.

C2.1 AMBA APB interface

The following sections describe the AMBA APB interface:

- [About the AMBA APB interface.](#)
- [AMBA APB interface signals.](#)
- [AMBA APB interface width.](#)
- [Alternative views of the register file.](#)

C2.1.1 About the AMBA APB interface

The AMBA APB interface is used to program CoreSight components.

The interface supports:

- Simple, non-pipelined operation.
- Implementation of 8-bit, 16-bit, or 32-bit Completers.
- Stalling by the Completer.
- Error responses from the Completer.

For more information, see the *Arm® AMBA® APB Protocol Specification*.

Some legacy debug components implement a JTAG TAP Controller to access their functionality.

The bus that connects all CoreSight components is referred to as the Debug APB interface.

C2.1.2 AMBA APB interface signals

[Table C2-1](#) shows the signals that comprise the AMBA APB interface.

-
- Note**
- The signal suffix **DBG** indicates that the Debug APB interface is used to access CoreSight components.
 - The clamp value is the value that an output must be clamped to when the component is powered down or disabled.

For more information, see the *Arm® AMBA® APB Protocol Specification*.

Table C2-1 Signals on the Debug APB interface

Name	Direction		Clamp value	Description
	Requester	Completer		
PCLKDBG	Input	Input	-	The rising edge of PCLKDBG synchronizes all transfers on the AMBA 3 APB interface.
PRESETDBGn	Input	Input	-	This signal resets the interface and is active-LOW.
PADDRDBG[31:2]	Output	Input	0	This bus indicates the address of the transfer. It is not necessary to implement unused bits. ^a
PSELDBG	Output	Input	0	This signal indicates that the Completer device is selected and a data transfer is required. There is a PSELDBG signal for each Completer.
PENABLEDBG	Output	Input	0	This signal indicates the second and subsequent cycles of an AMBA APB interface transfer.

Name	Direction		Clamp value	Description
	Requester	Completer		
PWRITEDBG	Output	Input	0	When HIGH, PWRITEDBG indicates a write access. When LOW, it indicates a read access.
PWDATADB[31:0]	Output	Input	0	PWDATADB[31:0] is the write data bus. When PWRITEDBG is HIGH, it indicates that the write data bus is driven by the Requester during write cycles. The write data bus can be up to 32-bits wide.
PREADYDBG	Input	Output	1	This signal is used by the Completer to extend an AMBA APB interface transfer.
PRDATADB[31:0]	Input	Output	0	PRDATADB[31:0] is the read data bus. When PWRITEDBG is LOW, it indicates that the read data bus is driven by the selected Completer during read cycles. The read data bus can be up to 32-bits wide.
PSLVERRDBG	Input	Output	1	This signal is returned in the second cycle of the transfer, and indicates an error response. Only use this signal for indicating that a component is not available, for example because it is powered down.

a. The use of PADDRDBG[31] to split the memory map and indicate the difference between external and internal accesses is deprecated. For information on how components that require it can differentiate between external and internal access, see [Debug APB interface memory map](#).

C2.1.3 AMBA APB interface width

The AMBA APB interface is 32-bits wide.

[CoreSight programmers' model](#) describes the model compatible with a 32-bit AMBA APB interface.

C2.1.4 Alternative views of the register file

There might be several ways to access the registers of a component. It can be useful, for example, to provide special instructions to make debug registers in a processor visible as registers in a linked coprocessor. Provided the debug functionality of the component is also accessible through the AMBA APB interface, alternative methods to access registers are permitted.

C2.2 AMBA ATB interface

The AMBA ATB interface carries trace data around a SoC.

Every CoreSight component or platform with trace capabilities has an AMBA ATB interface, and is either a Transmitter or Receiver on the AMBA ATB:

- A component or system that generates trace data is a Transmitter.
- A component or system that receives trace data is a Completer.

The AMBA ATB interface supports the following features:

- Stalling of data, using valid and ready responses.
- Byte-sized packets, together with control signals to indicate the number of bytes that are valid in a cycle.
- Originating component marker, giving each data packet an associated ID.
- Any trace protocol or data agnostic requirements for the format of the data.
- Check-pointing of data from all originating components.

For more information, see the *Arm[®] AMBA[®] ATB Protocol Specification*.

Chapter C3

Event Interface

A CoreSight system uses the event interface to transfer events between components. It is most commonly used for communicating cross-trigger events between debug components and a *Cross Trigger Interface* (CTI).

The event interface signals are:

EVENTCLK Clock. This signal is typically mapped onto an existing clock signal.

EVENTRESETn

Reset. This signal is typically mapped onto an existing reset signal.

EVENT This signal indicates the event, and is typically mapped onto a signal of a different name that describes its purpose.

An event is indicated by a rising edge on **EVENT**. Therefore an event can be signaled at most once every two **EVENTCLK** cycles.

If the event has a duration, the falling edge on **EVENT** indicates its completion.

The interface defines no back-pressure mechanism, so events that are close together might merge. For example, if the event interface crosses an asynchronous boundary to a slower clock domain, events in close succession might merge into a single event.

The source of an event interface is known as an event interface transmitter. The destination of an event interface is known as an event interface receiver.

Chapter C4

Channel interface

This chapter describes the channel interface. It contains the following sections:

- [About the channel interface.](#)
- [Channels.](#)
- [Channel interface signals.](#)
- [Channel connections.](#)
- [Synchronous and asynchronous conversions.](#)

C4.1 About the channel interface

The channel interface is a special type of event interface that enables CoreSight components to communicate events, as described in [Event Interface](#). The channel interface supports:

- A variable number of event channels.
- Bidirectional communication.
- Synchronous or asynchronous communication.

Some examples of useful events are:

- If two or more processors are required to stop at the same time, they must signal to each other when they have stopped.
- To perform advanced profiling functions, profiling events from many different sources in the system must be shared.

[Figure C4-1](#) shows a channel interface that connects multiple CTIs, which are provided by CoreSight technology. The channel interface can be supported directly by a CoreSight component, if necessary.

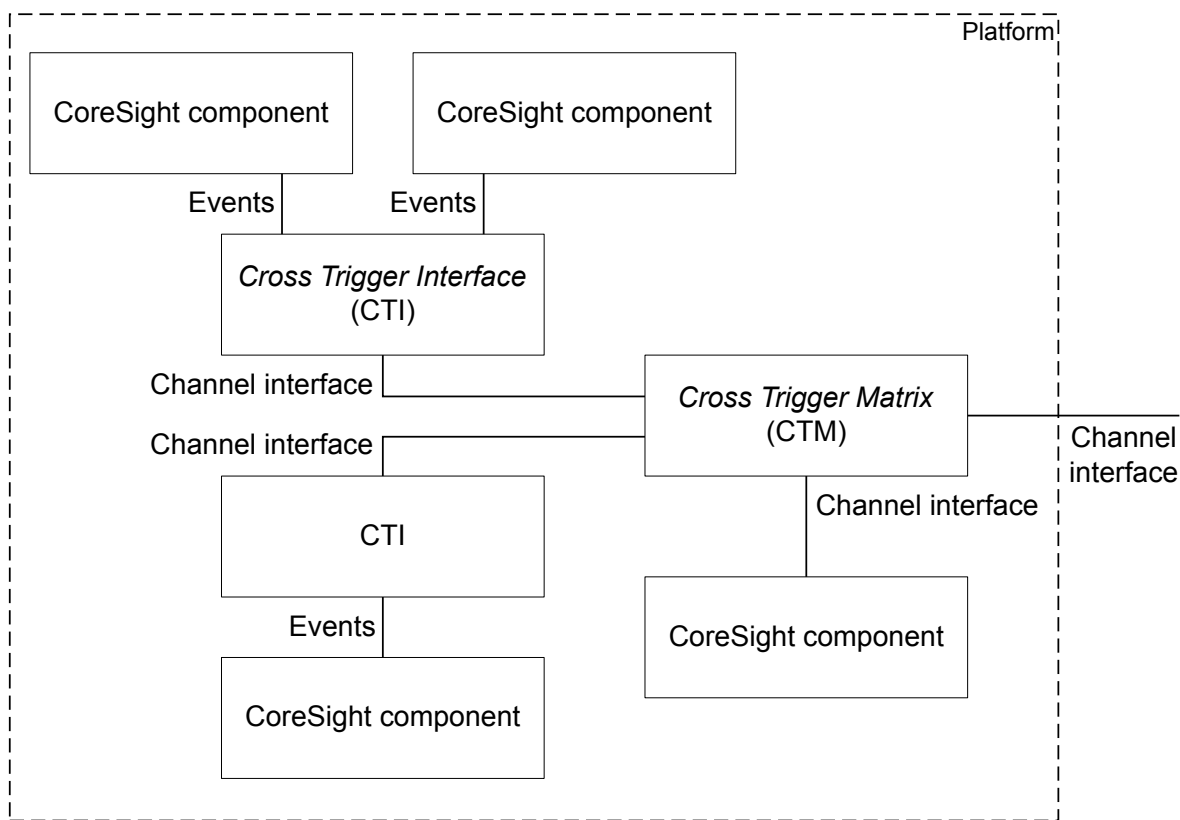


Figure C4-1 Implementation of a CTI-based channel interface

Note

When using the CTI:

- Some systems require more event signals than are supported by a CTI.
- In a platform-oriented system, it is necessary to connect event signals together within the platform and export only a set of standard interfaces for extension at higher levels.

The channel interface is designed to enable components to communicate events with minimal overhead. However, if multiple events are presented to the channel interface in close succession, they might be interpreted as a single event. [Figure C4-2](#) illustrates this limitation for a situation where events that are generated in a fast clock domain, Clock A, are passed to a slower clock domain, Clock B. In Clock A, two separate events can be seen, but these events are too close together for Clock B, resulting in Clock B interpreting them as a single event.

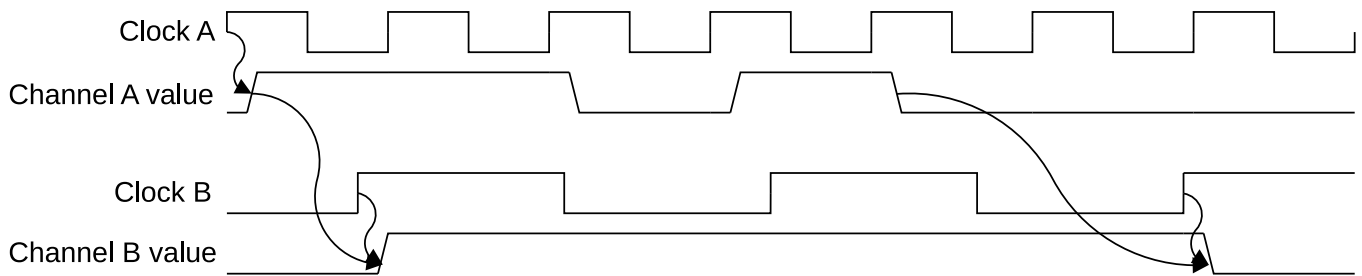


Figure C4-2 Event merging in the channel interface

This limitation makes the channel interface unsuitable for counting events that occur in rapid succession. An example of this type of operation is counting the number of instructions that are executed by a processor over time.

The channel interface is suitable for the following:

- Transmission of an event that happens only once, for example a trigger signal to an ETB or TPIU to end trace capture.
- Transmission of a low-speed signal level where precision is not important.
- Transmission of a signal subject to handshaking using another channel in the channel interface.
- Transmission of a signal subject to software handshaking, for example an interrupt request.
- Transmission of events to be counted that do not occur close together, for example the number of times a peripheral causes an interrupt.

C4.2 Channels

The channel interface comprises two types of signals:

- Channel outputs, which transmit events that are generated by a component.
- Channel inputs, which listen for events that are generated by other components.

A component uses its channel outputs to transmit events to the channel inputs of all components in the system, except its own.

The interface supports an IMPLEMENTATION DEFINED number of channels. Arm recommends that at least four channels are implemented.

Components must treat all channels identically. It must be possible for the debugger to control which channels are used for which purposes.

If a system consists of subsystems with different numbers of channels, and there is a requirement to pass events between these subsystems, the following rules must be observed:

- A subset of the channels from the subsystem with the greater number of channels is connected to all the channels in the other subsystem.
- The set of channels that is connected is always a contiguous set, starting from channel 0.

For example, in a system where subsystem A has eight channels and subsystem B has four channels, channels 0-3 from subsystem A are connected to channels 0-3 in subsystem B. Channels 4-7 are not connected to subsystem B.

C4.3 Channel interface signals

Table C4-1 shows the set of signals that are required by an asynchronous channel interface. The clamp value is the value that an output must be clamped to when the component is powered down or disabled.

Table C4-1 Asynchronous channel interface signals

Name	Direction	Clamp value	Description
CHIN[n-1:0]	Input	-	Channel input
CHINACK[n-1:0]	Output	1	Channel input acknowledge
CHOUT[n-1:0]	Output	0	Channel output
CHOUTACK[n-1:0]	Input	-	Channel output acknowledge

Figure C4-3 shows how the asynchronous interface uses a basic four-phase handshaking protocol. The same protocol is used by CHOUT and CHOUTACK.

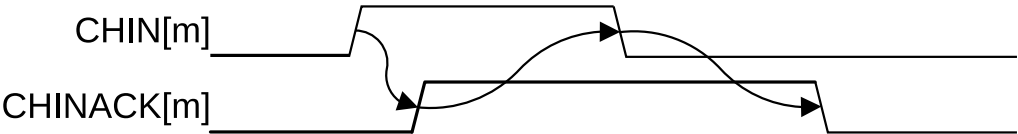


Figure C4-3 Channel interface handshaking

Table C4-2 shows the set of signals that are required by a synchronous channel interface.

Table C4-2 Synchronous channel interface signals

Name	Direction	Clamp value	Description
CHCLK	Input	-	Clock
CHIN[n-1:0]	Input	-	Channel input
CHOUT[n-1:0]	Output	0	Channel output

C4.4 Channel connections

The channel interface is bidirectional. Take care to connect the correct signals together. [Figure C4-4](#) shows how to connect two asynchronous channel interfaces together.

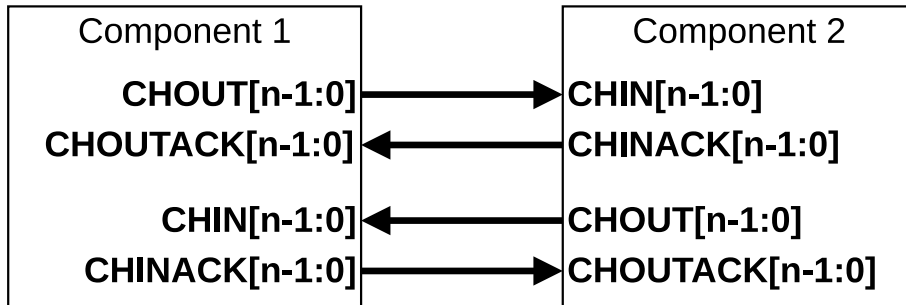


Figure C4-4 Asynchronous channel interface connection

[Figure C4-5](#) shows how to connect two synchronous channel interfaces together.

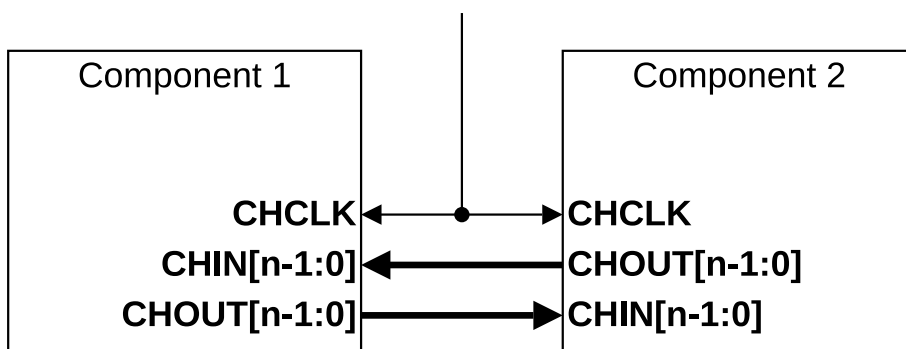


Figure C4-5 Synchronous channel interface connection

A component can support:

- Only the input channels, which is appropriate for components that do not generate events, but have to react to events from other components.
- Only the output channels, which is appropriate for components that generate events, but do not have to react to events from other components.
- Both the input and output channels.

If a component does not support both sets of channels, the unsupported outputs must be clamped as shown in [Table C4-1](#) and [Table C4-2](#).

If a component supports both input and output channels, the component must not reflect events on an input channel to the corresponding output channel.

C4.5 Synchronous and asynchronous conversions

Figure C4-6 shows a circuit that makes it possible to convert between synchronous and asynchronous versions of this interface.

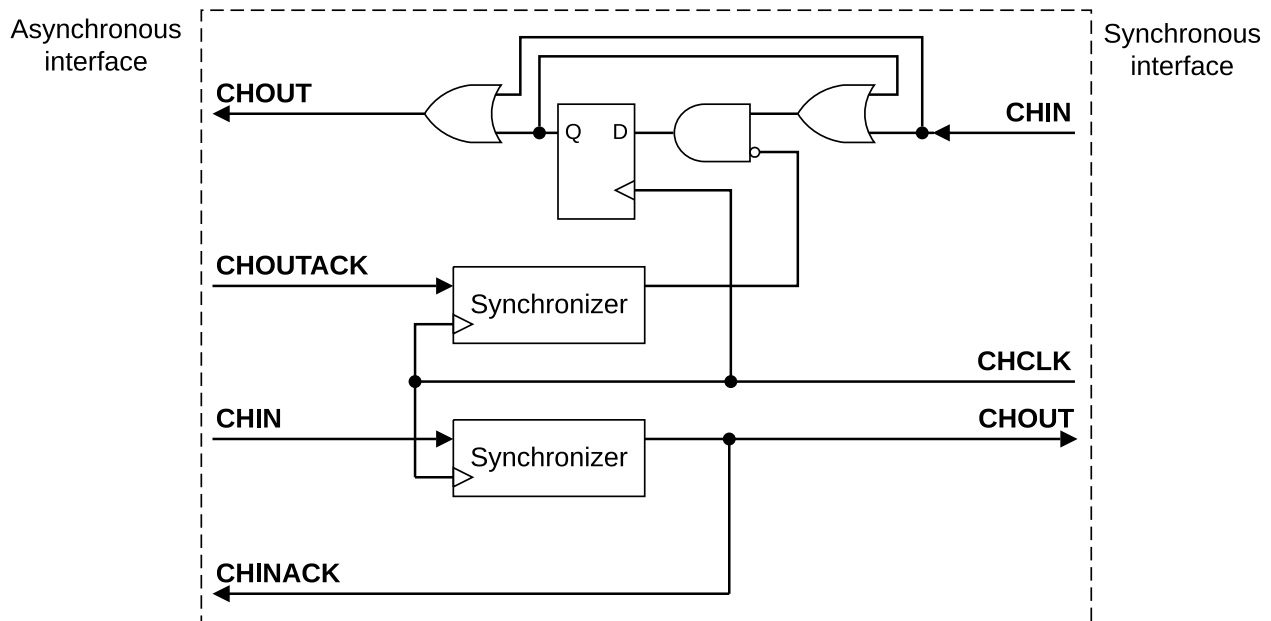


Figure C4-6 Asynchronous to synchronous converter

Implementing a synchronous to asynchronous converter increases the likelihood of events being merged as described in [About the channel interface](#).

Chapter C5

Authentication Interface

This chapter defines the system requirements that control access to debug and trace peripherals, and how those requirements are met by devices that comply with the CoreSight architecture. It contains the following sections:

- [About the authentication interface.](#)
- [Definitions of different types of debug.](#)
- [Authentication interface signals.](#)
- [Authentication rules.](#)
- [User mode debugging.](#)
- [Control of the authentication interface.](#)
- [Exemptions from implementing the authentication interface.](#)

C5.1 About the authentication interface

Use the authentication interface for restricting access to debug and trace functionality in the following ways:

- To prevent unauthorized people from modifying the behavior of the system, for example to prevent a mobile phone from reporting a fake identification number to the network. This requires authenticated access to invasive debug functions such as traditional core debug, but permits non-invasive tracing and profiling functions.
- To prevent unauthorized people from reverse engineering a product or discovering secrets that are stored within it, for example to read encryption keys. This requires authenticated access to all debug and trace functions.

The authentication interface does not prevent accidental access of debug functionality by rogue code, making a system impossible to debug. This type of access is managed by the Software lock mechanism that is optional in all CoreSight components, see [LSR and LAR, Software Lock Status Register and Software Lock Access Register](#).

C5.2 Definitions of different types of debug

This section defines different types of debug, including the debug of different Security states and invasive versus non-invasive debug.

C5.2.1 Definition of debug of a Security state

A debug operation associated with a particular Security state is any operation where instructions executing on-chip with privileges of that Security state would be able to perform a similar operation.

Examples include:

- A Non-secure debug operation where the operation can observe execution of Non-secure instruction execution.
- A Secure debug operation where the operation can observe execution of Secure instruction execution. Since Secure instruction execution is generally more privileged than Non-secure execution, a Secure debug operation might also be able to observe Non-secure instruction execution.
- In a system that supports the Arm Realm Management Extension (FEAT_RME), a Root debug operation usually has the highest privileges, and a Root debug operation that can observe Root instruction execution might also be able to observe instruction execution of other Security states.
- A Non-secure debug operation where the operation can access Non-secure memory. Such an operation generally cannot access memory of any other Security state.
- A Secure debug operation where the operation can access Secure memory. Such an operation generally cannot access memory of any other Security state.
- A Root debug operation where the operation can access Root memory. Such an operation generally cannot access memory of any other Security state. To access other memory of Security states, a debug operation corresponding to that Security state is needed.

Debug operations that monitor the time that is taken by execution in a more privileged Security state are generally considered to be debug operations for the lowest Security state, because the time taken can be measured by combining off-chip timing information with on-chip event generation information. Operations that affect the time that is taken by execution in a higher privileged Security state are considered debug operations for the higher privileged Security state.

C5.2.2 Definition of hypervisor debug

The meaning of hypervisor debug is IMPLEMENTATION DEFINED. For processors based on Arm architectures, see the relevant Arm® Architecture Reference Manual.

C5.2.3 Definition of invasive debug

Any operation that changes the defined behavior of the system is invasive.

Examples include any changes to the contents of memory and insertion of instructions into a processor pipeline, but not necessarily the act of changing the number of cycles that are taken to perform an operation, unless the number of cycles is defined architecturally.

An implementation can treat an IMPLEMENTATION DEFINED set of effects that change the observable, but not the defined behavior of the system, as invasive. Examples include most effects that change the number of cycles that are taken to perform an operation.

C5.3 Authentication interface signals

Table C5-1 shows the authentication interface signals that a component might support. If a component uses a non-invasive enable signal, it must import the invasive equivalent. For example, using **SPNIDEN** requires importing **SPIDEN**, and using **HNIDEN** requires importing **HIDEN**.

Table C5-1 Authentication interface signals

Signal	Description
AUTHCLK	Clock (Not used for asynchronous authentication) ^a
AUTHRESETn	Reset (Not used for asynchronous authentication) ^a
DBGEN	Invasive debug enable
NIDEN	Non-invasive debug enable
SPNIDEN	Secure non-invasive debug enable
SPIDEN	Secure invasive debug enable
HIDEN	Hypervisor invasive debug enable
HNIDEN	Hypervisor non-invasive debug enable
RLPIDEN	Realm invasive debug enable
RTPIDEN	Root invasive debug enable
NSPPIDEN	Non-secure Protected invasive debug enable
SAPIDEN	System Agent invasive debug enable

a. Use of the asynchronous authentication interface is deprecated.

The source of an authentication interface is known as the authentication interface Transmitter.

The destination of an authentication interface is known as an authentication interface Receiver.

C5.4 Authentication rules

Authentication interface implementations must observe the following rules:

1. Synchronous interfaces must sample all signals synchronously, on the rising edge of **AUTHCLK**. Typically, **AUTHCLK** is mapped onto another clock signal. It is **IMPLEMENTATION DEFINED** when a change of any of the authentication signals takes effect. For example, a processor core might ignore changes to the authentication signals while in Debug state. By extension, it is possible that a component only observes the signals on reset, but it is recommended that more frequent changes are permitted.
Asynchronous interfaces must sample all signals asynchronously.
Arm recommends that processors implementing the authentication interface specify a sequence of instructions that, when executed, wait until changes to the authentication signals have taken effect before continuing.
2. If **DBGEN** is LOW, invasive debug is not permitted.
Invasive debug is any debug operation that might cause the behavior of the system to be modified. Non-invasive debug, such as trace, is unaffected.
3. If **NIDEN** is LOW and **DBGEN** is LOW, neither invasive nor non-invasive debug is permitted.
4. If **NIDEN** is LOW and **DBGEN** is HIGH, both invasive and non-invasive debug are permitted. Arm recommends that these signals are not driven in this way.
To ensure that a non-invasive component is correctly enabled, it must import both **DBGEN** and **NIDEN**, and internally OR the result.
5. If **SPIDEN** is LOW, Secure invasive debug is not permitted.
6. If **SPNIDEN** is LOW and **SPIDEN** is LOW, all Secure debug is not permitted.
7. If **SPNIDEN** is LOW and **SPIDEN** is HIGH both invasive and non-invasive Secure debug are permitted. Arm recommends that these signals are not driven in this way. To ensure that a non-invasive component is correctly enabled, it must import **SPIDEN** in addition to **SPNIDEN**, and internally OR the result.

Note

Rules 5 to 7 are the equivalent of rules 2 to 4, but used for Secure debug. They are useful for systems that separate Secure and Non-secure data, for example systems implementing Arm Security Extensions. Secure non-invasive debug is any debug operation that enables a debugger to read Secure data. Secure invasive debug is any debug operation that enables a debugger to change Secure data. If a debug component supports Secure non-invasive debug functions by implementing the signal **SPNIDEN**, it must also observe the Secure invasive signal, **SPIDEN**.

8. If **SPIDEN** is HIGH and **DBGEN** is LOW, invasive debug is not permitted. Arm recommends that these signals are not driven in this way. To ensure that a component that supports Secure invasive debug is correctly controlled, Arm recommends importing both **DBGEN** and **SPIDEN**, and using the result of an internal AND operation with the imported signals as operands for authentication.
9. If **SPNIDEN** is HIGH and **NIDEN** is LOW, debugging is not permitted. Arm recommends that these signals are not driven in this way. To ensure that a component that supports Secure non-invasive debug is correctly controlled, it must import **NIDEN** in addition to **SPNIDEN**, and internally AND the result.
10. If **HIDEN** is LOW, hypervisor invasive debug is not permitted.
11. If **HNIDEN** is LOW and **HIDEN** is LOW, all hypervisor debug is not permitted.
12. If **HNIDEN** is LOW and **HIDEN** is HIGH both invasive and non-invasive hypervisor debug are permitted. Arm recommends that these signals are not driven in this way. To ensure that a non-invasive component is correctly enabled, it must import **HIDEN** in addition to **HNIDEN**, and internally OR the result.
13. If a component supports **HNIDEN**, it must also support **HIDEN** and **NIDEN**.
14. If a component supports **HIDEN**, it must also support **DBGEN**.
15. If **HIDEN** is HIGH and **DBGEN** is LOW, invasive debug is not permitted. Arm does not recommend that these signals are driven in this way. To ensure that a component that supports hypervisor invasive debug is correctly controlled, Arm recommends importing both **DBGEN** and **HIDEN**, and using the result of an internal AND operation with the imported signals as operands for authentication.
16. If **HNIDEN** is HIGH and **NIDEN** is LOW, debugging is not permitted. Arm does not recommend that these signals are driven in this way. To ensure that a component that supports Secure non-invasive debug is correctly controlled, it must import **NIDEN** in addition to **HNIDEN**, and internally AND the result.
17. If the value of any of the authentication signals changes, it is **IMPLEMENTATION DEFINED** when it takes effect.
Pipeline effects mean that it is not possible for these signals to be precise. Arm recommends not to use them to enable and disable debugging around specific regions of code without a full understanding of the pipeline behavior of the system.

18. The authentication interface provides two signals to define invasive debug for Root and Realm:

- **RLPIDEN**.
- **RTPIDEN**.

Note

Equivalent signals that define when Root and Realm non-invasive debug are permitted are not defined.

Root non-invasive debug is only enabled when Root invasive debug is enabled.

Realm non-invasive debug is only enabled when Realm invasive debug is enabled.

See [Root and Realm signals](#).

19. The authentication interface provides two signals to define Non-secure Protected and System Agent invasive debug:

- **NSPPIDEN**, to define when Non-secure Protected invasive debug is permitted.
- **SAPIDEN**, to define when System Agent invasive debug is permitted.

Note

Equivalent signals that define when Non-secure Protected and System Agent non-invasive debug are permitted are not defined.

Non-secure Protected non-invasive debug is only enabled when Non-secure Protected invasive debug is enabled.

System Agent non-invasive debug is only enabled when System Agent invasive debug is enabled.

The authentication rules can be summarized as follows:

- **SPIDEN**, **DBGEN**, **SPNIDEN**, and **NIDEN** enable Secure invasive debug, Non-secure invasive debug, Secure non-invasive debug, and Non-secure non-invasive debug, respectively.
- Because invasive functionality requires non-invasive functionality to function correctly, if invasive debug is enabled, non-invasive debug must also be enabled.
- Secure functionality must be disabled if the corresponding Non-secure functionality is disabled.

The following signal combinations are not permitted and behave as if **RLPIDEN** = 0:

- **DBGEN** = 0 & **RLPIDEN** = 1

The following signal combinations are not permitted and behave as if **RTPIDEN** = 0:

- **DBGEN** = 0 & **RTPIDEN** = 1
- **RTPIDEN** = 1 & **RLPIDEN** = 0

[Table C5-2](#), [Table C5-3](#), [Table C5-4](#), and [Table C5-5](#) show the equations that define whether a particular level of debug functionality is permitted for a debug component that supports the authentication interface:

Table C5-2 Component without Secure debug capabilities

Debug functionality	Equation
Invasive debug	DBGEN
Non-invasive debug	DBGEN NIDEN

Table C5-3 Component with Secure debug capabilities

Debug functionality	Equation
Non-secure invasive debug	DBGEN
Non-secure non-invasive debug	DBGEN NIDEN
Secure invasive debug	DBGEN & SPIDEN
Secure non-invasive debug	(SPIDEN SPNIDEN) & (DBGEN NIDEN)

Table C5-4 Component with hypervisor debug capabilities

Debug functionality	Equation
Hypervisor invasive debug	HIDEN & DBGEN
Hypervisor non-invasive debug	(HIDEN HNIDEN) & (DBGEN NIDEN)

Table C5-5 Component with Root and Realm debug capabilities, when the component has Secure debug capabilities

Debug functionality	Equation
Root Invasive debug	DBGEN & RLPIDEN & SPIDEN & RTPIDEN
Realm Invasive debug	DBGEN & RLPIDEN

Table C5-6 Component with Root and Realm debug capabilities, when the component does not have Secure debug capabilities

Debug functionality	Equation
Root Invasive debug	DBGEN & RLPIDEN & RTPIDEN
Realm Invasive debug	DBGEN & RLPIDEN

Table C5-7 Component with Non-secure Protected and System Agent debug capabilities

Debug functionality	Equation
Non-secure Protected invasive debug	DBGEN & NSPPIDEN
System Agent invasive debug	DBGEN & SAPIDEN

The **SPIDEN** and **SPNIDEN** signals have no dependencies on the **HIDEN** and **HNIDEN** signals, and conversely.

The **NSPPIDEN** and **SAPIDEN** signals have no dependencies on the **RLPIDEN**, **RTPIDEN**, **SPIDEN**, or **SPNIDEN** signals, and conversely.

Table C5-8 shows the restrictions for **SPIDEN** and **SPNIDEN** and their effects. Numbers in brackets indicate the rules that apply in each case, S indicates Secure, and NS indicates Non-secure.

Table C5-8 Authentication signal restrictions for SPIDEN and SPNIDEN

SPIDEN	DBGEN	SPNIDEN	NIDEN	Valid signal combination	Invasive debug permitted		Non-invasive debug permitted	
					S	NS	S	NS
0	0	0	0	Yes	No (2,5)	No (2)	No (3,6)	No (3)
0	0	0	1	Yes	No (2,5)	No (2)	No (6)	Yes
0	0	1	0	No ^a (2)	No (2,5)	No (2)	No (3,6)	No (3)
0	0	1	1	Yes	No (2,5)	No (2)	Yes	Yes
0	1	0	0	No (4)	No (5)	Yes (4)	No (6)	Yes (4)
0	1	0	1	Yes	No (5)	Yes	No (6)	Yes
0	1	1	0	No (4)	No (5)	Yes (4)	Yes (4)	Yes (4)
0	1	1	1	Yes	No (5)	Yes	Yes	Yes
1	0	0	0	No (7)	No (2)	No (2)	No (3)	No (3)
1	0	0	1	No (7)	No (2)	No (2)	Yes (7)	Yes
1	0	1	0	No ^a (1,2)	No (2)	No (2)	No (3)	No (3)
1	0	1	1	No ^a (1)	No (2)	No (2)	Yes	Yes
1	1	0	0	No (4,7)	Yes (7)	Yes (4)	Yes (4,7)	Yes (4)
1	1	0	1	No (7)	Yes (7)	Yes	Yes (7)	Yes
1	1	1	0	No (4)	Yes (4)	Yes (4)	Yes (4)	Yes (4)
1	1	1	1	Yes	Yes	Yes	Yes	Yes

a. These signal combinations were permitted in previous versions of the CoreSight architecture but are deprecated from v2.0 onwards.

Table C5-9 shows the restrictions for **HIDEN** and **HNIDEN** and their effects. Numbers in brackets indicate the rules that apply in each case, H indicates hypervisor, and NH indicates non-hypervisor.

Table C5-9 Authentication signal restrictions for HIDEN and HNIDEN

HIDEN	DBGEN	HNIDEN	NIDEN	Valid signal combination	Invasive debug permitted		Non-invasive debug permitted	
					H	NH	H	NH
0	0	0	0	Yes	No (2, 3)	No (2)	No (3)	No (3)
0	0	0	1	Yes	No (2, 3)	No (2)	No (3)	Yes
0	0	1	0	No (9)	No (2, 3)	No (2)	No (3)	No (3)
0	0	1	1	Yes	No (2, 3)	No (2)	Yes	Yes
0	1	0	0	No (4)	No (3)	Yes (4)	No (4)	Yes (4)
0	1	0	1	Yes	No (3)	Yes	No (4)	Yes
0	1	1	0	No (4)	No (3)	Yes (4)	Yes (4)	Yes (4)
0	1	1	1	Yes	No (3)	Yes	Yes	Yes

HIDEN	DBGEN	HNIDEN	NIDEN	Valid signal combination	Invasive debug permitted		Non-invasive debug permitted	
					H	NH	H	NH
1	0	0	0	No (7)	No (2)	No (2)	No (3)	No (3)
1	0	0	1	No (7)	No (2)	No (2)	Yes (5)	Yes
1	0	1	0	No (8, 9)	No (2)	No (2)	No (3)	No (3)
1	0	1	1	No (8)	No (2)	No (2)	Yes	Yes
1	1	0	0	No (4, 5)	Yes (5)	Yes (4)	Yes (4, 5)	Yes (4)
1	1	0	1	No (5)	Yes (5)	Yes	Yes (5)	Yes
1	1	1	0	No (4)	Yes (4)	Yes (4)	Yes (4)	Yes (4)
1	1	1	1	Yes	Yes	Yes	Yes	Yes

C5.4.1 Root and Realm signals

Arm® Realm Management Extension (RME) System Architecture and the *Arm® Architecture Reference Manual*, for *A-profile architecture* require that changes to the authentication status of the system only occur at specific times around a reset.

RTPIDEN effects are not permitted to change except when the system is in RME system reset.

To ensure an entire system observes the same value of **RTPIDEN** at all times, Arm recommends that the system samples **RTPIDEN** on leaving RME system reset and keeps this sampled signal stable while the system is not in RME system reset. The sampled signal is distributed to all components which consume **RTPIDEN**.

A component which consumes **RTPIDEN** might choose to only sample **RTPIDEN** when leaving reset. However, this is not mandatory.

RLPIDEN effects are not permitted to change after RMSD firmware is loaded.

To ensure an entire system observes the same value of **RLPIDEN** at all times, Arm recommends that the system samples **RLPIDEN** on leaving RME system reset and keeps this sampled signal stable while the system is not in RME system reset. However, an exception to this is MSD firmware explicitly permitting **RLPIDEN** to change. If MSD firmware permits **RLPIDEN** to change, a new **RLPIDEN** value is sampled under control of MSD firmware. The sampled signal is distributed to all components which consume **RLPIDEN**.

A component that consumes **RLPIDEN** might choose to only sample **RLPIDEN** when leaving reset. This is not mandatory. Such a component will be unable to observe changes in **RLPIDEN** after the component has left reset.

A system must ensure that the sequencing of resets is appropriate to ensure that the sampled **RTPIDEN** and **RLPIDEN** are further sampled by the components appropriately.

C5.4.1.1 LEGACY_TZ_EN

A component might operate in a system that can be configured to operate with or without RME. Arm recommends the component has an input signal, **LEGACY_TZ_EN**, when a component needs to be aware of the Root or Realm debug authentication status. **LEGACY_TZ_EN** defines whether the component is operating with RME enabled or disabled.

When **LEGACY_TZ_EN** is 1, **RTPIDEN** and **RLPIDEN** are ignored and the component behaves as if Root and Realm debug are disabled.

Note

A component is permitted to indicate support for RME when **LEGACY_TZ_EN** is 1. However, the component behaves as if RME is disabled, and Root debug and Realm debug are disabled.

LEGACY_TZ_EN is not permitted to change value after RME system reset has been deasserted. Arm recommends that either:

- **LEGACY_TZ_EN** is only sampled as the component leaves reset.

- The system samples **LEGACY_TZ_EN** only as it leaves RME system reset, and distributes this sampled signal to all components.

Note

In a system where a sampled **LEGACY_TZ_EN** is distributed around the system, system designers should take care with components which only sample **LEGACY_TZ_EN** as the component leaves reset. The system design should ensure that all components use an up to date and stable indication of **LEGACY_TZ_EN**.

Note

LEGACY_TZ_EN is not part of the authentication interface.

C5.5 User mode debugging

Individual components can offer greater control over the permitted level of debugging. For example, some processors implementing Arm Security Extensions can grant permission to debug-specific Secure processes by permitting debugging of Secure User mode without permitting debugging of Secure privileged modes. This level of control is extended to the ETM. For more information, see the *Arm® Embedded Trace Macrocell Architecture Specification*.

Figure C5-1 shows how the signals of the CoreSight authentication interface interact with the two registers that are controlled by the Secure *Operating System* (OS), **SUIDEN** and **SUNIDEN**:

- If **DBGEN** is asserted, **NIDEN** is ignored and assumed asserted.
- If **SPIDEN** is asserted, **SPNIDEN** is ignored and assumed asserted.
- In all other cases, the permissions that are represented by all the boxes bounding each level of debug functionality must be granted before that level of debug functionality is enabled.

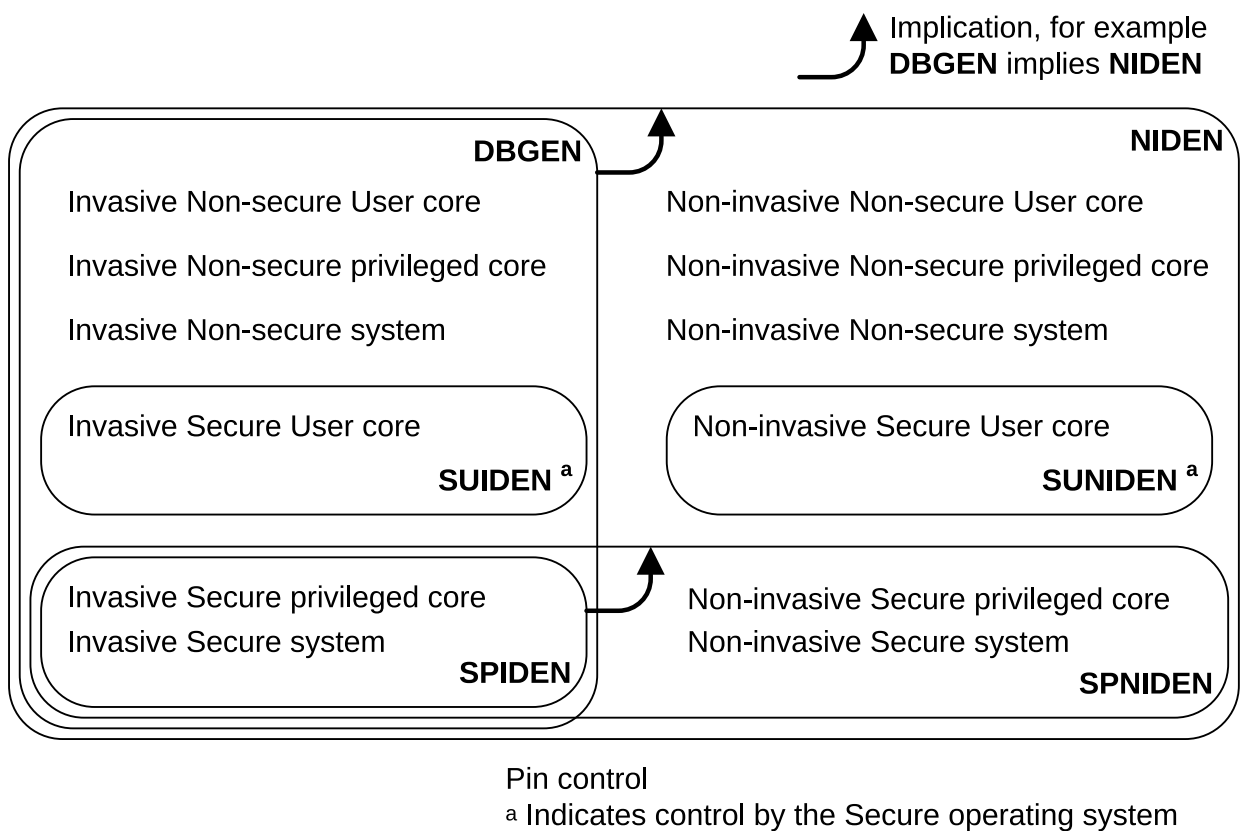


Figure C5-1 Interaction between CoreSight and Arm Security Extensions

C5.6 Control of the authentication interface

The authentication interface is controlled at the system level. For more information, see [Control of authentication interfaces](#).

C5.7 Exemptions from implementing the authentication interface

It is not necessary to implement the authentication interface to control debug functions that are only software-accessible. For these functions, it is sufficient to use standard mechanisms to control software access to privileged resources or different Security states.

Chapter C6

Timestamp Interface

A CoreSight system uses the wide timestamp interface to distribute a time value to debug components. Typically this time value is included in a trace stream to permit correlation of events in multiple trace streams.

[Table C6-1](#) shows the signals that are defined for the timestamp interface.

Table C6-1 Timestamp interface signals

Name	Description
TSCLK	Interface clock
TSRESETn	Interface reset
TSVALUEB[63:0]	Timestamp value, encoded as a natural binary number. A value of 0 indicates that the timestamp is UNKNOWN, which occurs when the timestamp value source is disabled or when the timestamp value is reset.

If a system with multiple components implements the timestamp interface, the same timestamp is used by all components. If systems use different clocks or different timestamp distribution mechanisms, there might be skew between the timestamp values that are observed by the components.

Some components might not implement a full 64-bit timestamp. These components use an IMPLEMENTATION DEFINED subset of the 64-bit timestamp value. Arm recommends using the subset that provides the largest set of unique observable timestamp values. This subset might depend on the clock speed of the component.

Arm recommends that new designs share the same source of time for both PEs and CoreSight components.

———— **Note** ————
Previous versions of this specification recommended that the timestamp resolution is at least 10% of the fastest processor in the system. This recommendation has been removed. However, Arm recommends that the timestamp resolution is reasonably high to allow for fine-grained correlation of traces.

The source of a timestamp is known as a timestamp Transmitter. The destination of a timestamp is known as a timestamp Receiver.

Chapter C7

Topology Detection at the Component Level

This chapter describes how to detect components that are connected to the AMBA ATB interface and where they are logically located in any corresponding hierarchical connection. It contains the following sections:

- [About topology detection at the component level.](#)
- [Interface types for topology detection.](#)
- [Interface requirements for topology detection.](#)
- [Signals for topology detection.](#)

C7.1 About topology detection at the component level

This chapter describes how to perform topology detection on each interface type. [Topology Detection](#) describes the topology detection requirements of CoreSight components. [Topology Detection at the System Level](#) describes how debuggers can use this information to detect the topology of a target system.

C7.2 Interface types for topology detection

A component has several interfaces that contain one or more signals. Each interface is defined in terms of the following parameters:

- A name, for example channel interface.
- A direction:
 - Transmitter, always connected to one or more Receivers of the same type.
 - Receiver, always connected to one or more Transmitters of the same type.
 - Bidirectional, always connected to one or more identical bidirectional interfaces.
 - Probe, read-only interface to trace the activity of a bus without affecting the behavior of that bus.

The following conditions apply to the interfaces used for topology detection:

- The list of interfaces must be defined for each block for the purposes of topology detection.
- Not all signals that are used in the implementation must be exposed in an interface, provided the signals that are not exposed are irrelevant for topology detection.
- Signals that are exposed in the interface must be strictly defined.

C7.2.1 Interfaces on standard components

Table C7-1 shows the interfaces present on common CoreSight components. For specific interface details, see the appropriate Technical Reference Manual.

Table C7-1 Interfaces on some example components

Programmable component	Interfaces
CoreSight ETM ^a CoreSight PTM ^b	• AMBA ATB interface, Transmitter. • One or more events, Transmitter. EXTOUT[n-1:0]. • One or more events, Receiver. EXTIN[n-1:0]. • Event, Transmitter. TRIGOUT. • Variant: CoreETM, Receiver. The number of core interfaces can be read from the programmers' model of the ETM or PTM. The state of DBGACK can be driven directly in all Arm cores, including those cores that are not CoreSight compliant.
CoreSight ETB	• AMBA ATB interface, Receiver: • Event, Transmitter. ACQCOMP. • Event, Transmitter. FULL. • Event, Receiver. TRIGIN. • Event, Receiver. FLUSHIN.
TPIU	• AMBA ATB interface, Receiver: • Event, Receiver. TRIGIN. • Event, Receiver. FLUSHIN.
Debug Ports and Access Ports	No topology detection interfaces.

Programmable component	Interfaces
HTM	• AMBA ATB interface, Transmitter: • 2x event, Transmitter. HTMEXTOUT[1:0]. • 2x event, Transmitter. HTMEXTIN[1:0]. • Event, Transmitter. HTMTRIGGER. • Variant: AHB, probe. The number of AHB interfaces can be read from the programmers' model of the HTM. The method to perform topology detection of this interface is not defined.
CoreSight Funnel	• One or more AMBA ATB interfaces, slave. • AMBA ATB interface, Transmitter.
CTI	• One or more events, Receiver. TRIGIN[7:0]. • One or more events, Transmitter. TRIGOUT[7:0]. • Channel, bidirectional.
VIC (PL190/192)	32x event, Transmitter: VICINTSOURCE[n] .

- An ETM that implements the ETMv3 or ETMv4 architecture.
- A PTM that implements the PFTv1 architecture.

C7.3 Interface requirements for topology detection

For all controllable signals, each interface type specifies:

- The signals on the Transmitter interface that must be controllable or observable.
- The signals on the Receiver interface that must be controllable or observable.
- The transitions on the interface that must be performed to trigger the following actions:
 - To initialize topology detection.
 - To assert the Transmitter interface.
 - To check whether the Receiver interface is asserted.
 - To deassert the Transmitter interface.
 - To check whether the Receiver interface is deasserted.

If the interface is bidirectional, each interface to be tested must in turn be treated as a Transmitter while the other interfaces of that type are treated as Receiver. See [Topology Detection at the System Level](#).

For signals that must be controllable, it must be possible to independently control the value of outputs, and read the value of inputs. See [Topology Detection](#).

Usually each Transmitter interface specifies one output, and the Receiver interface specifies the corresponding input. When choosing a signal, observe the conditions that are described in the following section:

- [Intermediate non-programmable components](#).
- [Multi-way connections](#).

C7.3.1 Intermediate non-programmable components

Sufficient control signals must be available to enable the interface to be driven to an active state so that it passes through any intermediate non-programmable components. For example, in AMBA ATB interfaces, **ATVALID** must be controllable, because if it is LOW, an intermediate bridge does not pass any control signals through it.

C7.3.2 Multi-way connections

In a multi-way connection:

- Asserting and deasserting a Transmitter signal might cause an effect to be seen on multiple Receivers.
- Asserting and deasserting a Receiver signal might cause an effect to be seen on multiple Transmitters.

Sufficient signals must be controllable to cause the arbitration logic to route between the Transmitter and Receiver.

C7.4 Signals for topology detection

Table C7-2 shows the controllable signals for each interface type that is listed in Table C7-1.

Table C7-2 Controllable signals for each interface type

Interface	Transmitter wires	Receiver wires
AMBA ATB interface	ATVALID	ATVALID , ATREADY
CoreETM	DBGACK ^a	DBGACK ^a
Event ^b	EVENT	EVENT , EVENTACK , if present
Channel, bidirectional	CHOUT[0]	CHIN[0] , CHINACK[0] , if asynchronous

a. Using **DBGACK** for topology detection is restricted by the fact that in some Arm processors it cannot be controlled, or it can only be controlled from a JTAG debugger. To perform topology detection on a processor that has this restriction, use a different IMPLEMENTATION DEFINED controllable signal, or the Device Affinity registers, **DEVAFF0-DEVAFF1**, which indicate the association of a processor with the ETM.

b. The event interface is defined for miscellaneous point-to-point connections that carry a one-bit signal. An event interface might implement an acknowledge signal, that, if implemented, must be controllable. To implement the event interface, substitute **EVENT** and **EVENTACK** for the equivalent signals in the appropriate interface.

Table C7-3 lists the signals that an interface must implement to support topology detection between Transmitters and Receivers of key interface types. Use the table with the algorithm given in [Detection algorithm](#). For the full specification of a signal, see the relevant interface specification.

Table C7-3 Topology detection sequences

Signal	AMBA ATB	Core ETM	Event interface	Channel interface
Transmitter preamble	ATVALID ← 0	DBGACK ← 0	EVENT ← 0	CHOUT[0] ← 0
Receiver preamble	ATREADY ← 0	None	EVENTACK ← 0, if present	CHINACK[0] ← 0, if present
Transmitter assert	ATVALID ← 1	DBGACK ← 1	EVENT ← 1	CHOUT[0] ← 1
Receiver check asserted	ATVALID == 1	DBGACK == 1	EVENT == 1	CHIN[0] == 1
Receiver post-assert	ATREADY ← 1	None	EVENTACK ← 1, if present	CHINACK[0] ← 1, if present
Transmitter deassert	ATVALID ← 0	DBGACK ← 0	EVENT ← 0	CHOUT[0] ← 0
Receiver check deasserted	ATVALID == 0	DBGACK == 0	EVENT == 0	CHIN[0] == 0
Receiver post-deassert	ATREADY ← 0	None	EVENTACK ← 0, if present	CHINACK[0] ← 0, if present

Part D **CoreSight System Architecture**

Chapter D1

About the System Architecture

The system architecture specifies:

- Rules that must be followed by all systems that implement CoreSight components.
- Additional information that is required by debuggers to use a CoreSight system.

The system architecture is described in the following chapters:

- [System Considerations](#).
- [Physical Interface](#).
- [Trace Formatter](#).
- [About ROM Tables](#).
- [Topology Detection at the System Level](#).
- [Compliance Requirements](#).

Chapter D2

System Considerations

This chapter describes system aspects that must be considered when integrating CoreSight components into a system. It has the following sections:

- [Clock and power domains](#) describes the requirements for the clock and power domain structure that is exposed to debuggers.
- [Control of authentication interfaces](#) describes the requirements for the signals in the authentication interface.
- [Memory system design](#) describes how to expose CoreSight registers to system software.

D2.1 Clock and power domains

CoreSight can be used in systems with many clock and power domains. CoreSight systems themselves, however, always define the following clock and power domains:

System domain This domain comprises most non-debug functionality. The clock frequencies in this domain can be asynchronous to the other domains and can vary over time in response to varying performance requirements. The clocks can be stopped, and the power can be removed, leading to the loss of all state information.

Debug domain This domain comprises most debug functionality. When debug functionality is not required, the power can be removed or the clocks can be stopped to reduce power consumption.

Always-on domain

This domain comprises the power controller and the interface to the debugger. The power is never removed, even when the device is dormant, which enables the debugger to connect to the device even when it is powered down.

When deviating from the default clock and power domains, observe the following rules:

- When implementing extra clock and power domains by subdividing one of the default clock and power domains, make sure that the clock and power domains respond appropriately to requests made using the debug interface. For example, an implementation can have two system clock domains, as long as both domains are permanently accessible whenever a System Power Up request is made.
- When implementing fewer clock and power domains by combining two or more of the default clock and power domains, make sure that all requests made using the debug interface are operational. For example, when combining the system and debug power domains, the combined domain must always be powered up whenever a System Power Up request or a Debug Power Up request is made.

The debugger can use the debug interface to make the following requests to the system:

- Power up everything in the system domain. When this request is made, all logic in the system domain must be kept permanently powered up, and be continuously accessible to the debugger.
- Power up everything in the debug domain. When this request is made, all logic in the debug domain must be kept permanently powered up, and be continuously accessible to the debugger.
- Reset everything in the debug domain. When this request is made, all logic in the debug domain must be reset to its initial state.

The debugger interface is managed by an implementation of the ADI. For more information, see the appropriate CoreSight Technical Reference Manual.

D2.2 Control of authentication interfaces

A CoreSight system prevents unauthorized debugging by disabling debug functionality, rather than by preventing access to the debug registers. This mechanism is controlled by the authentication interface. For more information about the authentication interface, see [Authentication Interface](#).

Each signal can be driven in one of the following ways:

- Tied LOW. This method is most appropriate for production systems where the specified debug functionality is not required, and prevents in-the-field debugging. There is usually an alternative development chip with the same functionality enabled.
- Tied HIGH. This method is most appropriate for prototype or development systems where authentication is not required.
- Connected to a fuse that is blown in production parts to disable debug functionality, which prevents in-the-field debugging.
- Driven by a custom authentication module, that unlocks debug functionality after a successful authentication sequence. This method provides the most flexibility. In systems where high security is required, Arm recommends using a challenge-response mechanism that is based on an on-chip random number generator or a hardware key unique to that device.

When debugging of a particular Security state is enabled, this can mean that debug functionality is visible to the outside world, and sometimes to software running in a less privileged Security state.

Arm recommends that devices support a lifecycle model that avoids disclosing higher privileged information to lower privileged observers in production devices. This can mean any of the following:

- Separate development devices are produced which can have debug of higher Security states enabled, and production devices might have no debug enabled, or only debug of Non-secure state.
- Development devices can have debugging of Secure state, Root state, or Realm state enabled only by authorized developers. All such data in these higher privileged Security states must be replaced by test data suitable for development purposes, where losses are minimal if the test data is disclosed. Production devices never have debugging enabled for these higher privileged Security states. These devices can be loaded with the real secure data.

D2.3 Memory system design

This section describes how to expose CoreSight registers to system software.

D2.3.1 Debug APB interface memory map

Components and the interconnect are not required to differentiate between external and internal accesses unless one of the following applies:

- The component implements the Software lock mechanism that is described in the programmers' model, see [LSR and LAR, Software Lock Status Register and Software Lock Access Register](#).

Note

From v3.0 onwards, implementation of the Software lock is deprecated. If a component implements the Software lock, but is accessed using an interconnect that does not support indicating the difference between external and internal accesses, Arm strongly recommends that the component is configured as follows:

- LSR is RAZ to indicate that the Software lock mechanism is not implemented. See also [LSR and LAR, Software Lock Status Register and Software Lock Access Register](#).
 - PADDRDBG[31] is tied to HIGH to force all accesses to be interpreted as external accesses. See also [AMBA APB interface signals](#).
-

- The component implements an OS lock mechanism, for example an ETM.

For the limited set of components that must differentiate between external and internal accesses, Arm recommends that two views of the component are provided in the memory system, one for internal accesses, and one for external accesses:

- The two views can be located anywhere in the Debug APB interface address space.
- Arm strongly recommends that a ROM Table provides a pointer only to the external view.
- Arm recommends that one of the address bits is used to differentiate the views.

An example of this configuration is shown in [Figure D2-1](#), where a 4KB component of a PE at address 0xB0000000 uses two adjacent views at addresses 0x00002000 and 0x00003000 in the APB memory map. Accesses to addresses between 0x00002000 and 0x00002FFF, for which address bit[12] has a value of 0b0, are external accesses, and accesses to addresses between 0x00003000 and 0x00003FFF, for which address bit[12] has a value of 0b1, are internal accesses.

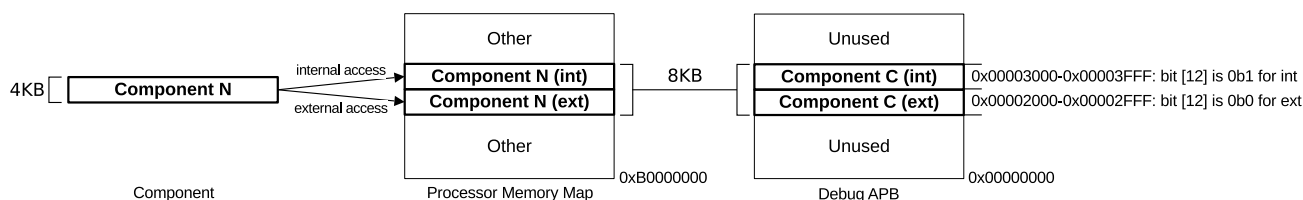


Figure D2-1 Example of an AMBA APB interface memory map for a component with external and internal views

Note

From v3.0 onwards, the use of PADDRDBG[31] to split the memory map into two 2GB segments to indicate the difference between external and internal accesses is deprecated. An older implementation having its external and internal views 2GB apart can be regarded as a special case of a version 3.0 implementation: one that uses address bit[31] to differentiate between external and internal accesses.

If a component requires more than one view, and those views are not related to the external and internal views described in this section, the arrangement of those views in the memory system is IMPLEMENTATION DEFINED.

Examples of implementations of a system with three components are shown in [Figure D2-2](#) and [Figure D2-3](#):

- In the example that is shown in [Figure D2-2](#), one of the components, Component C, requires differentiation between external and internal accesses. The component provides two adjacent views allowing external accesses at addresses for which bit[12] equals 0b0, and internal accesses at addresses for which bit[12] equals 0b1.
- In the example that is shown in [Figure D2-3](#), all three components require differentiation between external and internal accesses. As in the example that is shown in [Figure D2-2](#), address bit[12] is used to differentiate between external and internal accesses, leading to a configuration where the external views are interleaved with the internal views.

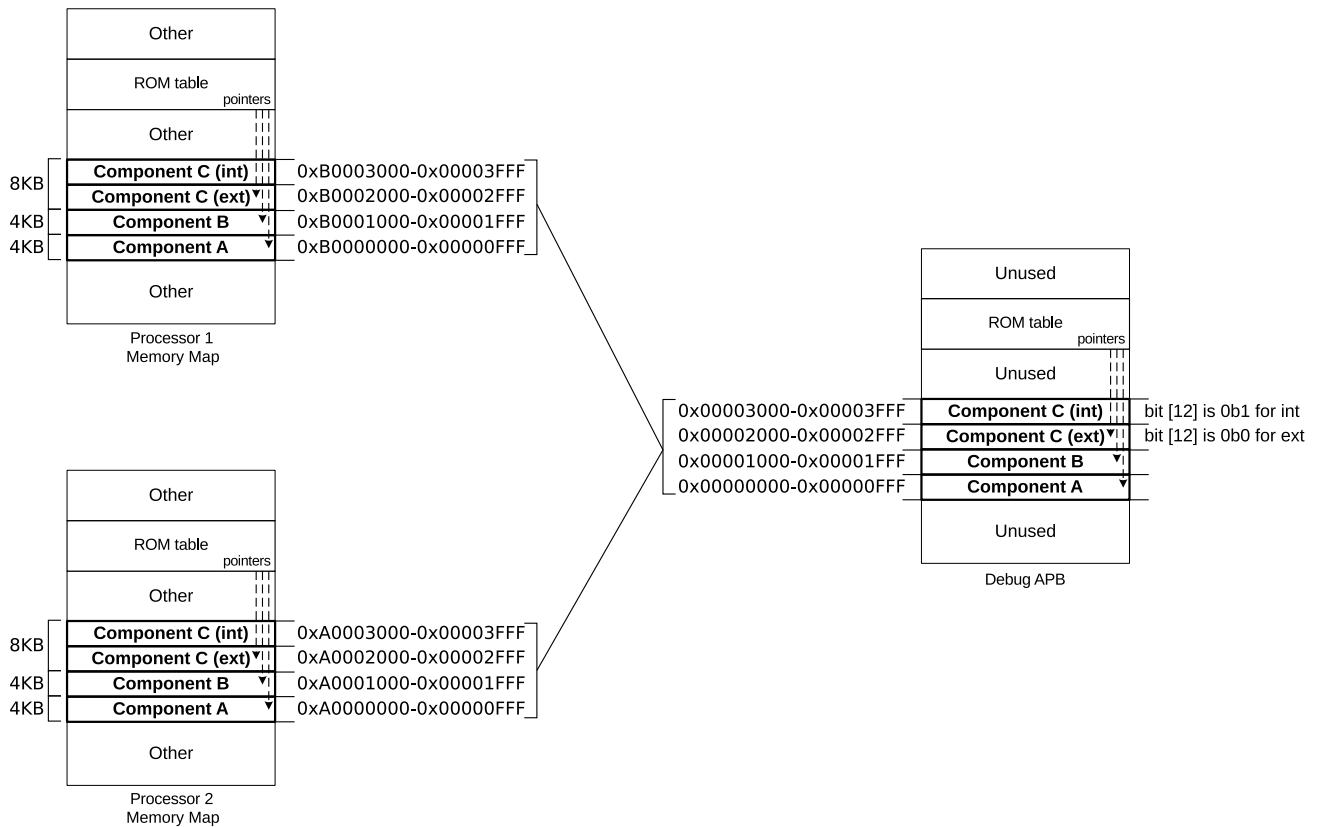


Figure D2-2 Example that includes one component with external and internal views

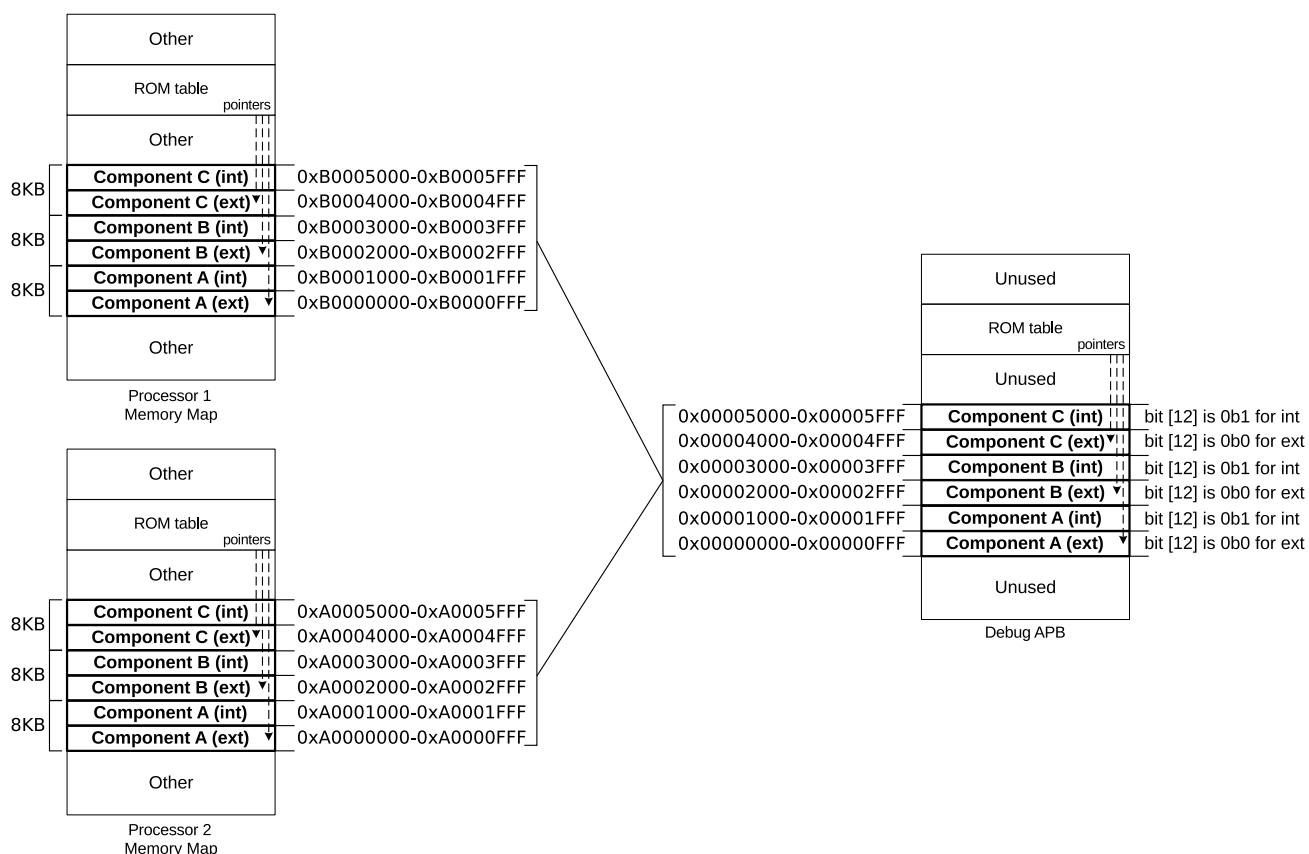


Figure D2-3 Example that includes three components with external and internal views

D2.3.2 Access to the Debug APB interface

When designing a CoreSight system, ensure that the registers of CoreSight components are visible to privileged software.

Note

Do not prevent Non-secure software from accessing the registers of CoreSight components, even if those components can debug secure software. Doing so seriously restricts debugging of Non-secure software.

For system bus transaction Requesters that support privileged and unprivileged modes, Arm recommends the following:

- If the Requester does not have an MMU or MPU, the system prevents access to the CoreSight components by unprivileged software.
- If the Requester does have an MMU or MPU, the MMU or MPU is used to prevent access to the CoreSight components by unprivileged software.
- CoreSight components are marked as Device memory.

Chapter D3

Physical Interface

This chapter describes the external pin interface, timing, and connector type that is required for the trace port on a target system. It contains the following sections:

- [Arm JTAG 20.](#)
- [CoreSight 10 and CoreSight 20 connectors.](#)
- [Arm MICTOR.](#)
- [Target Connector Signal details.](#)

D3.1 Arm JTAG 20

The Arm JTAG 20 connector is a 20-way 2.54mm pitch connector. It supports the following interfaces:

- JTAG interface, which is based on IEEE 1149.1-1990 and includes the Arm **RTCK** signal.
- *Serial wire debug* (SWD) interface.
- *Serial Wire Output* (SWO) interface.

Figure D3-1 shows the Arm JTAG 20 connector pinout.

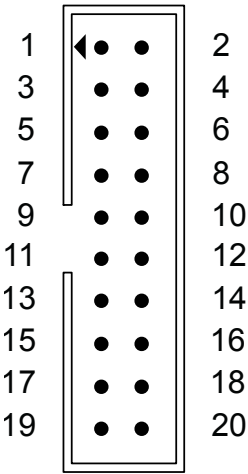


Figure D3-1 Arm JTAG 20 connector pinout

Table D3-1 shows the Arm JTAG 20 pinout as used on the target board.

Table D3-1 Arm JTAG 20 interface pinout table

Pin	Signal name
1	VTREF
2	NC
3	nTRST
4	GND
5	TDI
6	GND
7	TMS/SWDIO
8	GND
9	TCK/SWCLK
10	GND
11	RTCK
12	GND
13	TDO/SWO
14	GND
15	nSRST

Pin	Signal name
16	GND
17	DBGRRQ / TRIGIN
18	GND
19	DBGACK / TRIGOUT
20	GND

See [Target Connector Signal details](#) for a description of the signals in [Table D3-1](#).

D3.2 CoreSight 10 and CoreSight 20 connectors

The CoreSight 10 and CoreSight 20 connectors are used for debug targets requiring JTAG, SWD, SWO, and low-bandwidth trace connectivity.

This section describes 10-way and 20-way connectors that are mounted on debug target boards. These connectors are specified as 0.050 inch pitch two-row pin headers, Samtec FTSH or equivalent. For more information, see the Samtec website, www.samtec.com.

The connectors are grouped into compatible sets according to the functions they support. Some targets support communication by both SWD and JTAG using the SWJ-DP block to switch between protocols.

Note

Some of the tables in this section have a column that is named MICTOR, which lists the equivalent pin numbers on a CoreSight-compatible *Matched Impedance Connector* (MICTOR) connector for the named CoreSight pin. A target can feature both CoreSight and MICTOR connectors, where the signals are connected in parallel as suggested by the pinout tables. This configuration enables the target to be debugged using either physical connector.

The connector pin layouts are described in:

- [Combined CoreSight 10 and CoreSight 20 pin names.](#)
- [CoreSight 10 pinouts.](#)
- [CoreSight 20 pinouts including trace.](#)

D3.2.1 Combined CoreSight 10 and CoreSight 20 pin names

[Table D3-2](#) summarizes the combined pin names for the CoreSight 10 and CoreSight 20 connectors.

Table D3-2 Summary of pin names

Pin	Combined pin name	Pin	Combined pin name
1	VTref	11	Gnd/TgtPwr+Cap
2	TMS/SWDIO	12	TraceClk/RTCK
3	GND	13	Gnd/TgtPwr+Cap
4	TCK/SWCLK	14	TraceD0/SWO
5	GND	15	GND
6	TDO/SWO/EXTa/TraceCtl	16	TraceD1/nTRST
7	Key	17	GND
8	TDI/EXTb	18	TraceD2/TrigIn
9	GNDDetect	19	GND
10	nSRST	20	TraceD3/TrigOut

See [Target Connector Signal details](#) for a description of the signals in [Table D3-2](#).

The following sections describe the use of pins 6, 8, 11, and 13:

- [EXTa and EXTb pins.](#)
- [GND/TgtPwr+Cap pins.](#)

D3.2.1.1 EXTa and EXTb pins

Some pins on the connector have functions that are only used for certain connection layouts. Where a pin is not required for a particular debug communication protocol, it can be reused for a user-defined target function. These pins are labeled **EXTa** and **EXTb** in the tables in this chapter. Select any alternate functions carefully, and consider the effects of connecting debug equipment that is capable of communicating using multiple protocols.

D3.2.1.2 GND/TgtPwr+Cap pins

There are two usage options for these pins:

Target boards

Standard target boards can connect these two pins directly to signal ground, GND.

Some special target boards, for example, boards that are used for evaluation or demonstration purposes, can use these pins to supply power to the target board. In this case, the target board must include capacitors between each of the pins and signal ground. The capacitors must be situated extremely close to the connector so that they maintain an effective AC ground, that is, high frequency signal return path. Typical values for the capacitors are 10nF.

Debug equipment

Debug communication equipment that is designed to work with special valuation or demonstration target boards provides an operating current, typically up to 100mA, at a target-specific supply voltage, for example, 3.3V, 5V, or 9V. Arm recommends that the debug equipment includes some protection when it is connected to standard target boards that have connected these pins directly to GND, for example, a current limiting circuit. This debug equipment must include capacitors between each of these power pins and signal ground. The capacitors must be situated extremely close to the connector so that they maintain an effective AC ground, ensuring a high frequency signal return path. Typical values for the capacitors are 10nF.

Standard debug communication equipment can connect these pins directly to GND. It is also possible for these pins to have only a high frequency signal return path to ground, using 10nF capacitors. This option is also compatible in the unlikely case where a target board has a connection between the debug connector **TgtPwr** pins, but is powered from another source.

D3.2.2 CoreSight 10 pinouts

There are two pinouts for a 10-pin connector:

- Pinout that supports communication using SWD.
- Pinout for JTAG.

The pinouts are arranged to facilitate dynamic switching between the protocols.

Note

SWD is the preferred protocol for debugging because it provides more data bandwidth over fewer pins, which increases the bandwidth that is available to application functions. JTAG can be used where the target is communicating with a tool chain that does not support SWD, or with test tools performing board-level boundary scan testing, where it might be acceptable to sacrifice the functional pins multiplexed with JTAG.

[Table D3-3](#) shows the CoreSight 10 for targets using SWD or JTAG for debug communication, and includes an optional SWO signal for conveying application and instrumentation trace.

Table D3-3 CoreSight 10 for SWD or JTAG systems

Pin name for SWD	Pin number		Pin name for JTAG	Pin number	
	10-way	MICTOR		10-way	MICTOR
VTref	1	12	VTref	1	12
SWDIO	2	17	TMS	2	17
GND	3	-	GND	3	-
SWCLK	4	15	TCK	4	15
GND	5	-	GND	5	-
SWO	6	11	TDO	6	11
Key	7	-	Key	7	-
NC/EXTb	8	19	TDI	8	19
GNDDetect	9	-	GNDDetect	9	-
nSRST	10	9	nSRST	10	9

The SWD layout is typically used in a CoreSight system that uses an SWJ-DP operating in SWD mode.

The JTAG layout is typically used in a CoreSight system that includes a JTAG-DP, or a system with an SWJ-DP operating in JTAG mode, possibly because it is cascaded with other JTAG TAPs.

Note

A target board can use this connector for performing board-level boundary scans but then switch its SWJ-DP into SWD mode for debugging according to the layout shown in [Table D3-3](#). This method frees up pins 6 and 8 for either application functions or SWO.

It is not necessary to choose the switching mode at the time of chip or board development. The connector can be switched and the target board can be operated in either SWD or JTAG mode.

D3.2.3 CoreSight 20 pinouts including trace

20-way connectors include support for a narrow trace port of up to four data bits, operating at moderate speeds of up to 100 MSamples/sec.

[Table D3-4](#) shows the CoreSight 20 for targets using SWD or JTAG for debug communication, and includes an optional **SWO** signal for conveying application or instrumentation trace. Alternatively, a target trace port operating in CoreSight normal or bypass modes might convey the **TraceCtl** signal on pin 6.

Both pin 6 and pin 8 in the SWD layout are shown with alternative extra signals, **EXTa** and **EXTb**, which provides flexibility to communicate other signals on these pins. For example, future target systems and trace equipment might convey two more trace data signals on these pins.

Table D3-4 CoreSight 20 for future SWD or JTAG systems

Pin name for SWD	Pin number		Pin name for JTAG	Pin number	
	20-way	MICTOR		20-way	MICTOR
VTref	1	12	VTref	1	12
SWDIO	2	17	TMS	2	17
GND	3	-	GND	3	-
SWCLK	4	15	TCK	4	15

Pin name for SWD	Pin number		Pin name for JTAG	Pin number	
	20-way	MICTOR		20-way	MICTOR
GND	5	-	GND	5	-
SWO/EXTa/TraceCtl	6	11	TDO	6	11
Key	7	-	Key	7	-
NC/EXTb	8	(19)	TDI	8	19
GNDDetect	9	-	GNDDetect	9	-
nSRST	10	9	nSRST	10	9
Gnd/TgtPwr+Cap	11	-	Gnd/TgtPwr+Cap	11	-
TraceClk	12	6	TraceClk	12	6
Gnd/TgtPwr+Cap	13	-	Gnd/TgtPwr+Cap	13	-
TraceD0	14	38	TraceD0	14	38
GND	15	-	GND	15	-
TraceD1	16	28	TraceD1	16	28
GND	17	-	GND	17	-
TraceD2	18	26	TraceD2	18	26
GND	19	-	GND	19	-
TraceD3	20	24	TraceD3	20	24

The SWD layout is typically used in a CoreSight system that uses an SWJ-DP operating in SWD mode.

The JTAG layout is typically used in a CoreSight system that includes a JTAG-DP, or a system with an SWJ-DP operating JTAG mode, possibly because it is cascaded with other JTAG TAPs. This layout is the recommended debug connection for a processor that is built with support for instruction trace, for example, a processor that includes an ETM.

————— **Note** —————

A target board can use this layout for performing board-level boundary scans but then switch its SWJ-DP into SWD mode for debugging according to the layout shown in [Table D3-4](#). This method frees up pins 6 and 8 for either application functions or SWO.

It is not necessary to choose the switching mode at the time of chip or board development. The connector can be switched and the target board can be operated in either SWD or JTAG mode.

D3.3 Arm MICTOR

The following sections describe:

- [Target system connector.](#)
- [Target connector description.](#)
- [Decoding requirements for Trace Capture Devices.](#)
- [Electrical characteristics.](#)

D3.3.1 Target system connector

The specified target system connector is the AMP MICTOR. This connector supports:

- JTAG interface, which is based on IEEE 1149.1-1990 and includes the Arm **RTCK** signal.
- Trace port interface, with up to 16 data pins.
- *Serial Wire Debug* (SWD) interface.
- *Serial Wire Output* (SWO) interface.
- Optional power supply pin.
- Reference voltage pin to enable support of a range of target voltages.
- Optional system reset request pin.
- Optional extra trigger pins for communicating with the target.

For tracing with large port widths that have more than 16 data pins, two connectors are required. See [Single target connector pinout](#) and [Dual target connector pinout](#).

The AMP MICTOR connector is a high-density matched-impedance connector. This connector has several important attributes:

- Direct connection to a logic analyzer probe using a high-density adapter cable with termination, for example HPE5346A from Agilent.
- Matching impedance characteristics, enabling the connector to be used at high speeds.
- Many ground fingers to ensure good signal integrity.
- Inclusion of one or both of the JTAG and SWD runtime control signals, enabling a single debug connection to the target.

[Table D3-5](#) lists the AMP part numbers for the four possible connectors.

Table D3-5 Connector part numbers

AMP part number	Description
2-767004-2	Vertical, surface mount, board to board or cable connectors
767054-1	
767061-1	
767044-1	Right angle, straddle mount, board to board or cable connector

D3.3.2 Target connector description

This section contains details of the physical layout of the connector and recommended board orientation as follows:

- [Single target connector pinout.](#)
- [Dual target connector pinout.](#)

D3.3.2.1 Single target connector pinout

Figure D3-2 shows how the connector and PCB can be oriented on the target system, as seen from above the PCB. The trace connector is mounted near the edge of the board to minimize the intrusiveness of the TPA when the interconnect is a direct PCB to PCB link.

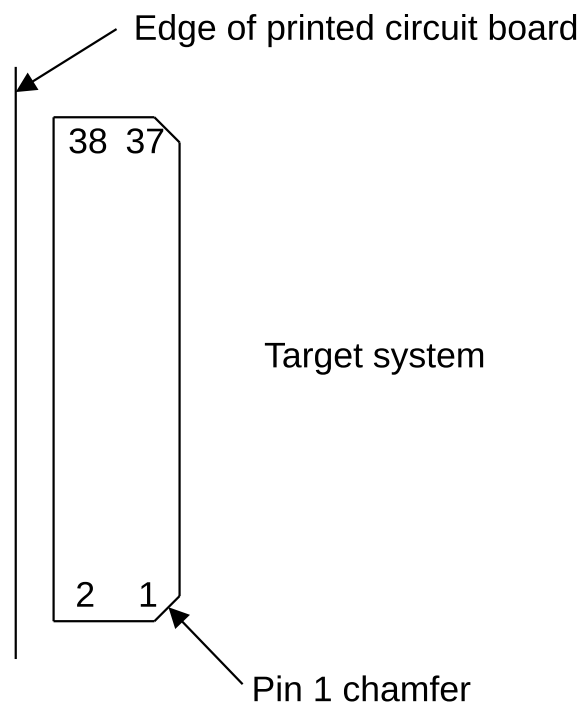


Figure D3-2 Recommended orientation for a single connector

Table D3-6 shows the connections on a single MICTOR connector, and indicates the differences with the connections specified in the *Arm® Embedded Trace Macrocell Architecture Specification*. If a signal is not implemented on the target system, it must be replaced with a logic 0 connection.

Table D3-6 Single target connector pinout

Pin	Signal	Pin	Signal
38	TRACEDATA[0]	37	TRACEDATA[8]
36	TRACECTL	35	TRACEDATA[9]
34	Logic 1	33	TRACEDATA[10]
32	Logic 0	31	TRACEDATA[11]
30	Logic 0	29	TRACEDATA[12]
28	TRACEDATA[1]	27	TRACEDATA[13]
26	TRACEDATA[2]	25	TRACEDATA[14]
24	TRACEDATA[3]	23	TRACEDATA[15]
22	TRACEDATA[4]	21	nTRST
20	TRACEDATA[5]	19	TDI
18	TRACEDATA[6]	17	TMS / SWDIO
16	TRACEDATA[7]	15	TCK / SWCLK

Pin	Signal	Pin	Signal
14	VSupply	13	RTCK
12	VTRef	11	TDO / SWO
10	No connection, was EXTTRIG	9	nSRST
8	TRIGOUT / DBGACK	7	TRIGIN / DBGRQ
6	TRACECLK	5	GND
4	No connection	3	No connection
2	No connection	1	No connection

D3.3.2.2 Dual target connector pinout

Figure D3-3 shows the arrangement for two connectors. Arm recommends that they are placed in line, with pins 1 separated by 1.35inches.

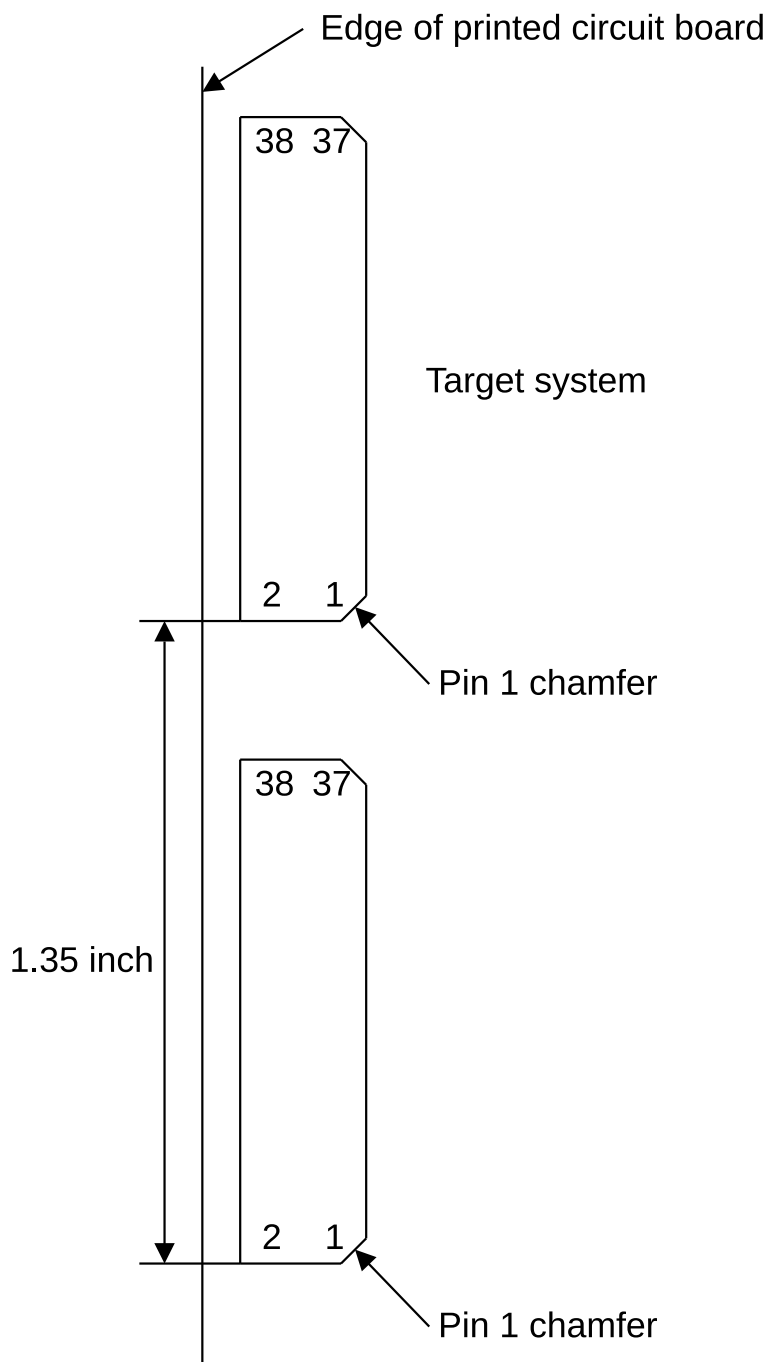


Figure D3-3 Recommended orientation for two connectors

Table D3-7 shows the connections for the secondary MICTOR connector. For the primary connector, see Table D3-6. If a signal is not implemented on the target system, it must be replaced with a logic 0 connection.

Table D3-7 Dual target connector pinout

Pin	Signal	Pin	Signal
38	TRACEDATA[16]	37	TRACEDATA[24]
36	Logic 0	35	TRACEDATA[25]

Pin	Signal	Pin	Signal
34	Logic 1	33	TRACEDATA[26]
32	Logic 0	31	TRACEDATA[27]
30	Logic 0	29	TRACEDATA[28]
28	TRACEDATA[17]	27	TRACEDATA[29]
26	TRACEDATA[18]	25	TRACEDATA[30]
24	TRACEDATA[19]	23	TRACEDATA[31]
22	TRACEDATA[20]	21	No connection
20	TRACEDATA[21]	19	No connection
18	TRACEDATA[22]	17	No connection
16	TRACEDATA[23]	15	No connection
14	No connection	13	No connection
12	VTRef	11	No connection
10	No connection	9	No connection
8	No connection	7	No connection
6	TRACECLK	5	GND
4	No connection	3	No connection
2	No connection	1	No connection

D3.3.3 Decoding requirements for Trace Capture Devices

Table D3-8 shows the conditions that must be decoded by *Trace Capture Devices* (TCDs), for example a TPA or a logic analyzer.

Table D3-8 Trace Capture Device decoding

TRACECTL	TRACEDATA[0]	TRACEDATA[1]	Trigger	Capture	Description
0	x	x	No	Yes	Normal trace data
1	0	0	Yes	Yes	Trigger packet
1	0	1	Yes	No	Trigger
1	1	x	No	No	Trace disable

D3.3.3.1 Normal trace data

When trace data is indicated, only the full field of **TRACEDATA[n:0]** has to be stored. **TRACECTL** can be discarded to permit more efficient packing of data within the TCD.

D3.3.3.2 Trigger packet

Although CoreSight does not use the encoding for a trigger packet, it must be implemented to maintain cross compatibility with ETMv3.x trace ports. **TRACEDATA** signals must be stored because there is further information that is emitted on this cycle, but the **TRACECTL** signal can be discarded. For more information, see the *Arm® Embedded Trace Macrocell Architecture Specification*.

D3.3.3.3 Trigger

A trigger is used as a marker to enable the TCD to stop capture after a predetermined number of cycles. No data is output on this cycle, and **TRACEDATA[n:0]** and **TRACECTL** must not be captured.

D3.3.3.4 Trace disable

This signal indicates that the current cycle must not be captured because it contains no useful information.

D3.3.4 Electrical characteristics

Debug equipment must be able to deal with a wide range of signal voltage levels. Typical ASIC operating voltages can range from 1V to 5V, although 1.8V to 3.3V is common.

It is important to keep the track length differences as small as possible to minimize skew between signals. Crosstalk on the trace port must be kept to a minimum because it can cause erroneous trace results. To minimize the chance of unpredictable responses, avoid stubs on the trace lines, especially at high frequencies. If stubs are necessary, make them as small as possible.

The trace port clock line, **TRACECLK** must be series-terminated as close as possible to the pins of the driving ASIC.

The maximum capacitance that is presented by the trace connector, cabling, and interfacing logic must be less than 15pF.

There are no inherent restrictions on operating frequency, other than ASIC pad technology and TPA limitations. It is mandatory, however, to observe the following guidelines for maximizing the speed at which trace capture is possible.

Figure D3-4 shows the timing for **TRACECLK**.

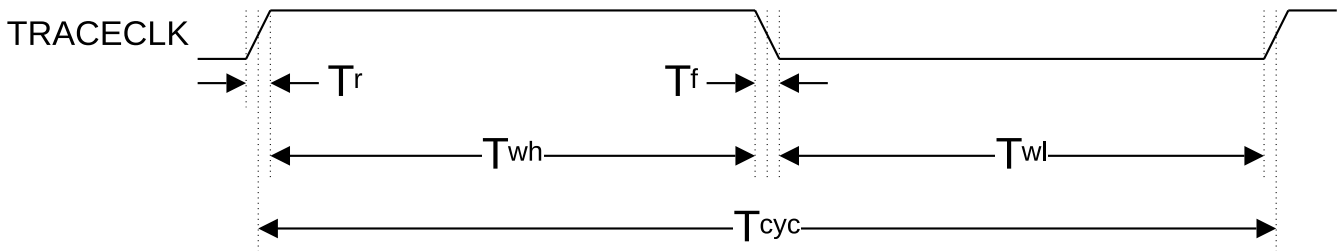


Figure D3-4 TRACECLK specification

Arm recommends that trace ports provide a **TRACECLK** that is as symmetrical as possible, because both edges are used to capture trace. Figure D3-5 shows the setup and hold requirements for the trace data pins, **TRACEDATA[n:0]** and **TRACECTL**, in relation to **TRACECLK**.

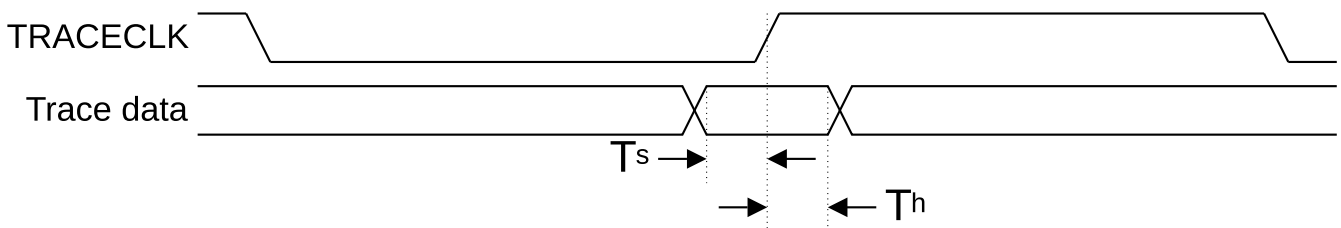


Figure D3-5 Trace data specification

Arm recommends observing the following principles:

- Make sure that the setup time T_s and the hold time T_h are as large as possible, and make sure that both are positive, which is required by some TPAs.
- Allow the TPAs to delay each trace data signal individually by up to a whole clock period, to compensate for trace ports where T_s and T_h are not balanced or vary between data signals.

D3.4 Target Connector Signal details

The signals on the target connector pins are:

- VTRef output.
- TRACECLK output.
- TRACECTL output.
- TRACEDATA[n:0] output.
- Logic one input.
- Logic 0 input.
- TRIGIN / DBGRQ input.
- TRIGOUT / DBGACK output.
- nSRST input.
- nTRST input.
- TDI input.
- TMS input.
- SWDIO input/output.
- TCK / SWCLK input.
- RTCK output.
- TDO output.
- SWO output.
- VSupply output.
- GND.
- No connection.

D3.4.1 VTRef output

The VTRef signal is intended to supply a logic-level reference voltage to enable debug equipment to adapt to the signaling levels of the target board.

Note

VTRef does not supply operating current to the debug equipment.

Target boards must supply a voltage of 1-5V $\pm 10\%$, implying a minimum of 0.9V, and a maximum of 5.5V. The DC output impedance of the target board must be low enough to ensure that the output voltage does not change by more than 1% when supplying a nominal signal current of 0.4mA. Debug equipment that connects to this signal must use it as a signal rather than a power supply pin and not load it more heavily than a signal pin. The recommended maximum source or sink current is 0.4mA.

D3.4.2 TRACECLK output

The trace port must be sampled on both edges of the TRACECLK clock signal. There is no requirement for TRACECLK to be linked to a core clock.

D3.4.3 TRACECTL output

This signal, together with **TRACEDATA[1:0]**, indicates whether trace information can be stored this cycle. It is not necessary to store **TRACECTL**.

D3.4.4 TRACEDATA[n:0] output

This signal can be any size and represents the data that is generated from the trace system. To decompress the data, an understanding of the data stream is required, because the data can be wrapped up within the Formatter protocol or consist of direct data from a single trace source. See also [Trace Formatter](#).

D3.4.5 Logic one input

This signal pin is at a voltage level that represents logic 1, typically a resistor pull-up to VTRef.

D3.4.6 Logic 0 input

This signal pin is at a voltage level that represents logic 0, typically a resistor pull-down to GND.

D3.4.7 TRIGIN / DBGRQ input

This signal is used to change the behavior of on-chip logic, for example by connecting it to a CTI. Arm recommends that this pin is pulled to a defined state, LOW, to avoid unintentional requests to any connected on-chip logic.

D3.4.8 TRIGOUT / DBGACK output

This signal can be connected to on-chip trigger generation logic such as a CTI to enable events to be propagated to external devices.

D3.4.9 nSRST input

This signal is an open-collector output from the run control unit to the target system reset, or an input to the run control unit so that a reset initiated on the target can be reported to the debugger.

On the target, pull up this pin to HIGH to avoid unintentional resets when there is no connection.

For more details on the use of **nSRST**, see the *Arm® Debug Interface Architecture Specification*.

D3.4.10 nTRST input

The **nTRST** signal is an open-collector input from the run control unit to the Reset signal on the target JTAG port. On the target, pull up this pin to HIGH to avoid unintentional resets when there is no connection.

D3.4.11 TDI input

TDI is the Test Data In signal from the run control unit to the target JTAG port. Arm recommends that this pin is set to a defined state.

D3.4.12 TMS input

TMS is the Test Mode Select signal from the run control unit to the target JTAG port. On the target, this pin must be pulled up to HIGH to ensure that when there is no connection, the effect of any spurious **TCKs** is benign. For connectors that share JTAG and SWD signals, this pin is shared with the **SWDIO** signal.

D3.4.13 SWDIO input/output

The Serial Wire Data I/O pin sends and receives serial data to and from the target during debugging. For connectors that share JTAG and SWD signals, this pin is shared with the **TMS** signal.

D3.4.14 TCK / SWCLK input

TCK / SWCLK is the Test Clock signal from the run control unit to the target JTAG or SWD port. Arm recommends that this pin is pulled to a defined state.

D3.4.15 RTCK output

RTCK is the Return Test Clock signal from the target JTAG port to the run control unit. Some targets, such as Arm7TDMI-S™, must synchronize the JTAG port to internal clocks. To facilitate meeting this requirement, a returned, and re-timed, **TCK** can be used to dynamically control the **TCK** rate.

D3.4.16 TDO output

TDO is the Test Data Out from the target JTAG port to the run control unit. This signal must be set to its inactive drive state, tristate, when the JTAG state machine is not in the Shift-IR or Shift-DR states. For connectors that share JTAG and SWD signals, this pin is shared with the **SWO** signal.

D3.4.17 SWO output

The Serial Wire Output pin can be used to provide trace data. For connectors that share JTAG and SWD signals, this pin is shared with the **TDO** signal.

D3.4.18 VSupply output

The VSupply pin enables the target board to supply operating current to debug equipment so that an external power supply is not required. It is used in one of the following ways:

- The pin is unused.
- Provides a second reference voltage for the non-trace signals. When used in this way, VTRef provides a reference voltage for the trace signals, and VSupply provides a reference voltage for the following signals:
 - **TRIGIN/DBGREQ.**
 - **TRIGOUT/DBGACK.**
 - **nSRST.**
 - **TDO/SWO.**
 - **RTCK.**
 - **TCK/SWCLK.**
 - **TMS/SWDIO.**
 - **TDI.**
 - **nTRST.**
- Enables the target board to supply operating current to debug equipment so that an external power supply is not required.
 - The VDD power rail typically drives the pin on the target board.
 - Target board documentation indicates the VSupply pin voltage and the current available. Target boards must supply a voltage that is nominally between 2V and 5V with a tolerance of $\pm 10\%$, amounting to a minimum of 1.8V and a maximum of 5.5V. A target board that drives this pin must provide a minimum supply current of 250mA, where 400mA is recommended.
 - To enable establishing the need for an external power supply to power the debug equipment, debug equipment must indicate the required supply voltage range and the current power consumption over that range.
 - Target boards might have a limited amount of current available for external debug equipment, so a backup mechanism to power the debug equipment must be provided in case VSupply is not connected or insufficient.

Arm expects that the VSupply pin is unlikely to be used to supply operating current to debug equipment.

Arm recommends that the SoC designer and the probe designer liaise on the use of separate reference voltages for trace and non-trace signals to ensure proper use of the VSupply pin.

D3.4.19 GND

This pin must be connected to 0V on the target board to provide a signal return and logic reference.

D3.4.20 No connection

No connection must be made to this pin.

Chapter D4

Trace Formatter

This chapter describes trace formatter requirements for devices that comply with the CoreSight architecture. It contains the following sections:

- [About trace formatters.](#)
- [Frame descriptions.](#)
- [Modes of operation.](#)
- [Flush of trace data at the end of operation.](#)

D4.1 About trace formatters

Formatters are methods for wrapping Trace Source IDs into the output Trace Data stream. This chapter specifies the format that is used by Trace Sinks to embed AMBA ATB interface source IDs into a single trace stream. For more information about the AMBA ATB protocol, see [AMBA ATB interface](#).

The formatter protocol has the following features:

- It permits trace from several sources to be merged into a single stream and later separated.
- It does not place any requirements or constraints on the data that is emitted from trace sources.
- It is suitable for high-speed real-time decoding.
- It can be transmitted and stored as a bitstream without the need for separate alignment information.
- It can be decoded even if the start of the trace is lost.
- It indicates the position of the trigger signal around which trace capture is centered to the TPA, eliminating the need for a separate pin.
- It indicates when the trace port is inactive to the TPA, eliminating the need for a separate flow control pin.

If the embedded trigger and flow control information is not required by the TPA, and only a single trace source is used, it is possible to disable the formatting to achieve better data throughput.

D4.2 Frame descriptions

The formatter protocol outputs data in 16-byte frames. Each frame consists of:

- 7 bytes of data.
- 8 mixed-use bytes, each of which contains:
 - One bit to indicate the use of the remaining seven bits.
 - Seven bits that can be data or a change of trace source ID.
- One byte of auxiliary bits, where each bit corresponds to one of the 8 mixed-use bytes:
 - If the corresponding byte was data, this bit indicates the remaining bit of that data.
 - If the corresponding byte was an ID change, this bit indicates when that ID change takes effect.

Figure D4-1 shows how these bytes are arranged in a formatter frame.

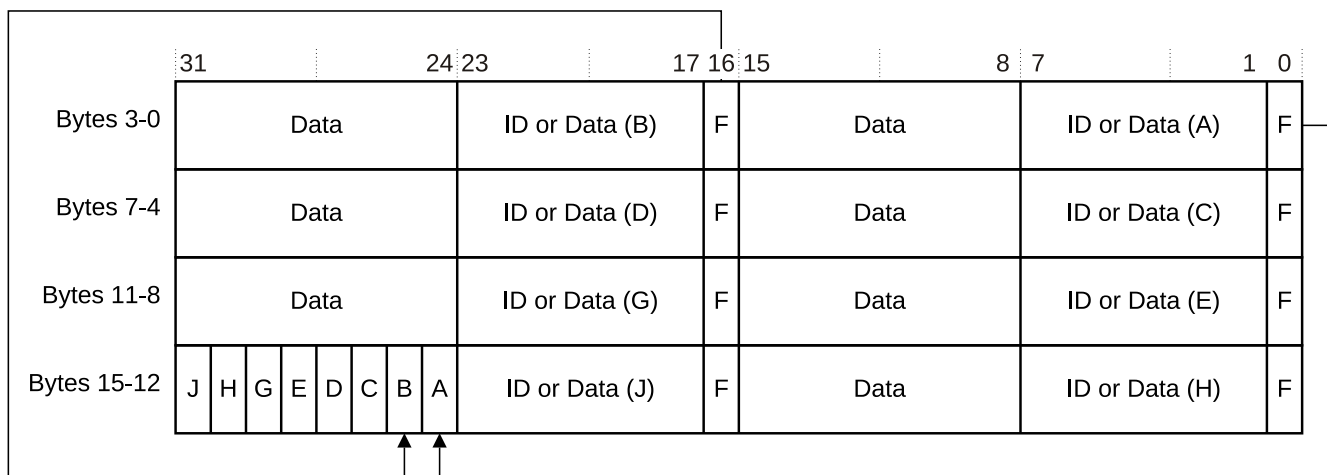


Figure D4-1 Formatter frame structure

Each time the ID changes to an ID that is not a special trace source ID, at least 1 byte of data must be output for that ID. Table D4-1 shows the meaning of each bit in a formatter frame. It is output least significant bit first, starting with bit 0.

Table D4-1 Meaning of bits in a formatter frame

Byte number	Bits	Description
0	0	ID or Data control for bits[7:1], marked F in Figure D4-1.
	7:1	Depends on bit 0: 0 = Data[7:1]. 1 = New ID.
1	7:0	Data[7:0].
2	7:0	ID or Data, see byte 0.
3	7:0	Data[7:0].
4	7:0	ID or Data, see byte 0.
5	7:0	Data[7:0].
6	7:0	ID or Data, see byte 0.
7	7:0	Data[7:0].

Byte number	Bits	Description
8	7:0	ID or Data, see byte 0.
9	7:0	Data[7:0].
10	7:0	ID or Data, see byte 0.
11	7:0	Data[7:0].
12	7:0	ID or Data, see byte 0.
13	7:0	Data[7:0].
14	7:0	ID or Data, see byte 0.
15	0	Auxiliary bit for byte 0, see bit in Figure D4-1 marked A. The meaning of this bit depends on whether byte 0 contains data or a new ID: Data = Data[0]. New ID: 0 = Byte 1 corresponds to the new ID. 1 = Byte 1 corresponds to the old ID. The new ID takes effect from the next data byte after byte 1.
	1	Auxiliary bit for byte 2, marked B in Figure D4-1 . See bit 0. If byte 2 contains a new ID, this bit indicates whether the new ID takes effect from byte 3 or the following data byte.
	2	Auxiliary bit for byte 4, marked C in Figure D4-1 . See bit 0.
	3	Auxiliary bit for byte 6, marked D in Figure D4-1 . See bit 0.
	4	Auxiliary bit for byte 8, marked E in Figure D4-1 . See bit 0.
	5	Auxiliary bit for byte 10, marked G in Figure D4-1 . See bit 0.
	6	Auxiliary bit for byte 12, marked H in Figure D4-1 . See bit 0.
	7	Auxiliary bit for byte 14, marked J in Figure D4-1 . See bit 0. If byte 14 is a new ID, this bit is reserved. It must be zero, and must be ignored when decompressing the frame. The new ID takes effect from the first data byte of the next frame.

D4.2.1 Frame example

Two trace sources with IDs of 0x03 and 0x15 generate trace data and are interleaved on the trace bus, presenting one word of data at a time.

The following stream of bytes is output by the formatter:

0x07, 0xAA, 0xA6, 0xA7, 0x2B, 0xA8, 0x54, 0x52, 0x52, 0x54, 0x07, 0xCA, 0xC6, 0xC7, 0xC8, 0x1C.

[Figure D4-2](#) shows the corresponding frame.

	31				24				23				17				16				15				8				7				1				0			
Bytes 3-0	Data 0xA7								Data 0x53								0				Data 0xAA								ID 0x03								1			
Bytes 7-4	Data 0x52								Data 0x2A								0				Data 0xA8								ID 0x15								1			
Bytes 11-8	Data 0xCA								ID 0x03								1				Data 0x54								Data 0x29								0			
Bytes 15-12	0	0	0	1	1	1	0	0	Data 0x64								0				Data 0xC7								Data 0x63								0			

Figure D4-2 Example formatter frame

Table D4-2 shows how this frame is decoded.

Table D4-2 Decoding the example formatter frame

Byte	Observation	Interpretation	Data	ID
0	Bit[0] is set.	This byte represents the new ID 0x03. Bit[0] of byte 15 is clear, so the new ID takes effect immediately.	-	0x03
1		Data byte for the trace with the new ID 0x03.	0xAA	0x03
2	Bit[0] is clear	This byte is a data byte for the trace with the current ID 0x03. Bit[0] of the data is taken from bit[1] of byte 15.	0xA6	0x03
3		Data byte.	0xA7	0x03
4	Bit[0] is set	This byte represents the new ID 0x15. Bit[2] of byte 15 is set, so the next data byte continues to use the current ID 0x03.	-	0x03
5		Data byte for the trace with the current ID 0x03.	0xA8	0x03
6	Bit[0] is clear.	This byte is a data byte for the trace with the new ID 0x15. Bit[0] of the data is taken from bit[3] of byte 15.	0x55	0x15
7		Data byte.	0x52	0x15
8	Bit[0] is clear.	This byte is a data byte. Bit[0] of the data is taken from bit[4] of byte 15.	0x53	0x15
9		Data byte.	0x54	0x15
10	Bit[0] is set.	This byte represents the new ID 0x03. Bit[5] of byte 15 is clear, so the new ID takes effect immediately.	-	0x03
11		Data byte for the trace with the new ID 0x03.	0xCA	0x03
12	Bit[0] is clear.	This byte is a data byte for the trace with the current ID 0x03. Bit[0] of the data is taken from bit[6] of byte 15.	0xC6	0x03
13		Data byte.	0xC7	0x03
14	Bit[0] is clear.	This byte is a data byte. Bit[0] of the data is taken from bit[7] of byte 15.	0xC8	0x03
15		Auxiliary bits.	-	-

D4.2.2 Frame synchronization packet

The frame synchronization packet enables a TPA or trace decompressor to find the start of a frame. It is output periodically between frames. It is output least significant bit first, starting with bit[0]. In continuous mode, the TPA might discard all frame synchronization packets after the start of a frame is found. See [Modes of operation](#) for more information about continuous mode.

Figure D4-3 shows a frame synchronization packet.

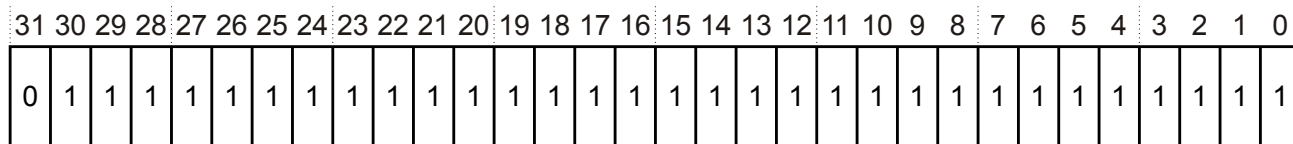


Figure D4-3 Full frame synchronization packet

This sequence cannot occur at any other time, if ID 0x7F has not been used. See [Special trace source IDs](#) for more information on reserved source IDs.

Note

Frame synchronization packets and frame data are always multiples of 32-bits, but do not always start on a 32-bit boundary. Because halfword synchronization packets can occur within frames and between frames, they can also start on 16-bit boundaries. See also [Halfword synchronization packet](#).

D4.2.3 Halfword synchronization packet

Halfword synchronization packets enable a TPA to detect when the trace port is idle and there is no trace to be captured. They observe the following rules:

- They are output between frames or within a frame.
- If they appear within a frame, they are always aligned to a 16-bit boundary.
- They are output least significant bit first, starting with bit[0].
- They are only generated in continuous mode. If a TPA detects a halfword synchronization packet, it must discard it, because it does not form part of a formatter frame. See [Modes of operation](#) for more information about continuous mode.

[Figure D4-4](#) shows a halfword synchronization packet.

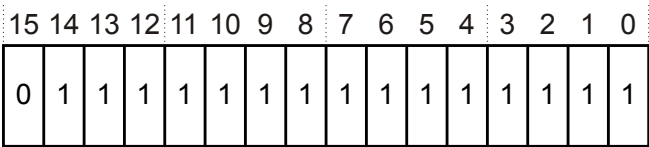


Figure D4-4 Halfword synchronization packet

Note

Frame synchronization packets and frame data are always multiples of 32-bits, but do not always start on a 32-bit boundary. Because halfword synchronization packets can occur within frames and between frames, they can also start on 16-bit boundaries.

D4.2.4 Special trace source IDs

The following IDs are for special purposes and must not be used under normal operation:

- | | | | | | | | |
|-----------|--|------|---|-----------|---|-----------|-----------|
| 0x00 | This ID indicates a NULL trace source. Any data associated with this ID must be ignored by the debugger, which is required if there is insufficient trace data available to complete a formatter frame. | | | | | | |
| 0x70-0x7A | Reserved. | | | | | | |
| 0x7B | <p>This ID indicates a flush response. Trace that is output with the flush response ID signifies that all trace that was generated previous to a flush request has been output.</p> <p>Each byte of trace that is output with ID 0x7B constitutes a separate flush response, whereby the value of the byte can be one of the following:</p> <table border="0"><tr><td>0x00</td><td>All active trace sources have indicated a flush response.</td></tr><tr><td>0x01-0x6F</td><td>The trace source ID with this value has indicated a flush response.</td></tr><tr><td>0x70-0xFF</td><td>Reserved.</td></tr></table> | 0x00 | All active trace sources have indicated a flush response. | 0x01-0x6F | The trace source ID with this value has indicated a flush response. | 0x70-0xFF | Reserved. |
| 0x00 | All active trace sources have indicated a flush response. | | | | | | |
| 0x01-0x6F | The trace source ID with this value has indicated a flush response. | | | | | | |
| 0x70-0xFF | Reserved. | | | | | | |

Note

The use of trace source ID 0x7B is also permitted on ATB, with the same payload semantics.

- | | |
|------|-----------|
| 0x7C | Reserved. |
|------|-----------|

0x7D	This ID indicates a trigger within the trace stream and is accompanied by one byte of data for each trigger. The value of each data byte indicates the ID of the trigger. A data byte with a value of zero indicates that the trigger ID is UNKNOWN.
------	--

Note

The use of trace source ID 0x7D is also permitted on ATB, with the same payload semantics.

0x7E	Reserved.
------	-----------

0x7F	This ID must never be used because it conflicts with the synchronization packet encodings.
------	--

D4.2.5 Data overheads

The formatter protocol adds an overhead of 6% to the bandwidth requirement of a trace port. It also requires one byte of extra trace every time the source ID changes. Arm recommends that components that arbitrate between trace sources switch between different source IDs as infrequently as possible.

Under certain conditions, the formatter can be bypassed to eliminate this overhead. For more information, see [Bypass](#).

D4.2.6 Repeating the trace source ID

If a large amount of consecutive trace is generated by a single source ID, the ID must be repeated periodically. This mechanism ensures that the debugger can determine the source of the trace even when the beginning of the trace has been lost.

Arm recommends repeating the source ID approximately every ten frames.

D4.2.7 Indication of alignment points

In most trace protocols, it is necessary to periodically indicate the beginning of a packet. This process is called alignment synchronization. Most protocols achieve alignment synchronization by periodically outputting a packet similar to the Frame Synchronization Packet.

When using a protocol that is unable to output such a packet, use the formatter protocol to indicate the position of synchronization points by using two source IDs. The trace source starts outputting trace using the first ID, then switches to the second ID at the first alignment point. It switches back to the first ID at the next alignment point, and continues switching at each subsequent alignment point.

A trace source that uses ID switching in this manner cannot use bypass mode. See [Bypass](#).

D4.3 Modes of operation

The formatter can operate in one of three modes. Not all modes are supported by all components that implement a formatter. For example, an ETB does not need to support continuous mode.

D4.3.1 Bypass

In this mode, the trace is output without modification. No formatting information is inserted into the trace stream. When using bypass mode, observe the following rules:

- Only one trace source ID is in use.
- If the trace is to be output over a trace port, the **TRACECTL** pin must be implemented, and the TPA must support this pin. This configuration enables the TPA to determine the position of the trigger and detect when no trace is available for capture. See [Decoding requirements for Trace Capture Devices](#).
- The debugger does not need to report the position of the trigger as seen by the trace sink. In bypass mode, the trigger ID `0x7D` is not generated.

To ensure that all trace is output from a trace sink when stopping trace, extra data might be added to the end of the trace stream. See [Bypass mode](#).

D4.3.2 Normal

The formatter is enabled, and the **TRACECTL** pin is used to determine the position of the trigger and detect when no trace is available for capture. Halfword synchronization packets are not generated. The TPA does not have to decode any part of the trace stream.

This mode is the easiest to support by TPAs designed for ETMs.

D4.3.3 Continuous

The formatter is enabled, but the **TRACECTL** pin is not used. The TPA must decode part of the formatter protocol to determine the position of the trigger and detect when no trace is available for capture.

D4.4 Flush of trace data at the end of operation

To support AMBA ATB protocol flushes, the formatter must ensure that all trace is output, because the trace that remains after a flush might be insufficient to complete a frame or use all the pins in the trace port. This section describes how the remaining trace must be formatted.

Note

When tracing is resumed, some leftover trace that is generated by the flush sequence might be output before any new trace is output. Look for the first synchronization packet in the protocol before starting to decompress the trace.

Trace that is output with the flush response ID $0 \times 7B$ signifies that all trace that was generated previous to a flush request has been output. See also [Special trace source IDs](#).

D4.4.1 Bypass mode

When running in bypass mode, if the formatter cannot guarantee that all trace has been output, it must output an extra sequence at the end of the trace. This mechanism ensures that all trace stored in the formatter is output, even if, for example, there is insufficient trace to use all the pins of a trace port.

The sequence consists of a single bit that is set, followed by a series of zeros. This sequence does not represent real trace data and must always be removed before decompression when the trace sink has been requested to stop trace output.

The following two examples show sequences that can be observed on a 32-bit trace port. [Figure D4-5](#) shows an example of how the last AMBA ATB protocol transaction left three bytes within the formatter.

31	24 23	16 15	8 7	0
0xAA [Real Data]	0x55 [Real Data]	0xAA [Real Data]	0x55 [Real Data]	
0x01	0x55 [Real Data]	0xAA [Real Data]	0x55 [Real Data]	
0x00	0x00	0x00	0x00	
0x00	0x00	0x00	0x00	

Figure D4-5 End of session example 1

[Figure D4-6](#) shows an example of how the trace finishes on a 32-bit trace port boundary.

31	24 23	16 15	8 7	0
0x55 [Real Data]	0xAA [Real Data]	0x55 [Real Data]	0xAA [Real Data]	
0x55 [Real Data]	0xAA [Real Data]	0x55 [Real Data]	0xAA [Real Data]	
0x00	0x00	0x00	0x01	
0x00	0x00	0x00	0x00	

Figure D4-6 End of session example 2

D4.4.2 Normal and continuous mode

When running in normal or continuous mode, the formatter must complete the frame currently being output, using the null ID encoding, ID 0×00 . Any data that is associated with this ID can be ignored. More frames of data corresponding to the null ID can be generated to ensure that all trace has been output.

Chapter D5

About ROM Tables

The chapter describes ROM Tables. It includes the following sections:

- [ROM Tables Overview.](#)
- [ROM Table Types.](#)
- [Component and Peripheral ID Registers for ROM Tables.](#)

D5.1 ROM Tables Overview

ROM Tables hold information about debug components.

- Systems with a single debug component do not require a ROM Table. However, a designer might choose to implement such a system to include a ROM Table.
- Systems with more than one debug component must include at least one ROM Table.

A ROM Table connects to a bus controlled by a *Memory Access Port* (MEM-AP). In other words, the ROM Table is part of the address space of the memory system that is connected to a MEM-AP. More than one ROM Table can be connected to a single bus.

A ROM Table always occupies 4KB of memory.

D5.2 ROM Table Types

The following types of ROM Tables in *Arm® Debug Interface Architecture Specification (ADIV6.0)* are permitted to be used with components that comply with CoreSight version 3.0:

Class 0x1 ROM Tables

In a Class 0x1 ROM Table implementation:

- The Component class field, CIDR1.CLASS, is 0x1, which identifies the component as a Class 0x1 ROM Table.
- The PIDR4.SIZE field must be 0.
- A ROM Table must occupy a single 4KB block of memory.
- A Class 0x1 ROM Table is a read-only device.

For a detailed description of the Class 0x1 ROM Table entries and registers, see *Arm® Debug Interface Architecture Specification (ADIV6.0)*.

Class 0x9 ROM Tables

In a Class 0x9 ROM Table implementation:

- The Component class field, CIDR1.CLASS, is 0x9, which identifies the component as a CoreSight Component.
- The DEVTYPE and DEVID registers contain information about the ROM Table.
- The PIDR4.SIZE field must be 0.
- A ROM Table must occupy a single 4KB block of memory.
- Class 0x9 ROM Table entries are 32 or 64 bits wide.

For a detailed description of the Class 0x9 ROM Table entries and registers, see *Arm® Debug Interface Architecture Specification (ADIV6.0)*.

Note

Class 0x9 ROM Tables can be used alongside Class 0x1 ROM Tables, and both Class 0x9 and Class 0x1 ROM Tables might be present in systems that comply with CoreSight v3.

D5.3 Component and Peripheral ID Registers for ROM Tables

Any ROM Table must implement a set of Component and Peripheral ID Registers, that start at offset `0xFD0` in the ROM Table. [PIDR0-PIDR7, Peripheral Identification Registers](#) in [CoreSight programmers' model](#) describes these registers. This section only describes particular features of the registers when they relate to a ROM Table.

D5.3.1 Identifying the debug SoC, system, or subsystem

The Unique Component Identifier in a ROM table uniquely identifies the SoC, platform, or subsystem described by the ROM table. For example:

- A cluster of components grouped together with a ROM table hierarchy pointing to all the components is uniquely identified by the outermost ROM Table in the cluster.
- A subsystem of all components connected to a single MEM-AP is uniquely identified by the outermost ROM Table in the subsystem. This ROM Table is usually the first component pointed to by the MEM-AP.
- An SoC, consisting of multiple MEM-APs implementing the ADIV5, is uniquely identified by the collective Unique Component Identifiers from all of the outermost ROM Tables pointed to by each of the Memory Access Ports.
- An SoC, consisting of multiple MEM-APs implementing the ADIV6, is uniquely identified by the Unique Component Identifiers from the outermost ROM Table providing pointers to each of the MEM-APs. This ROM Table is usually the first component pointed to by the DP.

An SoC, system, or subsystem might be configurable when being built. For example, a cluster of processors might permit the number of processors to be configurable. The ROM Table, which describes such a collection of components, might have the same Unique Component Identifier for all configurations of the system. However, this is only permitted when components are either included or excluded, and is not permitted to be the same when the location of any component in the address map changes or components significantly change in function. In effect, a ROM Table Unique Component Identifier uniquely identifies a superset configuration of the collection of components. ROM Tables with the same Unique Component Identifier might only describe a subset of this superset.

An Anonymous ROM Table does not have a Unique Component Identifier. See the *Arm[®] Debug Interface Architecture Specification ADIV6.0* for more information.

Chapter D6

Topology Detection at the System Level

This chapter describes topology detection at the system level. It contains the following sections:

- [About topology detection at the system level.](#)
- [Detection.](#)
- [Components that are not recognized.](#)
- [Detection algorithm.](#)

D6.1 About topology detection at the system level

[Topology Detection](#) describes the topology detection requirements of CoreSight components. [Topology Detection at the Component Level](#) describes how to perform topology detection on each interface type. This chapter describes how debuggers can use this information to detect the topology of a target system.

D6.2 Detection

When connecting to a CoreSight system, a debugger performs the following steps:

1. It finds the DP.
2. It ensures that the system is powered up, and that its clocks are running. The DP provides facilities to assist with this assessment.
3. It looks for a ROM Table with the location of all components.
4. It compares the Peripheral ID of the ROM Table against a list of saved system descriptions. For information on this ID, see PIDR0-PIDR7 in *Arm[®] Debug Interface Architecture Specification ADIV6.0*.
5. If a description of the system with this ID is saved, it uses that description. If not, the debugger continues with the following steps:
 - a. It identifies each component.
 - b. It looks up information that is known about that component to determine what interfaces are supported and how to control them for topology detection.
 - c. It performs topology detection. See [Detection algorithm](#).
 - d. It saves the description for later use.

D6.2.1 Saved descriptions

Because topology detection can be invasive, it is important that the description of the system is saved when discovered, so that the debugger can be connected non-invasively in the future. The debugger must have the possibility to force topology detection to be redone, in case two different targets are accidentally assigned the same ROM Table ID.

Note

Software running on the system must be able to determine the topology of the CoreSight system, and keep functioning when topology detection registers are enabled.

D6.3 Components that are not recognized

When an unrecognized component is encountered, the JEDEC code and CoreSight component class of the component is used to indicate which type of component has been encountered and who to ask for further information. Alternatively, [DEVARCH](#), if present, can be used to determine the generic architecture of a component. The component must be otherwise ignored.

D6.4 Detection algorithm

Arm recommends that a debugger connecting to a system executes the following algorithm, to determine the topology of the system:

```
for each component, c
    execute (component preamble) for c
for each interface type, t
    for each transmitter interface and bidirectional interface of type t, m
        execute (transmitter preamble) for interface m
    for each receiver interface and bidirectional interface of type t, s
        execute (receiver preamble) for interface s
    for each transmitter interface and bidirectional interface of type t, m
        execute (transmitter assert) for interface m
    for each receiver interface and bidirectional interface of type t, s
        if (receiver check asserted) for interface s
            record connection between m and s
    for each slave interface and bidirectional interface of type t, s
        execute receiver post-assert for interface s
    execute (transmitter deassert) for interface m
    for each receiver interface and bidirectional interface of type t, s
        if not (receiver check deasserted) for interface s
            raise error
    for each receiver interface and bidirectional interface of type t, s
        execute (receiver post-deassert) for interface s
for each component, c
    execute (component postamble) for c
```

[Signals for topology detection](#) describes preambles, and assert and deassert sequences for common interfaces. If a component does not specify a preamble or postamble, they are as follows:

Component preamble

Set [ITCTRL.IME](#) to 0b1.

Component postamble

Clear [ITCTRL.IME](#) to 0b0.

Note

After a device has been in integration mode, it might behave differently than before. After performing integration or topology detection, reset the system to ensure correct behavior of CoreSight and other connected system components that are affected by the integration or topology detection.

Chapter D7

Compliance Requirements

This chapter describes the requirements for CoreSight compliance. It contains the following sections:

- [About compliance classes.](#)
- [CoreSight debug.](#)
- [CoreSight trace.](#)
- [Multiple DPs.](#)

D7.1 About compliance classes

This chapter defines the requirements that a system must meet to claim CoreSight compliance. It refers to specific revisions of components available from Arm.

These requirements are aimed at interoperability between debuggers, and only cover behavior that is visible to such tools. The following behavior is specified:

- Minimum functionality. This functionality must be available in all compliant systems.
- Optional functionality. Arm recommends that debuggers aiming to support compliant systems support this functionality.

Note

Systems can implement extra functionality, provided it does not affect the use of the minimum functionality. Debuggers might not be able to support this extra functionality.

Two levels of compliance are defined:

- CoreSight debug, which is the basic level of compliance. A processor supporting CoreSight debug does not need to comply with the CoreSight visible component architecture, although doing so makes it easier to build a CoreSight system.
- CoreSight trace, which includes all the requirements for CoreSight debug, and adds trace functionality.

The level of compliance is claimed for each individual processor in the system. For example, a system incorporating three processors might claim CoreSight trace for the first processor, CoreSight debug for the second, and no CoreSight compliance for the third.

D7.2 CoreSight debug

This section defines the CoreSight debug compliance class.

Note

A CoreSight component is a component that implements the CoreSight visible component architecture.

D7.2.1 Minimum debug functionality

Systems claiming CoreSight debug compliance must conform to the following rules:

- Each CoreSight system must contain exactly one DP, and the JTAG or Serial Wire interface of the DP must be accessible to debug tools. For more information about implementing multiple systems containing DPs, see [Multiple DPs](#). The permitted DPs are as follows:
 - A JTAG-DP.
 - A SW-DP.
 - A combined DP that offers the functionality of both JTAG-DP and SW-DP.
- Each CoreSight component must comply with all the following requirements:
 - It is accessible through at least one of the following:
 - A MEM-AP.
 - A DP.
 - It is discoverable through a valid ROM Table, that must itself conform to the requirements for CoreSight components.
 - When starting from the DP, the component is visible in exactly one ROM table. This does not preclude the component being accessible through multiple MEM-APs, but that it must only be reported in a single ROM table.
- A processor claiming CoreSight debug compliance must observe at least one of the following requirements:
 - It conforms to the CoreSight visible component architecture, while conforming to the requirements for CoreSight components.
 - It is accessible using a JTAG TAP Controller that is connected in series with the JTAG TAP Controller of the JTAG-DP, connected to the **TDI** side of the JTAG-DP as [Figure D7-9](#) shows.
 - It is accessible using a JTAG TAP Controller that is connected in a chain of TAP Controllers that are controlled by a JTAG-AP.
- All debug functionality for a component must be visible and accessible when a debug power up has been requested, where that debug power up request is at least one of the following:
 - A Debug Power Up request in the DP programmers' model, except where it is ignored due to security restrictions.
 - A Debug Power Up request from a Granular Power Requester or ROM table, where that request covers the domain containing the component and no finer-grained requests exist for that component. Such a request might be ignored due to security restrictions.
- All debug functionality must be operational when a System Power Up has been requested, where that system power up request is at least one of the following:
 - A system power up request in the DP programmers' model, except where it is ignored due to security restrictions.
 - A system power up request from a Granular Power Requester or ROM table, where that request covers the domain containing the component and no finer-grained requests exist for that component. Such a request might be ignored due to security restrictions.
- Arm recommends that debug functionality is reset to its initial state when Debug Reset is requested in the DP programmers' model or through a Granular Power Requester or ROM table.
- For each CoreSight component and JTAG controlled processor, all inputs and outputs that are defined as type event are connected to a CTI component, unless there is only one component in the system with event inputs or outputs, in which case no CTI is required. For Arm JTAG controlled processors, the required connections are documented in the *CoreSight Technology System Design Guide*.
- All channel interfaces of CoreSight components, for example interfaces that are present on CTIs, are connected together, so that the channels are shared between all components. CoreSight technology from Arm provides a CTM for connecting three or more channel interfaces together where required.

- Unless stated otherwise in this specification, extra logic between components that is visible to the tools is not permitted. See also [Variant interfaces](#).
- Arm recommends all system designs to be CoreSight compliant, but recognizes that this recommendation might not always be achievable. If a system requires certain operations to be performed before it complies with the CoreSight compliance criteria, clearly state what these operations are, and clearly state that it is not CoreSight compliant until they have been performed.

D7.2.2 Optional debug functionality

CoreSight debug systems can also implement visibility of system components through a MEM-AP.

D7.2.2.1 Single-core debug

[Figure D7-1](#) shows the simplest CoreSight debug configuration for a single-core system. In this configuration, no trace capabilities are provided. The processor is accessed via a JTAG-AP, to ensure that it can be powered down without affecting other components on the master JTAG TAP chain.

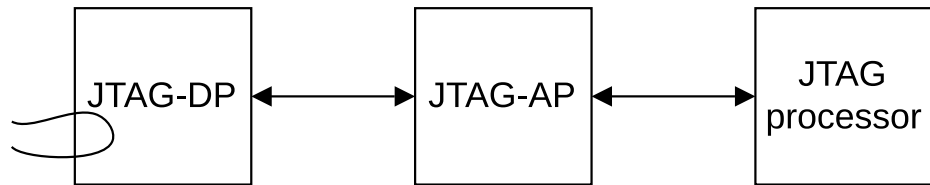


Figure D7-1 Single core with JTAG debug access

D7.2.2.2 Multi-core debug

[Figure D7-2](#) shows a multi-core CoreSight debug system:

- One of the processors is a fully compliant CoreSight component.
- Cross triggering is supported between processors.
- Both processors provide access to program the CTI and processor with interfaces that comply with the CoreSight architecture.

An alternative method to provide memory access that is not shown in the figure is to use AHB-AP.

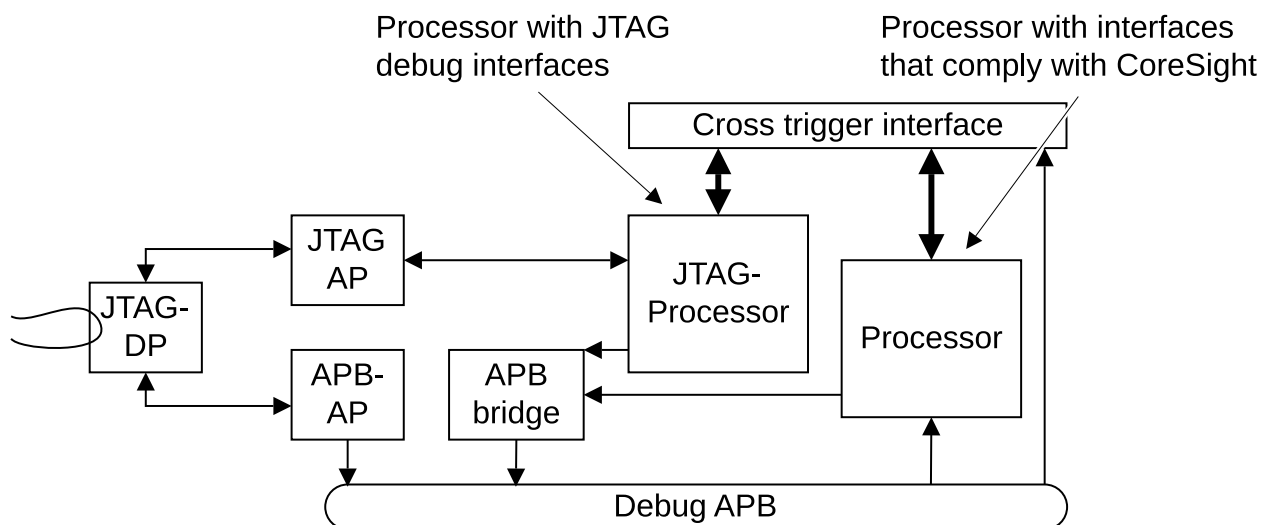


Figure D7-2 Multi-core system

D7.3 CoreSight trace

This section defines the CoreSight trace compliance class.

D7.3.1 Minimum trace functionality

Systems claiming CoreSight trace compliance must comply with the minimum requirements for CoreSight debug, plus the following:

- All Arm-compatible processors claiming CoreSight trace compliance must implement an Arm CoreSight ETM.
 - Processors that are not Arm-compatible must implement a trace solution that complies with the following requirements:
 - It must implement the CoreSight visible component architecture.
 - It must provide the processor with at least instruction trace as a CoreSight trace source.
 - The system must implement one or more trace sinks:
 - If a TPIU is implemented, its output is connected to a compliant connector as defined in [Physical Interface](#).
 - All CoreSight trace sources must drain into one or more of the trace sinks:
 - Where two or more trace sources drain into the same trace sink, they are connected through one or more CoreSight trace funnels.
 - The same trace cannot travel through multiple active paths to reach the same endpoint. This requirement is met by a combination of hardware design which prevents such multiple paths, and programming of the system to prevent trace from actively travelling on multiple paths.
- The following figures show examples of compliant and non-compliant scenarios.

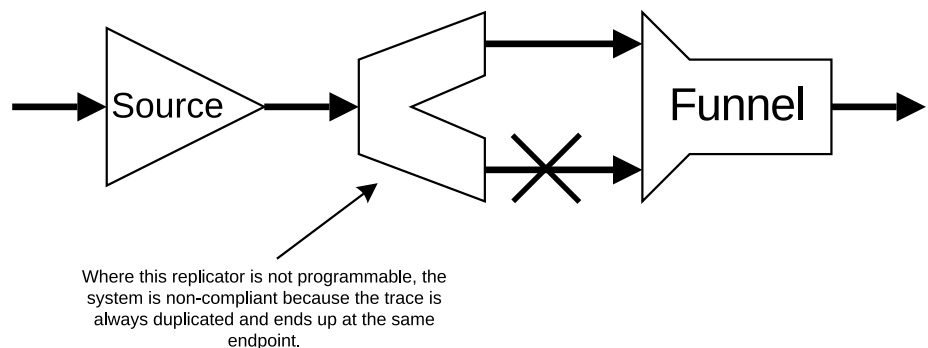


Figure D7-3 Non-compliant Replicator and CoreSight trace funnel connection

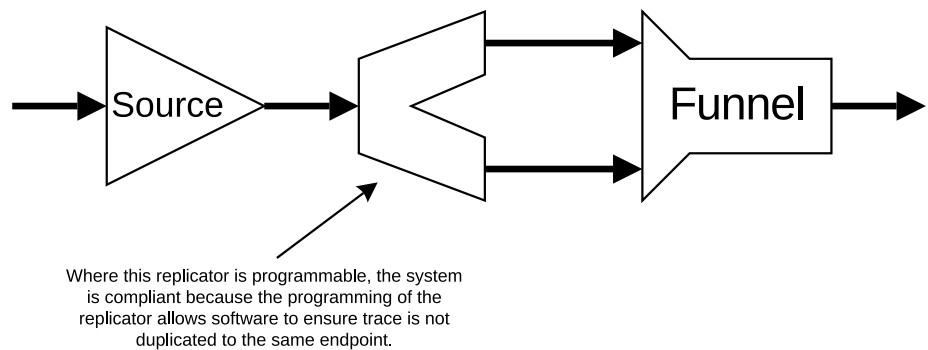


Figure D7-4 Compliant Replicator and CoreSight trace funnel connection

A particular example that must be avoided is feedback. See example in [Figure D7-5](#).

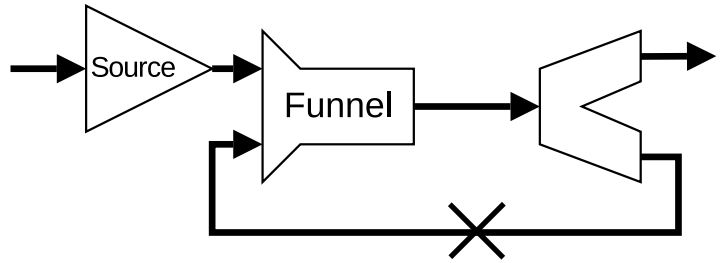


Figure D7-5 Non-compliant feedback loop

D7.3.2 Optional trace functionality

CoreSight debug systems can also implement CoreSight debug optional functionality and tracing of AHB buses using the Arm *AHB Trace Macrocell* (HTM).

D7.3.2.1 Basic single-core trace

Figure D7-6 shows an example system with single-core trace using the CoreSight infrastructure. The ETM, which complies with the CoreSight architecture, outputs directly to a TPIU for direct output of core trace off-chip. The tracing of only a single trace source enables the TPIU to be configured in bypass mode because source IDs do not need to be embedded in the trace data.

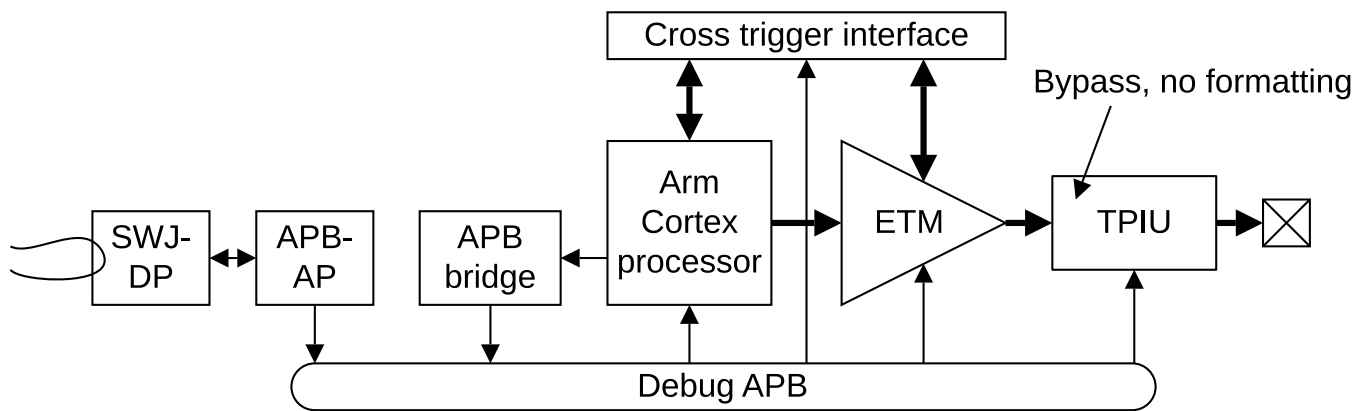


Figure D7-6 Single-core trace with formatting bypass

D7.3.2.2 Advanced single-core trace

Figure D7-7 shows an example system with full trace capabilities in a single-core system. The ETM provides Arm processor tracing, and the HTM provides bus tracing. The CoreSight trace funnel combines trace from both sources into a single trace stream, that is then replicated to provide on-chip storage using the CoreSight ETB and output off-chip using the TPIU. Components can be configured via the Debug-APB and cross triggered using the CTIs, through the CTM.

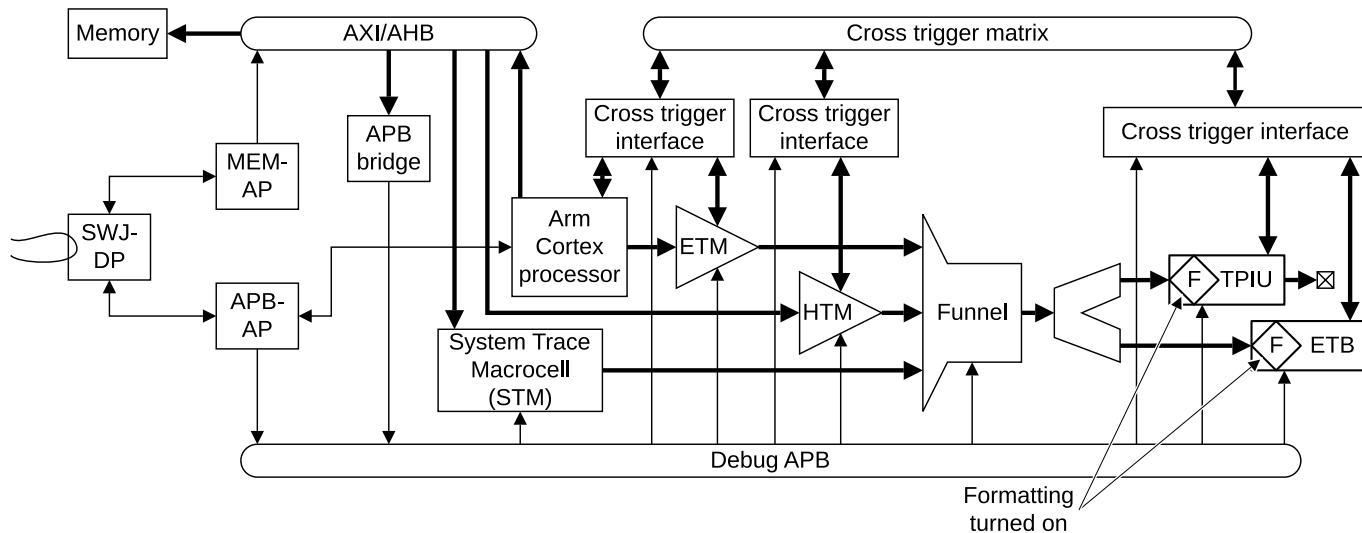


Figure D7-7 Full CoreSight trace with single core

D7.3.2.3 Multi-core trace

Figure D7-8 shows a system with an Arm processor and a DSP. A third smaller subsystem is added to support merging of multiple CoreSight AMBA ATB interfaces into a single trace stream.

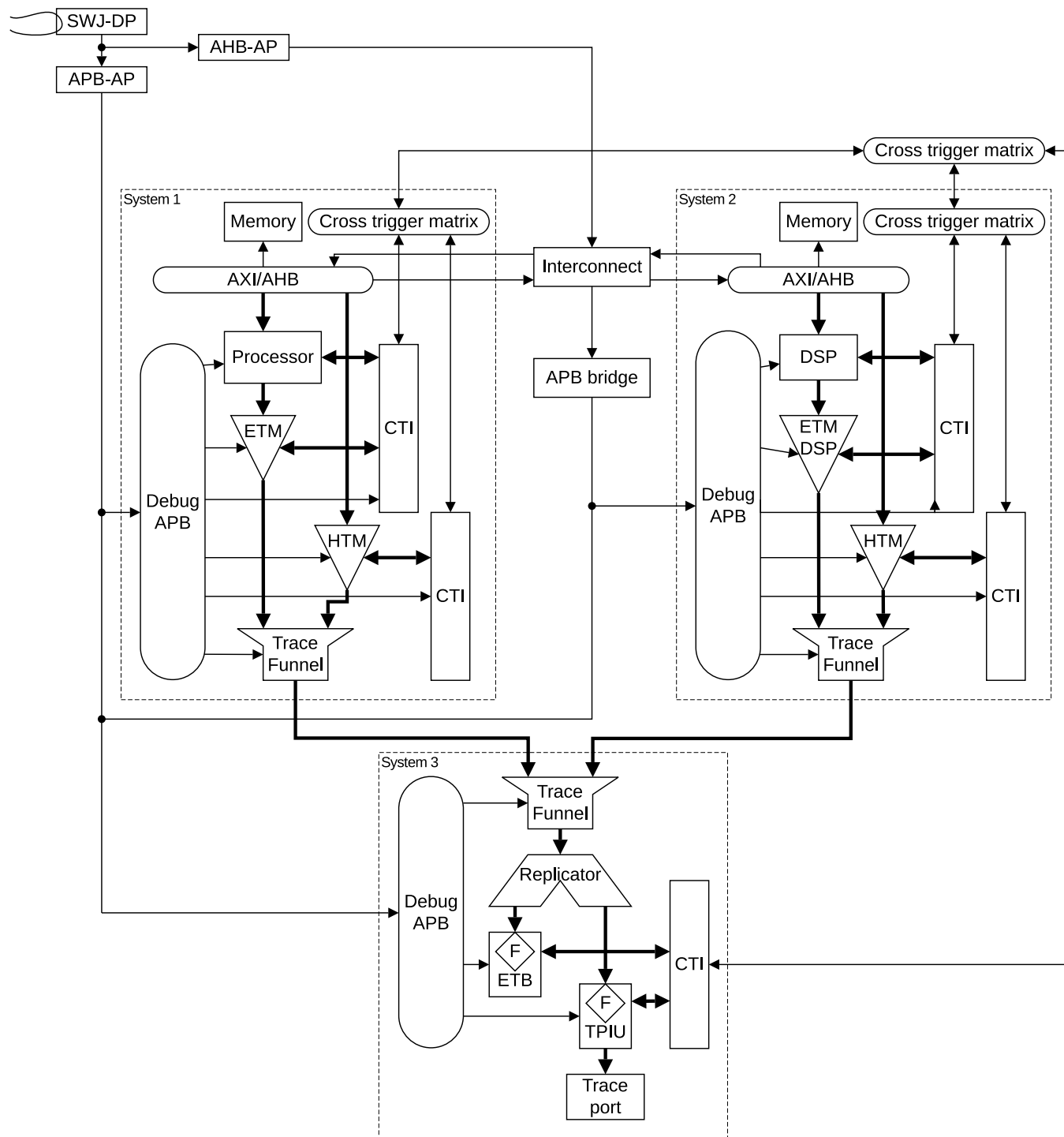


Figure D7-8 Full system trace with Arm processor and CoreSight compliant DSP

D7.4 Multiple DPs

In the context of this specification, a system is defined as one of the following:

- All components that are accessible through a single DP, a MEM-AP, or a JTAG-AP.
- All components before a JTAG-DP in a serial JTAG TAP chain.

The following rules apply to the arrangement of multiple DPs:

- Connections between JTAG TAP Controllers cannot be interleaved between systems. For example, if there are two systems sharing a JTAG TAP chain, each with a JTAG-DP and two JTAG processors connected in series with the JTAG-DP, the connections that are shown in [Figure D7-9](#) are permitted, while the connections shown in [Figure D7-10](#) are not permitted.

In [Figure D7-9](#):

- System 1 is defined as the two processors before the first JTAG-DP in the TAP chain.
- System 2 is defined as the two processors before the second JTAG-DP in the TAP chain.

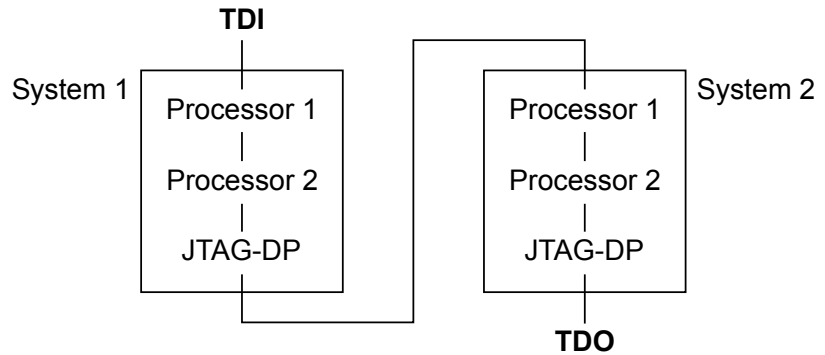


Figure D7-9 JTAG connections across systems

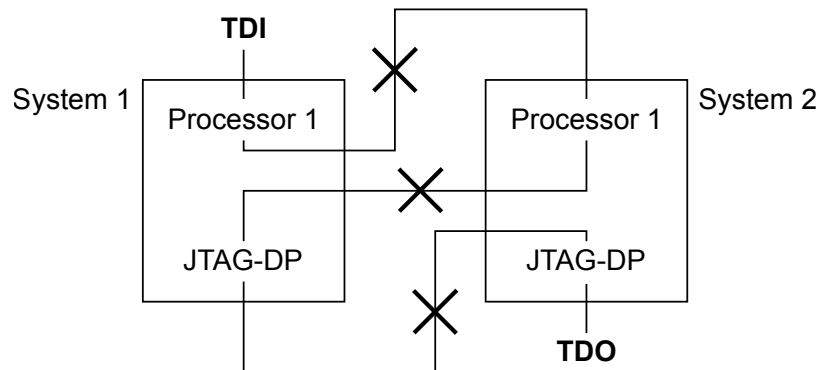


Figure D7-10 Non-compliant interleaved JTAG connections across systems

- Extra JTAG TAP Controllers can be implemented in series with JTAG TAP Controllers of the CoreSight systems. For example, in [Figure D7-11](#), processor A is not part of either CoreSight system 1 or 2. The debugger considers processor A to be part of system 2, because the JTAG-DP closest to the **TDO** side of processor A is in system 2. If the debugger does not recognize processor A, then it is ignored, otherwise the debugger attempts topology detection on system 2 with processor A, and fails to find any connections between processor A and system 2.

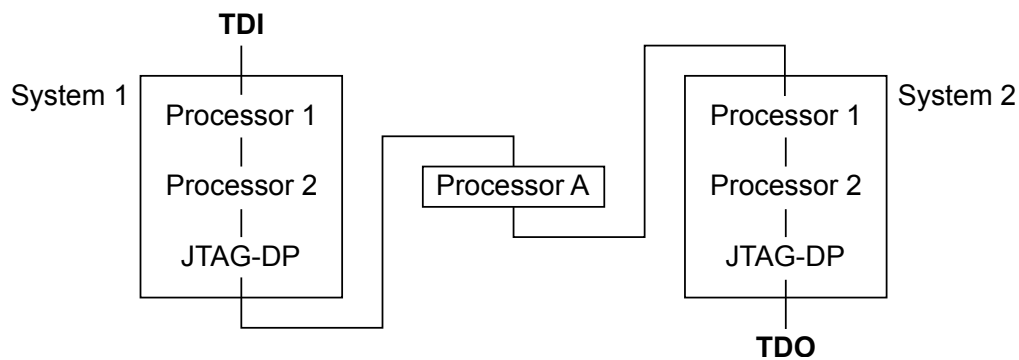


Figure D7-11 Systems with extra JTAG TAP Controllers

- A JTAG-DP must not be accessed through the JTAG-AP of another system, as shown in [Figure D7-12](#).

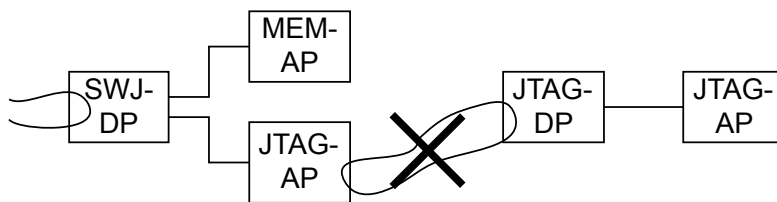


Figure D7-12 Non-compliant JTAG-DP connection

Part E
Appendixes

Chapter E1

Power Requester

This appendix describes the power requester which Arm recommends that some CoreSight components implement. It contains the following sections:

- [About the power requester.](#)
- [Register descriptions.](#)
- [Powering non-visible components.](#)

E1.1 About the power requester

The power requester belongs to the component Class 0x9, CoreSight component.

The power requester can control the application or removal of power for up to 32 power domains.

E1.2 Register descriptions

Table E1-1 shows the power requester registers, in order of their address offset in the 4KB block where the programmers' model resides. The remainder of the chapter describes how to implement the registers, in alphabetical order.

Table E1-1 Power requester register summary

Offset	Type	Name	Description
0x000	RW	CDBGWUPREQ	Debug Power Request Register
0x004	RO	CDBGWUPACK	Debug Power Request Acknowledge Register
0x008 - 0xEFC	RES0	-	Reserved
0xF00	RW	ITCTRL	Integration Mode Control Register
0xF04 - 0xF9C	RES0	-	Reserved
0xFA0	RW	CLAIMSET	Claim Tag Set Register
0xFA4	RW	CLAIMCLR	Claim Tag Clear Register
0xFA8 - 0xFAC	RES0	-	Reserved
0xFB0	WO	LAR	Software Lock Access Register
0xFB4	RO	LSR	Software Lock Status Register
0xFB8	RO	AUTHSTATUS	Authentication Status Register
0xFBC	RO	DEVARCH	Device Architecture Register
0xFC0 - 0xFC4	RES0	-	Reserved
0xFC8	RO	DEVID	Device configuration Register
0xFCC	RO	DEVTYPE	Device Type identifier Register
0xFD0 - 0xFDC	RO	PIDR4-PIDR7	Peripheral Identification Registers
0xFE0 - 0xFEC	RO	PIDR0-PIDR3	
0xFF0 - 0xFFC	RO	CIDR0-CIDR3	Component Identification Registers

E1.2.1 AUTHSTATUS, Authentication Status Register

For a full description of this register, see [AUTHSTATUS, Authentication Status Register](#).

The AUTHSTATUS characteristics are:

Purpose Reports the required security level and status of the authentication interface. Where functionality changes on a given security level, this change in status must be reported in this register.

Usage constraints

None.

Default

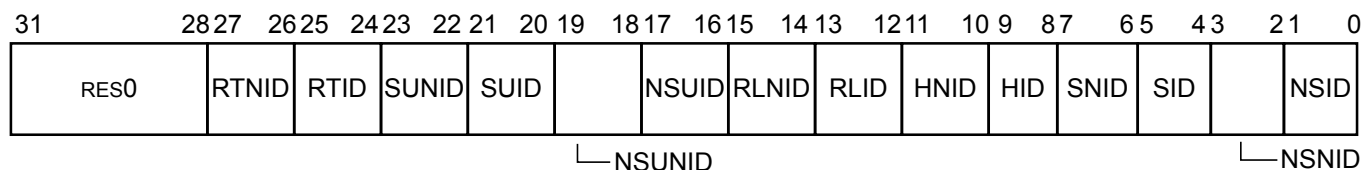
RO

Configurations Included in all implementations.

Attributes A 32-bit register.

E1.2.1.1 Field Descriptions

The AUTHSTATUS bit assignments are:



Bits[31:28]

RES0. See [AUTHSTATUS, Authentication Status Register](#).

RTNID, bits [27:26]

Root non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

RTID, bits [25:24]

Root invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

SUNID, bits[23:22]

Secure Unprivileged non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

SUID, bits[21:20]

Secure Unprivileged invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

NSUNID, bits[19:18]

Non-secure Unprivileged non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

NSUID, bits[17:16]

Non-secure Unprivileged invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

RLNID, bits[15:14]

Realm non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

RLID, bits[13:12]

Realm invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

HNID, bits[11:10]

Hypervisor non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

HID, bits[9:8]

Hypervisor invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

SNID, bits[7:6]

Secure non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

SID, bits[5:4]

Secure invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

NSNID, bits[3:2]

Non-secure non-invasive debug. See [AUTHSTATUS, Authentication Status Register](#).

NSID, bits[1:0]

Non-secure invasive debug. See [AUTHSTATUS](#), Authentication Status Register.

E1.2.1.2 Accessing AUTHSTATUS

AUTHSTATUS can be accessed at the following address:

Component	Offset
GPR	0xFB8

E1.2.2 CDBGPWRUPACK, Debug Power Request Acknowledge Register

The CDBGPWRUPACK characteristics are:

Purpose Returns the status of the power requests that [CDBGPWRUPREQ](#) issues.

Usage constraints

The register can monitor the power requests for up to 32 power domains.

CDBGPWRUPACK is accessible as follows:

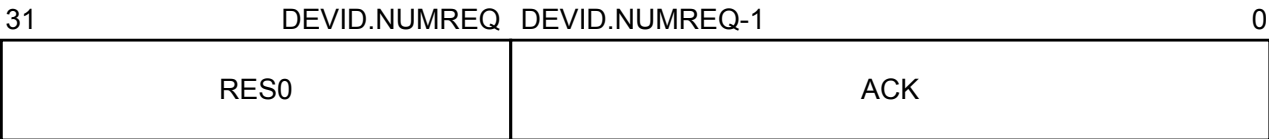
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit register.

E1.2.2.1 Field Descriptions

The CDBGPWRUPACK bit assignments are:



Bits[31:DEVID.NUMREQ]

RES0

ACK, bits[DEVID.NUMREQ-1:0] The size of this field is IMPLEMENTATION DEFINED, and equals the value of [DEVID.NUMREQ](#).

Permitted values of bit[n] are:

0	Power domain n is not powered.
1	Power domain n is powered.

E1.2.2.2 Accessing CDBGPWRUPACK

CDBGPWRUPACK can be accessed at the following address:

Component	Offset
GPR	0x004

E1.2.3 CDBGPWRUPREQ, Debug Power Request Register

The CDBGPWRUPREQ characteristics are:

Purpose Controls whether a power request is active for a power domain.

Usage constraints

CDBGPWRUPREQ can issue power requests for up to 32 power domains.
CDBGPWRUPREQ is accessible as follows:

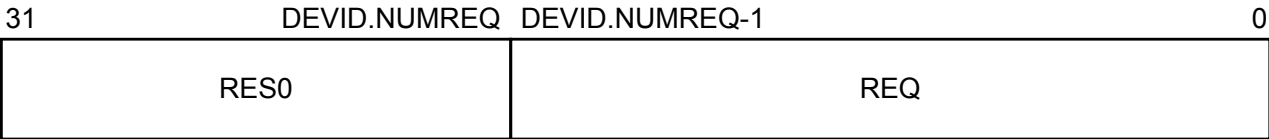
Default
RW

Configurations Included in all implementations.

Attributes A 32-bit register.

E1.2.3.1 Field Descriptions

The CDBGPWRUPREQ bit assignments are:



Bits[31:DEVID.NUMREQ]
RES0.

REQ, bits[DEVID.NUMREQ-1:0] The size of this field is IMPLEMENTATION DEFINED, and equals the value of [DEVID.NUMREQ](#).
Permitted values of bit[n] are:

0	Power request for power domain <i>n</i> is not active.
1	Power request for power domain <i>n</i> is active.

E1.2.3.2 Accessing CDBGPWRUPREQ

CDBGPWRUPREQ can be accessed at the following address:

Component	Offset
GPR	0x000

E1.2.4 CIDR0-CIDR3, Component Identification Registers

This section describes the bit assignments for GPR components that implement the CIDR0-CIDR3. For a full description of the CIDR, see [Component and Peripheral Identification Registers](#).

The CIDR characteristics are:

Purpose Provide information to identify a CoreSight component.

Usage constraints

CIDR0-CIDR3 are accessible as follows:

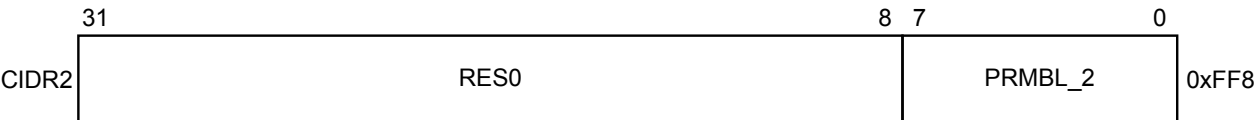
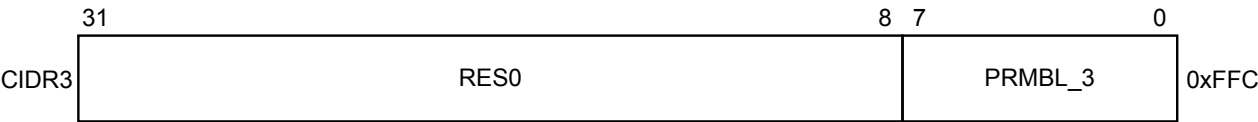
Default
RO

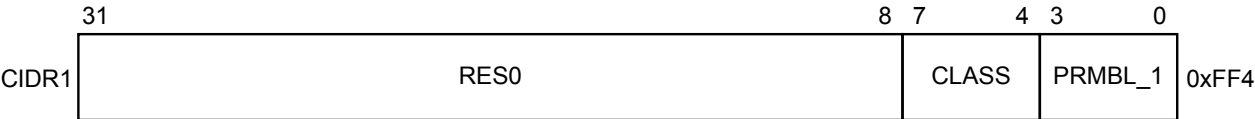
Configurations Included in all implementations.

Attributes Four 32-bit management registers.

E1.2.4.1 Field Descriptions

The CIDR0 bit assignments are:





CIDR3 bits[31:8]

RES0.

PRMBL_3, CIDR3 bits[7:0]

0xB1.

CIDR2 bits[31:8]

RES0.

PRMBL_2, CIDR2 bits[7:0]

0x05.

CIDR1 bits[31:8]

RES0.

CLASS, CIDR1 bits[7:4]

0x9 CoreSight component.

PRMBL_1, CIDR1 bits[3:0]

0x0.

CIDR0 bits[31:8]

RES0.

PRMBL_0, CIDR0 bits[7:0]

0x0D.

E1.2.4.2 Accessing the CIDR

CIDR0-CIDR3 can be accessed at the following address:

Component	Offset			
	CIDR0	CIDR1	CIDR2	CIDR3
GPR	0xFF0	0xFF4	0xFF8	0xFFC

E1.2.5 CLAIMCLR, Claim Tag Clear Register

For a full description of this register, and how to deploy it in a claim tag protocol, see [CLAIMSET and CLAIMCLR, Claim Tag Set Register and Claim Tag Clear Register](#).

The CLAIMCLR characteristics are:

Purpose Clears claim tags and returns the current claim tag values.

Usage constraints

Must be used with [CLAIMSET](#).

To indicate the width of the area that represents valid claim tags, a component must use [CLAIMSET](#).

If CLAIMCLR and CLAIMSET are implemented, all debug agents that use them must implement a common claim tag protocol.

The value of CLAIMCLR must be zero at reset.

CLAIMCLR is accessible as follows:

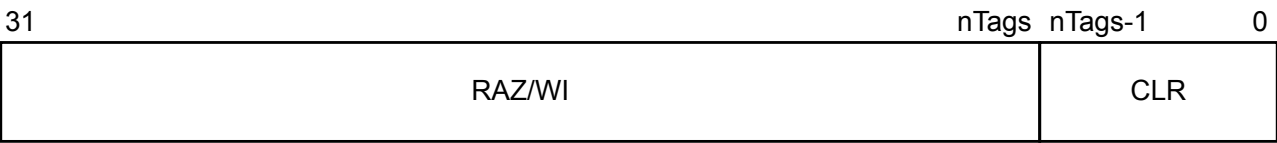
Default
RW

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.5.1 Field Descriptions

The CLAIMCLR bit assignments are:



Bits[31:nTags]

RAZ/WI.

CLR, bits[nTags-1:0]

The size n of this field is IMPLEMENTATION DEFINED, and equals the number of bits set in [CLAIMSET](#).

Permitted values of bit[n] are:

- | | |
|----------------|--|
| Write 0 | No effect. |
| Write 1 | Clear the claim tag for bit[n]. |
| Read 0 | The debug functionality that is tagged by bit[n] is available. |
| Read 1 | The debug functionality that is tagged by bit[n] is claimed. |

E1.2.5.2 Accessing CLAIMCLR

CLAIMCLR can be accessed at the following address:

Component	Offset
GPR	0xFA4

E1.2.6 CLAIMSET, Claim Tag Set Register

For a full description of this register, and how to deploy it in a claim tag protocol, see [CLAIMSET and CLAIMCLR, Claim Tag Set Register and Claim Tag Clear Register](#).

The CLAIMSET characteristics are:

Purpose Sets claim tags and returns the valid claim tags.

Usage constraints

Must be used with [CLAIMSET](#).

The bits indicating valid claim tags must be consecutive.

If CLAIMCLR and CLAIMSET are implemented, all debug agents that use them must implement a common claim tag protocol.

CLAIMSET is accessible as follows:

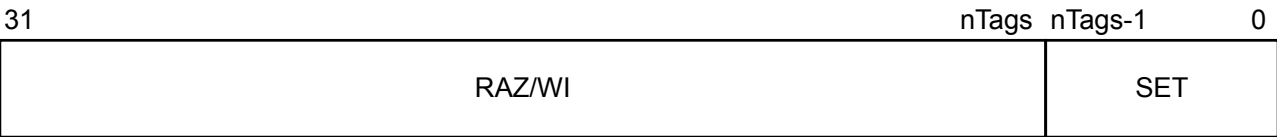
Default
RW

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.6.1 Field Descriptions

The CLAIMSET bit assignments are:



Bits[31:nTags]

RAZ/WI.

SET, bits[nTags-1:0]

The size n of this field is IMPLEMENTATION DEFINED.

Permitted values of bit[n] are:

Write 0	No effect.
Write 1	Set the claim tag for bit[<i>n</i>].
Read 0	The claim tag that is represented by bit[<i>n</i>] is implemented.
Read 1	The claim tag that is represented by bit[<i>n</i>] is not implemented.

E1.2.6.2 Accessing CLAIMSET

CLAIMSET can be accessed at the following address:

Component	Offset
GPR	0xFA0

E1.2.7 DEVARCH, Device Architecture Register

This section describes the bit assignments for GPR components that implement DEVARCH. For a full description of DEVARCH, see [DEVARCH, Device Architecture Register](#).

The DEVARCH characteristics are:

Purpose Identifies the architect and architecture of a CoreSight component. The architect might differ from the designer of a component, for example when Arm defines the architecture but another company designs and implements the component.

Usage constraints

DEVARCH is accessible as follows:

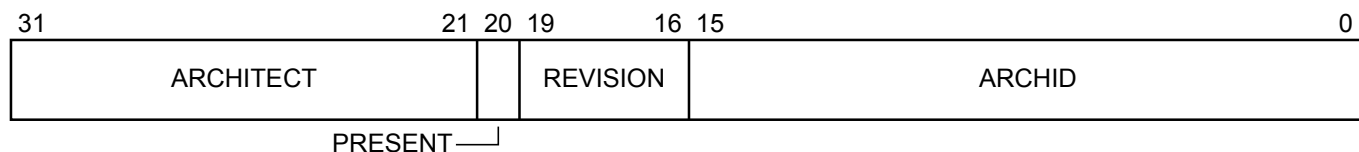
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.7.1 Field Descriptions

The DEVARCH bit assignments are:



ARCHITECT, bits[31:21]

0x23B The architect is Arm.

PRESENT, bit[20]

1 DEVARCH is present.

REVISION, bits[19:16]

0x0

ARCHID, bits[15:0]

0xA34 Power requester.

E1.2.7.2 Accessing DEVARCH

DEVARCH can be accessed at the following address:

Component	Offset
GPR	0xFBC

E1.2.8 DEVID, Device configuration Register

This section describes the bit assignments for GPR components that implement DEVID. For a full description of DEVID, see [DEVID, Device Configuration Register](#).

The DEVID characteristics are:

Purpose Indicates how many power domains the power requester supports.

Usage constraints DEVID is accessible as follows:

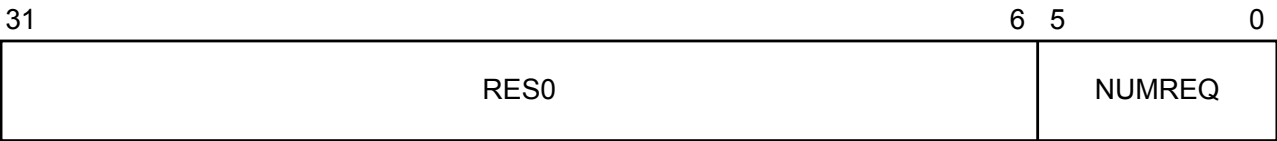
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.8.1 Field Descriptions

The DEVID bit assignments are:



Bits[31:6]

RES0

NUMREQ, bits[5:0]

IMPLEMENTATION DEFINED. Number of power domains to be managed by [CDBGPWRUPREQ](#) and [CDBGPWRUPACK](#).

E1.2.8.2 Accessing DEVID

DEVID can be accessed at the following address:

Component	Offset
GPR	0xFC8

E1.2.9 DEVTYPE, Device Type Register

This section describes the bit assignments for GPR components that implement DEVTYPE. For a full description of DEVTYPE, see [DEVTYPE, Device Type Identifier Register](#).

The DEVTYPE characteristics are:

Purpose If the part number field is not recognized, a debugger can report the information that is provided by DEVTYPE about the component instead.

Usage constraints

DEVTYPE is accessible as follows:

Default
RO

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.9.1 Field Descriptions

The DEVTYPE bit assignments are:

31	8	7	4	3	0
RES0		SUB		MAJOR	

Bits[31:8]

RES0. Ensures that the bits that are not associated with the component type have a well-defined value.

SUB, bits[7:4]

0x3 Power requester.

MAJOR, bits[3:0]

0x4 Debug Control.

E1.2.9.2 Accessing DEVTYPE

DEVTYPE can be accessed at the following address:

Component	Offset
GPR	0xFCC

E1.2.10 ITCTRL, Integration Mode Control Register

This section describes the bit assignments for GPR components that implement ITCTRL. For a full description of ITCTRL, see [ITCTRL, Integration Mode Control Register](#).

The ITCTRL characteristics are:

Purpose A component can use this register to dynamically switch between functional mode and integration mode.
In integration mode, topology detection is enabled. For more information, see [Topology Detection](#).

Usage constraints
After switching to integration mode and performing integration tests or topology detection, reset the system to ensure correct behavior of CoreSight and other connected system components.
ITCTRL is accessible as follows:

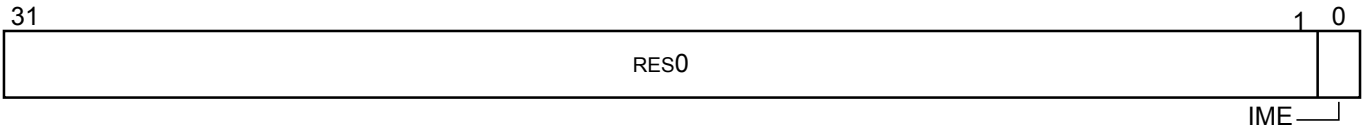
Default
RW

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.10.1 Field Descriptions

The ITCTRL bit assignments are:



Bits[31:1]
RES0.

IME, bits[0]
Permitted values of IME are:
0 The component must enter functional mode.
1 The component must enter integration mode, and enable support for topology detection and integration testing.
When no integration functionality is implemented, this field is RES0.

E1.2.10.2 Accessing ITCTRL Register

ITCTRL can be accessed at the following address:

Component	Offset
GPR	0xF00

E1.2.11 LAR, Lock Access Register

For a full description of this register, and how to deploy it in a Software lock mechanism, see [LSR and LAR, Software Lock Status Register and Software Lock Access Register](#).

The LAR characteristics are:

Purpose Components can use this register to enable write access to device registers.

Usage constraints LAR is accessible as follows:

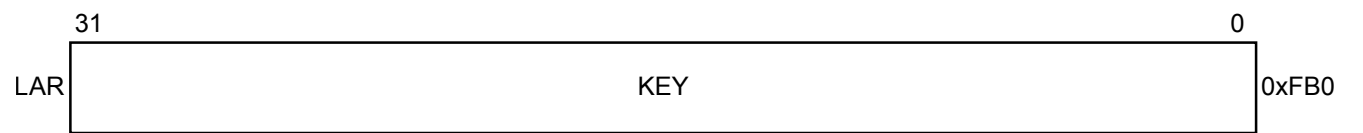
Default
WO

Configurations [LSR.SLI](#) indicates whether a Software lock mechanism is implemented. If a Software lock mechanism is implemented, LAR is implemented, and must be used with [LSR](#).

Attributes A 32-bit management register.

E1.2.11.1 Field Descriptions

The LAR bit assignments are:



KEY, bits[31:0]

Writing a value to this field controls write access to the control registers.

Permitted values of KEY are:

Write 0xC5ACCE55

Signals that LSR must permit writing to the control registers.

Write any other value

Signals that LSR must block writing to the control registers.

E1.2.11.2 Accessing LAR

LAR can be accessed at the following address:

Component	Offset
GPR	0xFB0

E1.2.12 LSR, Lock Status Register

For a full description of this register, and how to deploy it in a Software lock mechanism, see [LSR and LAR, Software Lock Status Register and Software Lock Access Register](#).

The LSR characteristics are:

Purpose Defines the parameters for a Software lock mechanism that can be implemented to control write access to device registers.

Usage constraints

LSR is accessible as follows:

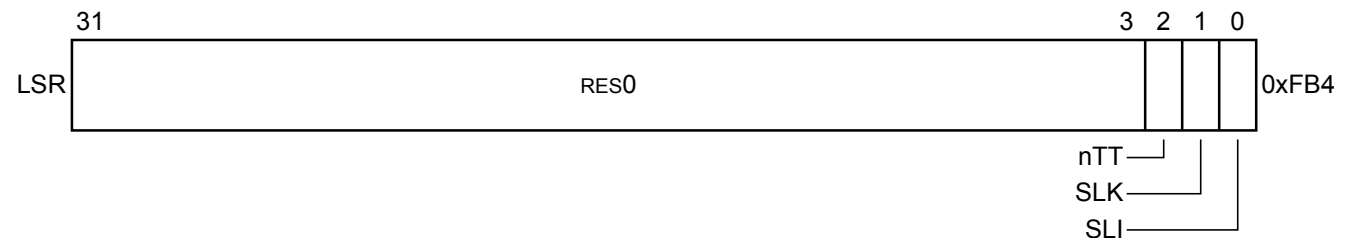
Default
RO

Configurations Included in all implementations.

Attributes A 32-bit management register.

E1.2.12.1 Field Descriptions

The LSR bit assignments are:



Bits[31:3]

RES0.

nTT, bits[2]

0 LAR is a 32-bit register.

SLK, bits[1]

This field is used to return the current software lock status.

Permitted values of SLK are:

- 0 Writing to the control registers must be permitted.
- 1 Writing to the control registers must be blocked.

SLI, bits[0]

This field indicates whether a Software lock mechanism is implemented.

Permitted values of SLI are:

- 0 Software lock mechanism is not implemented.
- 1 Software lock mechanism is implemented.

E1.2.12.2 Accessing LSR

LSR can be accessed at the following address:

Component	Offset
GPR	0xFB4

E1.2.13 PIDR0-PIDR7, Peripheral Identification Register

This section describes the bit assignments for GPR components that implement PIDR0-PIDR7. For a full description of the PIDR, see [PIDR0-PIDR7, Peripheral Identification Registers](#).

The PIDR characteristics are:

Purpose Provide information to identify a CoreSight component.

Usage constraints

PIDR0-PIDR7 are accessible as follows:

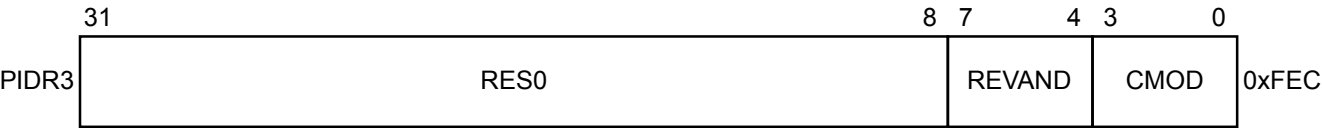
Default
IMPLEMENTATION DEFINED

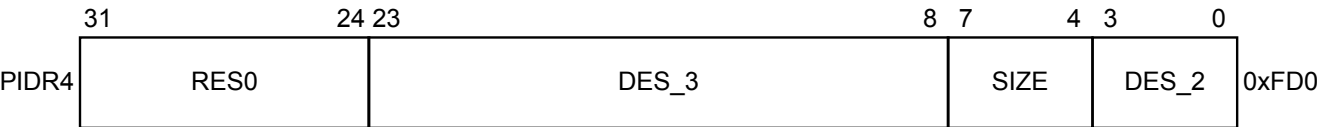
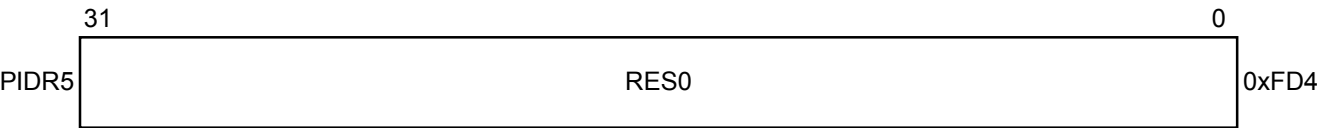
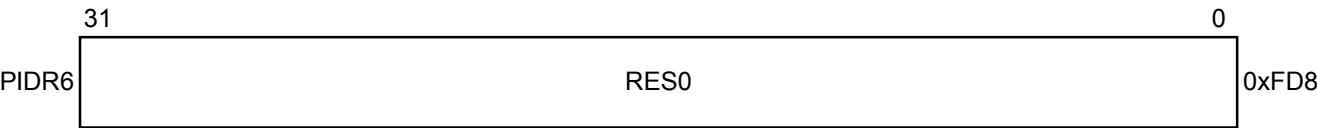
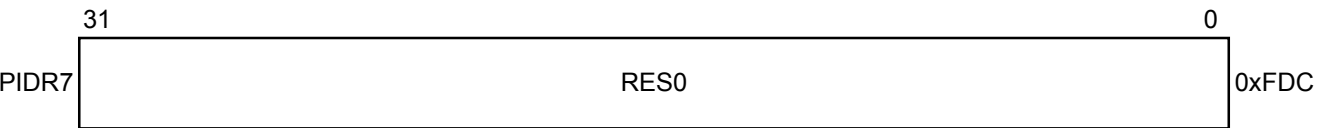
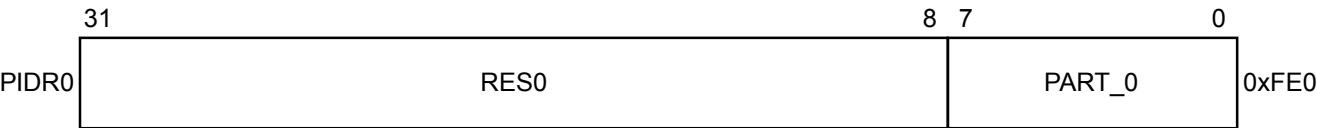
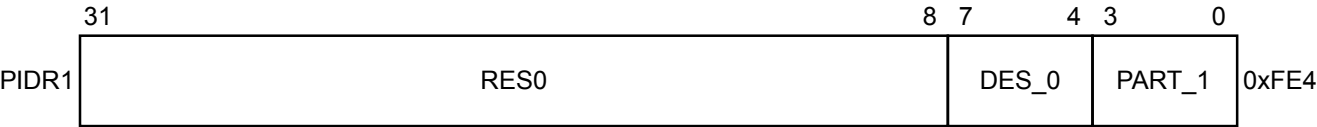
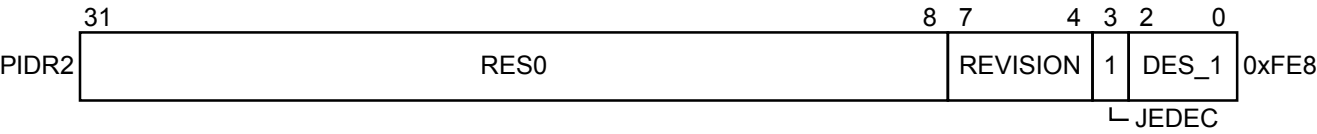
Configurations Included in all implementations.

Attributes Eight 32-bit management registers.

E1.2.13.1 Field Descriptions

The PIDR bit assignments are:





PIDR3 bits[31:8]

RES0.

REVAND, PIDR3 bits[7:4]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

CMOD, PIDR3 bits[3:0]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

PIDR2 bits[31:8]

RES0.

REVISION, PIDR2 bits[7:4]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

JEDEC, PIDR2 bits[3]

0b1.

DES_1, PIDR2 bits[2:0]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

PIDR1 bits[31:8]

RES0.

DES_0, PIDR1 bits[7:4]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

PART_1, PIDR1 bits[3:0]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

PIDR0 bits[31:8]

RES0.

PART_0, PIDR0 bits[7:0]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

PIDR7 bits[31:0]

RES0.

PIDR6 bits[31:0]

RES0.

PIDR5 bits[31:0]

RES0.

PIDR4 bits[31:8]

RES0.

DES_3, PIDR4 bits[23:8]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

SIZE, PIDR4 bits[7:4]

0x0 The GPR uses a single 4KB memory block.

DES_2, PIDR4 bits[3:0]

See register descriptions in [PIDR0-PIDR7, Peripheral Identification Registers](#).

E1.2.13.2 Accessing the PIDR

PIDR0-PIDR7 can be accessed at the following address:

Component	Offset							
	PIDR0	PIDR1	PIDR2	PIDR3	PIDR4	PIDR5	PIDR6	PIDR7
GPR	0xFE0	0xFE4	0xFE8	0xFEC	0xFD0	0xFD4	0xFD8	0xFDC

E1.3 Powering non-visible components

Some components do not have a visible programmers' model, for example a *Cross Trigger Matrix* (CTM), which is used in cross-triggering components in a CoreSight system. When requesting power for a visible component, power must be supplied to any associated non-visible components as well.

For example, if two *Cross Trigger Interfaces* (CTIs) are connected through a CTM, a response to a power request for the CTIs must also power the CTM.

Chapter E2

Revisions

This appendix describes the main technical changes between released versions of this specification.

Table E2-1 Differences between v1.0 and v2.0

Change	Location
Renamed register fields for consistency across Arm documentation.	Entire document.
Clarified that all registers are accessed in little-endian format.	About the programmers' model.
Added new registers to Class 0x9 CoreSight component.	DEVID1, Device Configuration Register 1. DEVID2, Device Configuration Register 2. DEVARCH, Device Architecture Register. DEVAFF0 - DEVAFF1, Device Affinity Registers.
Added new interfaces.	Event Interface. Timestamp Interface.
Updated channel interface.	Channel interface.
Updated the definition of the authentication interface to deprecate some previously permitted encodings.	Authentication Interface.
Updated the connector information.	Physical Interface.
Updated the permitted trace ID values to include 0x7D.	Special trace source IDs.
Added the power requester and ROM Table values.	Power Requester. ROM Tables Overview.

Table E2-2 Differences between v2.0 and v3.0

Change	Location
The use of LAR and LSR to implement the Software lock mechanism is deprecated.	LSR and LAR, Software Lock Status Register and Software Lock Access Register.
The use of PADDRDBG[31] to split the memory map and indicate the difference between external and internal accesses is deprecated.	Debug APB interface memory map
Use of the PIDR4.SIZE field is deprecated.	Components that occupy more than 4KB of address space and PIDR0-PIDR7, Peripheral Identification Registers.
The AUTHSTATUS description is extended to include optional fields that can be used to indicate hypervisor debug visibility.	AUTHSTATUS, Authentication Status Register.
The Authentication Interface is extended to support signals that control debug for a hypervisor.	Authentication Interface.
The rules for assigning a Unique Component Identifier and a revision number to a component have been updated.	Component and Peripheral Identification Registers.
Trace Formatter ID 0x7B, which was reserved in earlier versions, is allocated to indicate a flush response.	Special trace source IDs.
Introduced Class 0x9 ROM Tables, and adopted the designation Class 0x1 ROM Tables for the existing format.	About ROM Tables. <i>See Arm™ Debug Interface Architecture Specification (ADIV6.0).</i>
Support for Realm Management Extension is added	Component-specific registers for Class 0x9 CoreSight components. Authentication Interface.
Support in AUTHSTATUS for indicating Unprivileged debug	AUTHSTATUS, Authentication Status Register.

Table E2-3 Differences in issue G

Change	Location
Support for RME_GDI	Authentication Interface.
Support for Anonymous ROM tables	About ROM Tables.
Improved clarity to part number and revision information	PIDR0-PIDR7, Peripheral Identification Registers.
Support for an extended format for JEP106 designer IDs	Forms of the designer identification codes. PIDR0-PIDR7, Peripheral Identification Registers. DEVARCH, Device Architecture Register.
Improved clarity to compliance rules in relation to ADIV6	Compliance Requirements.
VSupply pin description update	VSupply output.